CAPÍTULO 1

COMPUTADORA DIGITAL

Una computadora digital es una combinación de dispositivos y circuitos electrónicos organizados de tal forma, que pueden realizar una **secuencia programada** de operaciones con un mínimo de intervención humana. A la secuencia de operaciones se le denomina **Programa**. Un programa es un conjunto de instrucciones codificadas que se almacenan en la memoria interna de la computadora junto con todos los datos que el programa requiere. Por otra parte, para que la computadora pueda ser útil, es necesario que ésta interactúe con el exterior pidiendo datos para ser procesados, y muestre de alguna forma los resultados que obtiene.

Modelo De Von Neumann

La mayoría de las computadoras digitales basan su arquitectura en el **modelo de Von Neumann**. En este modelo, una computadora está formada por cuatro unidades que interactúan entre sí. Estas cuatro unidades son: El procesador, La Unidad de Memoria, la Unidad de Entrada-Salida y La Unidad de Control, ver figura 1-1.

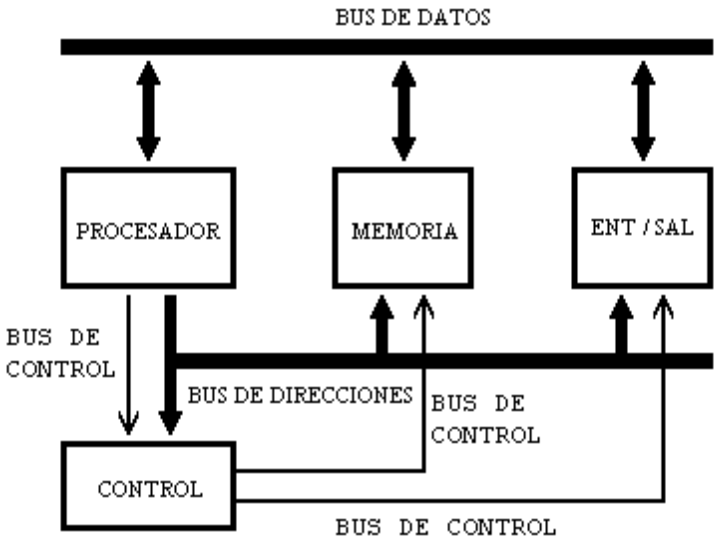


Figura 1-1. Modelo de Von Neumann

La interacción de estas cuatro unidades se lleva a cabo a través de tres conjuntos de conectores llamados conductos o buses: El bus de datos, el bus de direcciones y el bus de control.

El procesador

El procesador es el dispositivo que ejecuta las instrucciones almacenadas en la memoria, el programa. Para lograr esto debe ser capaz de leer y escribir en la memoria y en la unidad de entrada-salida.

Unidad de memoria

Es un conjunto o arreglo de registros binarios de longitud definida (8, 16, 32 o 64 bits), en los cuales se puede almacenar instrucciones codificadas y datos. Cada uno de estos registros se caracteriza por tener una dirección única, por lo que generalmente cada una de estas localidades se identifica por su dirección la cual es la posición que le toca a ese registro dentro del arreglo. Las direcciones se expresan siempre en hexadecimal.

Unidad de entrada-salida

Una unidad de entrada-salida, llamada también puerto, es un dispositivo empleado por la computadora para recibir o enviar datos al mundo exterior. Algunos de estos dispositivos son: Teclado, ratón, interruptores, impresora, monitor, discos magnéticos, tarjetas de comunicación, etc.

Estos dispositivos al igual que las localidades de memoria tienen direcciones únicas por medio de las cuales el procesador interactúa con ellos. Estas direcciones se expresan generalmente en hexadecimal.

Unidad de control

La función de la Unidad de control es coordinar la operación de todas las otras unidades ofreciendo señales de temporización para mantener el sistema en sincronización adecuada y de control para seleccionar si se desea leer o escribir a una localidad de memoria o a un dispositivo de entrada-salida.

Buses de datos, direcciones y control

El procesador utiliza el **bus de datos** para mover datos entre el procesador y la memoria, entre el procesador y los dispositivos de entrada-salida (puertos) o entre la memoria y los dispositivos de entrada-salida. El procesador utiliza el **bus de direcciones**, para indicar la dirección de memoria o puerto sobre la que quiere leer o escribir. Por último el **bus de control** se utiliza para habilitar el dispositivo sobre el que se va a leer o escribir e indicar cual de estas operaciones se va a realizar.

El Procesador

En la figura 1-2 se muestra un modelo simplificado de un procesador o **CPU** (del inglés Central Processing Unit).

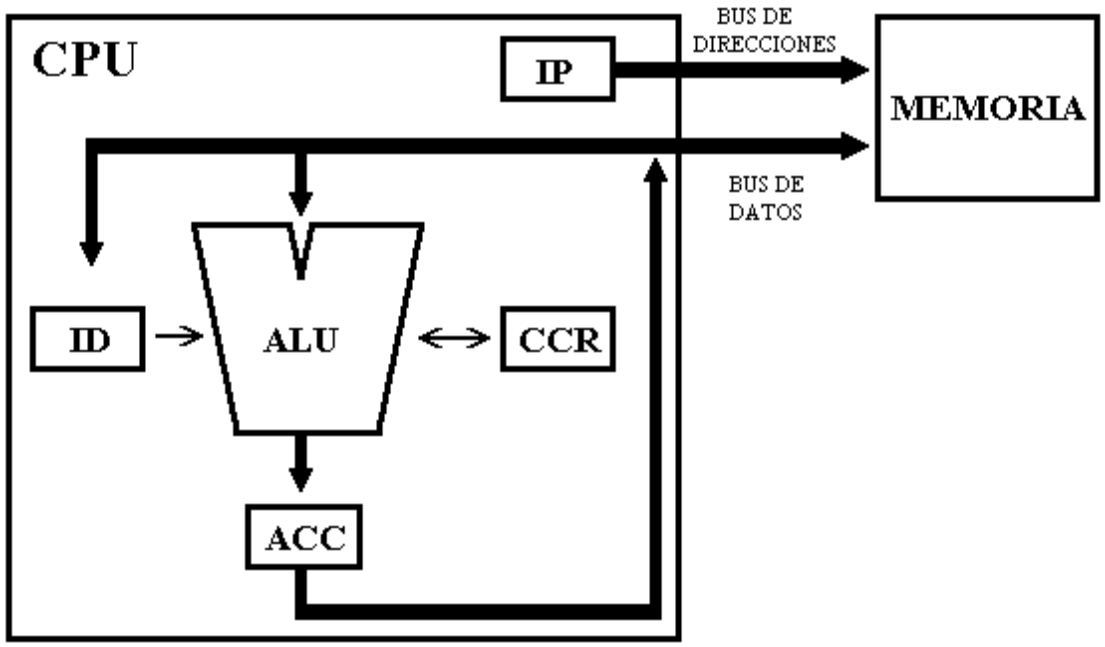


Figura 1-2. Modelo simplificado de un procesador

Generalmente las instrucciones que forman un programa se encuentran almacenadas en forma consecutiva a lo largo de la memoria. Para ejecutar un programa el procesador sólo necesita realizar cíclicamente los siguientes pasos:

1. Lee de la dirección de memoria indicada por el apuntador de instrucciones, IP, el byte de instrucción y la envía al decodificador de instrucciones, ID.

   El **apuntador de instrucciones**, es un registro interno del procesador, el cual contiene la dirección de memoria donde se encuentra la próxima instrucción a ser ejecutada. Al inicio, el apuntador de instrucciones contiene la dirección de la primera instrucción del programa.

2. Actualiza el IP para que apunte al siguiente byte.

3. Identifica o **decodifica** la instrucción mediante la circuitería del ID.
4. Si se requiere, lee el valor del operando de la dirección de memoria indicada por el IP.

5. Si se requiere, Actualiza el IP para que apunte al siguiente byte.

6. Ejecuta la instrucción. El ID genera las señales necesaria para que:

- La unidad aritmética y lógica, ALU, ejecute las operaciones indicadas por la instrucción

- Se almacene el resultado en el acumulador, ACC. El acumulador es un registro utilizado para almacenar el dato sobre los que se va a hacer una operación y el resultado de la operación.

- Se almacene en el registro de código de condiciones, CCR, el estado de la operación realizada. Uno de los bits de ese registro indica si el resultado de la operación fue cero, otro si fue negativo, otro si generó un acarreo, etc. Algunas instrucciones utilizan uno o más de los valores que una instrucción anterior dejó en este registro como parte de sus operandos.

Instrucciones Del Procesador

Cada procesador tiene un conjunto de instrucciones que puede realizar. Por medio de éstas se especifica el programa que queremos que ejecute el procesador. Este conjunto de instrucciones se le conoce como **juego de instrucciones**.

Las mayoría de las instrucciones están formadas por dos partes: la **operación** y los **operandos**. La operación es la acción que debe realizarse. Los operandos son los elementos sobre los que se realizará ésta acción. Por lo tanto, dependiendo de su tipo, las instrucciones pueden tener cero uno o más operandos. Sin embargo en todas las instrucciones existe la operación.

Cada operación está codificada mediante una secuencia binaria única llamada **código de la operación**. Como los operandos son también secuencias binarias, cada instrucción se codifica mediante una secuencia de bytes: uno o más para el código de la operación y cero, uno o más para los operandos. Un programa donde sus instrucciones están codificadas como secuencias binarias está en **lenguaje máquina**.

Para ilustrar lo anterior, supongamos un microprocesador básico en el que los códigos de operación sean de un byte y los operandos sean también de un byte. Supongamos también que el juego de instrucciones que tiene nuestro procesador básico contiene entre otras, las instrucciones mostradas en la tabla 1.1:

Tabla 1.1 Conjunto de instrucciones de un procesador básico.

Instrucción	Descripción	Código De la Operación	Instrucción Codificada
ACC ← *n*	Cargar en el acumulador del procesador un número *n*.	01	01 n
ACC ← [*n*]	Cargar en el acumulador del procesador el contenido de la dirección *n*.	02	02 n
[*n*] ← ACC	Almacenar el contenido del acumulador en la dirección *n*.	03	03 n
ACC ← ACC + *n*	Sumarle al acumulador el número *n* y guardar el resultado en el acumulador.	04	04 n
ACC ← ACC + [*n*]	Sumarle al acumulador el contenido de la dirección *n* y guardar el resultado en el acumulador.	05	05 n

Lenguajes Máquina, Ensamblador Y De Alto Nivel

A grandes rasgos, la evolución de la programación desde la programación en el lenguaje máquina, lenguaje de bajo nivel o ensamblador y los lenguajes de alto nivel es la siguiente:

- Las primeras computadoras eran programadas alimentando manualmente las secuencias binarias que constituían las instrucciones del programa mediante una batería de interruptores. Obviamente programación de esas primeras computadoras era una tarea muy tediosa y sujeta a errores. Posteriormente se substituyó la batería de interruptores por teclados hexadecimales que permitían que el programador cargara una secuencia de cuatro bits presionando una sola tecla del teclado hexadecimal.

- El primer avance significativo en la programación fue la invención del **lenguaje ensamblador**. En el lenguaje ensamblador las operaciones del procesador son codificadas mediante secuencias de caracteres en lugar de secuencias binarias. Estas palabras llamadas **mnemónicos**, son por lo general el nombre de la operación o una abreviatura de éste. En la tabla 1.2 se ilustran algunos mnemónicos de algunas de las instrucciones.

  Adicionalmente, en el lenguaje ensamblador, a las localidades de memoria en las que se almacenan datos se les pueden asignarse nombres simbólicos de modo que esas localidades pueden referenciarse usando sus nombres en lugar de sus direcciones.

Tabla 1.2 Mnemónicos de las instrucciones

Instrucción	Mnemónico	Instrucción en ensamblador
ACC ← *n*	mov	mov ax, n
ACC ← [*n*]	mov	mov ax, [dato]
[*n*] ← ACC	mov	mov [resultado], ax
ACC ← A + *n*	add	add ax, n
ACC ← A + [*n*]	add	add ax, [dato]

Un programa escrito en lenguaje ensamblador es traducido al lenguaje máquina de la computadora donde se va a ejecutar mediante un programa traductor llamado **ensamblador**. Como las instrucciones del lenguaje ensamblador son las mismas que el procesador puede ejecutar, cada procesador tiene su lenguaje ensamblador el cual es diferente del ensamblador de otro procesador.

- El siguiente avance en la programación consistió en la invención de los lenguajes de alto nivel. La principal ventaja de los lenguajes de alto nivel es que las instrucciones en esos lenguajes, se escriben en una forma más entendible por las personas ya que se parecen al lenguaje que empleamos en nuestra vida diaria. Aquí, el programa que traduce el programa escrito en lenguaje de alto nivel a un programa en lenguaje máquina se conoce como **compilador**. Otra ventaja de los lenguajes de alto nivel es que los programas escritos en éstos son hasta cierto punto independientes del procesador en que se van a ejecutar los programas. Para transportar un programa de un procesador a otro, casi siempre es suficiente con compilarlo con un compilador que genere código máquina para el nuevo procesador.

  Las desventajas de los lenguajes de alto nivel con respecto al lenguaje ensamblador son: En primer lugar, la mayoría de los lenguajes de alto nivel limitan al programador en cuanto a las tareas que éste puede hacer y en segundo lugar un programa producido usando un lenguaje de alto nivel es de mayor tamaño y más lento de uno equivalente escrito en lenguaje ensamblador.

Los siguientes dos ejemplos ilustran cómo ocurre la conversión entre las instrucciones de un programa de alto nivel y sus correspondientes instrucciones en lenguaje máquina:

1. Considere la siguiente proposición en el lenguaje C:

   ```
   c = a + b;
   ```

   Si suponemos que las variables a, b, c están en las localidades cuyas direcciones son 25, 33 y 37, respectivamente, entonces la proposición anterior correspondería al siguiente conjunto de instrucciones de nuestro procesador básico:

Instrucciones	Codificación
ACC ← [25]	02
	25
ACC ← ACC + [33]	05
	33
[37] ← ACC	03
	37

2. Considere la siguiente proposición en el lenguaje C:

   z = x + 13 + y

   Si suponemos que las variables x, y, z están en las localidades cuyas direcciones son 23, 44 y 17, respectivamente, entonces la proposición anterior correspondería al siguiente conjunto de instrucciones de nuestro procesador básico:

Instrucciones	Codificación
ACC ← [23]	02
	23
ACC ← ACC + 13	04
	13
ACC ← ACC + [44]	05
	44
[17] ← ACC	03
	17

Problemas

Para cada una de las siguientes proposiciones en el lenguaje C, escriba el correspondiente conjunto de instrucciones que deberá ejecutar el procesador básico y codifíquelas a lenguaje máquina. Añada las instrucciones que sean necesarias.

1. z = x + 13 * y
2. z = (x + 13) * y

Bibliografía

1. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

2. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documentos PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

3. Mano, M. Morris. Lógica Digital y Diseño de Computadores. Prentice-Hall Hispanoamericana, S. A. México. 1982.

4. Mano, M. Morris. Ingeniería Computacional: Diseño del Hardware. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Morgan, Christhopher L, y Waite Mitchel. Introducción al microprocesador 8086/8088 (16 bits). McGraw-Hill/Interamericana de México, S.A de C. V. México. 1988.

CAPÍTULO 2

EL MICROPROCESADOR 8086

Uno de los propósitos de los lenguajes de alto nivel es de ocultarle al programador los detalles del hardware de la computadora en la que se ejecutarán los programas que éste escribe. Sin embargo al programar en un lenguaje de bajo nivel o ensamblador se debe tener un conocimiento mayor del hardware. Dos de los aspectos que debe conocer el programador de ensamblador sobre la computadora en que se va a ejecutar sus programas es la organización de la memoria y el modelo de programación de su microprocesador.

Organización De La Memoria

Podemos visualizar a la memoria principal de una computadora como una colección de localidades de memoria cada una de un byte. Cada localidad de memoria tiene su propia dirección. Dado que cada dirección es un número entero, hay una relación directa entre el número de localidades de memoria y el tamaño que debe tener una dirección en bits. Una dirección de N bits nos permite direccionar 2^N direcciones diferentes.

En la memoria principal de una computadora se almacenan tanto los programas como los datos con que operan esos programas. Al estar ejecutando un programa, se requiere calcular las direcciones en las que están almacenadas las instrucciones y datos. El cálculo de las direcciones de memoria depende de cómo el procesador visualiza u organiza, la memoria. Dos de las formas en las que el microprocesador visualiza u organiza la memoria en una computadora son las llamadas memoria lineal y memoria segmentada.

Memoria Lineal

El esquema de memoria lineal es el más simple de todos ya que bajo éste el microprocesador ve a la memoria como un arreglo lineal de localidades de memoria de un byte y en el que las direcciones de las localidades de memoria son los índices de los elementos del arreglo.

El esquema de memoria lineal es usado por lo general en microprocesadores con capacidad de direccionamiento reducido, por lo general aquellos en los que la memoria no excede a los 64 KB.

Memoria Segmentada

En el esquema de memoria segmentada el microprocesador visualiza a la memoria como dividida en segmentos de memoria y a cada segmento lo visualiza como un arreglo lineal de localidades de memoria de un byte. Cada segmento tiene una dirección llamada **dirección de segmento**. La dirección de una localidad de memoria se forma a partir de la dirección del segmento que contiene a la localidad de memoria y al índice de la localidad de memoria dentro del segmento. A este índice se le conoce como **desplazamiento**.

El esquema de memoria segmentada es usado por lo general en microprocesadores con capacidad de direccionamiento que excede a los 64 KB. Uno de estos procesadores es el procesador 8086 de Intel.

Segmentación De La Memoria En El Microprocesador 8086

El CPU empleado en las primeras computadoras IBM PC y compatibles, el microprocesador 8086 de Intel, nos permite direccionar 1 MB = 2^{20} = 1,048,576 bytes. Los procesadores más modernos de Intel de la familia 80x86 (80286, 80386, 80486 y Pentium) empleados en las computadoras PC pueden direccionar un mayor número de localidades (hasta 16 MB = 2^{24} = 16,777,216 bytes para el 80286 y hasta 4 GB = 2^{32} = 4,294,967,296 bytes para el 80386, 80486 y Pentium) pero para mantener compatibilidad con el microprocesador 8086 tienen dos formas de operación: Una que imita al microprocesador 8086 llamado **modo real** y otro que explota todo el poder del procesador llamado **modo protegido**. El sistema operativo MSDOS a fin de mantener compatibilidad con el software existente, sólo permite la operación de los microprocesadores en modo real y para fines prácticos una computadora PC bajo ese sistema operativo se comportará como si tuviese un procesador 8086 (aunque más rápido).

En el microprocesador 8086 (o en los microprocesadores más modernos de la familia 80x86 operando en el modo real) visualiza la memoria como dividida en segmentos los cuales tienen las siguientes características:

- Los segmentos no son divisiones físicas. Un segmento es una ventana lógica a través de la cual el programa visualiza porciones de la memoria.

- Los segmentos empiezan cada 16 bytes. Como a un bloque de memoria de 16 bytes se le conoce como **párrafo**, los segmentos empiezan a cada párrafo.

- Hay hasta 65,536 segmentos (1,048,576/16 = 65,536).

- El primer segmento es el segmento 0, el siguiente es el segmento 1, etc. El número del segmento se conoce como la **dirección del segmento**.

- La **dirección real**, también llamada **dirección efectiva**, en la que empieza un segmento se obtiene multiplicando la dirección del segmento por 16.
- Los segmentos pueden ser tan pequeños como 16 bytes y tan grandes como 64 KB = 65,536 bytes.

- Los segmentos pueden traslaparse. Esto ocurre cuando un segmento es mayor a 16 bytes, tal como se muestra en la figura 2-1.

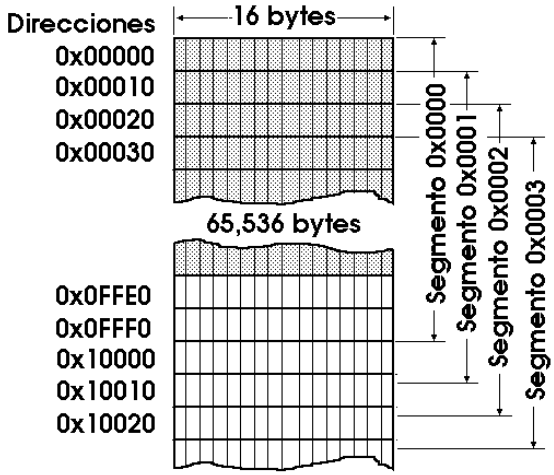


Figura 2-1. Mapa de la memoria segmentada

- Al cargar un programa a memoria, éste ocupa uno o más segmentos. Los segmentos asignados a un programa no tienen que estar uno junto al otro aunque frecuentemente lo están.

Se acostumbra expresar la dirección de una localidad de memoria mediante la combinación formada por la dirección del segmento en que se encuentra la localidad de memoria y el desplazamiento de la localidad con respecto al principio del segmento.

Segmento:Desplazamiento

Ambos, la dirección de segmento y el desplazamiento son valores de 16 bits. Estos dos valores se combinan para formar la dirección real o efectiva de la localidad de la siguiente manera:

Dirección real = segmento << 4 + desplazamiento

Debido al traslape entre los segmentos muchas combinaciones segmento:desplazamiento nos pueden dar la misma dirección real, por ejemplo la localidad de memoria cuya dirección real es 0x01000 puede expresarse mediante las siguientes combinaciones segmento:desplazamiento

```
Segmento:Desplazamiento      Dirección real
-----------------------------------------------------------
     0x0000:0x1000       0x00000 + 0x1000 = 0x01000
     0x0001:0x0FF0       0x00010 + 0x0FF0 = 0x01000
     0x0002:0x0FE0       0x00020 + 0x0FE0 = 0x01000
     ...
     0x0010:0x0F00       0x00100 + 0x0F00 = 0x01000
     ...
     0x0020:0x0E00       0x00200 + 0x0E00 = 0x01000
     ...
     0x00FF:0x0010       0x00FF0 + 0x0010 = 0x01000
     0x0100:0x0000       0x01000 + 0x0000 = 0x01000
```

Cuando el microprocesador quiere calcular una dirección real, carga en un registro el valor del segmento y en otro registro el valor del desplazamiento y realiza las operaciones de corrimiento y suma.

Modelo De Programación Del Microprocesador 8086

Los registros de un microprocesador son componentes dentro del microprocesador que nos permiten almacenar datos. Estos datos pueden representar valores sobre los cuales se van a realizar operaciones, resultados de las operaciones, direcciones de localidades de memoria donde se encuentran datos e instrucciones, direcciones de los dispositivos de entrada/salida sobre los que deseamos escribir o leer, o los datos a escribir o leídos de esos dispositivos.

El número, tamaño y uso de los registros de un microprocesador así como su conjunto de instrucciones determina la eficiencia con que el microprocesador realiza una tarea. A la descripción del número, tamaño y uso de los registros de un microprocesador se le conoce como el **modelo de programación del microprocesador**.

En la figura 2-2 se ilustra el arreglo de registros internos del microprocesador 8086. Los microprocesadores más poderosos de la familia del 8086 tienen otros registros adicionales pero no forman parte del estudio de este curso. Los registros del 8086 pueden clasificarse en tres grupos: registros de uso general, registros apuntadores y de índice y registros de segmentos. Adicionalmente tienen un registro de banderas que señala las condiciones respecto al funcionamiento de la unidad aritmética y lógica (ALU).



Figura 2-2

Registros De Propósito General

Los registros de propósito general, como su nombre lo indica, se utilizan en la forma en que lo desee el programador. Cada uno de estos registros se puede direccionar como un registro de 16 bits (AX, BX, CX, DX) o como un registro de 8 bits (AH, AL, BH, BL, CH, CL, DH, DL). Cada uno de los registros de 16 bits está formado por la concatenación de dos registros de 8 bits: AX = AH:AL, BX = BH:BL, CX = CH:CL y DX = DH:DL, donde el bit 0 del registro AH es el bit 8 del registro AX, etc.

Adicionalmente, algunas de las instrucciones del microprocesador emplean los registros de propósito general para tareas específicas. Por esta razón, a cada uno se le da su nombre (Acumulador, Base, Contador y Datos). Las funciones primarias de los registros de propósito general incluyen:

AX, AH, AL (Acumulador): a menudo conserva el resultado temporal después de una operación aritmética o lógica.

BX, BH, BL (Base): Se utiliza para guardar la dirección base de listas de datos en la memoria.

CX, CH, CL (Contador): Contiene el conteo para ciertas instrucciones de corrimientos y rotaciones, de iteraciones en el ciclo **loop** y operaciones repetidas de cadenas.

DX, DH, DL (Datos): Contiene la parte más significativa de un producto después de una multiplicación; la parte más significativa del dividendo antes de la división.

Registros Apuntadores Y De Índices

Aunque los registros apuntadores y de índices (a excepción del registro apuntador de instrucciones) son también de uso general, se utilizan principalmente para formar la dirección efectiva o real de una localidad de memoria. Los registros apuntadores y de índice contienen el desplazamiento con respecto a un segmento de un dato o una instrucción. Las funciones primarias de los registros apuntadores y de índice incluyen:

SP (Apuntador de pila): Contiene el desplazamiento con respecto al segmento de pila del tope de la pila del programa. El concepto de pila del programa se estudiará en el Tema VII: Procedimientos. El registro **SP** se emplea para direccionar datos en la pila cuando se ejecutan las instrucciones **push** y **pop** y cuando se llama o se regresa de un procedimiento mediante las instrucciones: **call** y **ret**, respectivamente.

BP (Apuntador de base): Contiene el desplazamiento con respecto al segmento de pila de datos almacenados en la pila de un programa.

SI (Índice fuente): Contiene el desplazamiento con respecto al segmento de datos de un elemento de un arreglo o cadena. Los arreglos y cadenas se estudiarán en el Tema VIII: Arreglos y cadenas.

DI (Índice destino): Contiene el desplazamiento con respecto al segmento extra de un elemento de una cadena o arreglo.

IP (Apuntador de instrucciones): Contiene siempre el desplazamiento con respecto al segmento de código de la localidad de memoria que contiene la siguiente instrucción que va a ejecutar el microprocesador.

Registros De Segmentos

Los registros de segmentos se utilizan para formar la dirección efectiva o real de una localidad de memoria. Los registros de segmentos contienen la dirección de segmento de un dato o una instrucción. El código y los datos de un programa en ejecución se encuentran cada uno en uno o más segmentos de memoria. La pila del programa ocupa otro segmento. Los registros de segmento son:

CS (Código): Tiene la dirección lógica del segmento en que se encuentra el código de un programa. Si el código ocupa más de un segmento, contiene la dirección lógica de uno de los segmentos.

DS (Datos): Tiene la dirección lógica del segmento en que se encuentran los datos estáticos de un programa. Si los datos ocupan más de un segmento, contiene la dirección lógica de uno de los segmentos.

ES (Extra): Este registro también tiene la dirección lógica de uno de los segmentos en que se encuentran los datos estáticos de un programa. Este registro se utiliza en ciertas operaciones del microprocesador para el manejo de cadenas.

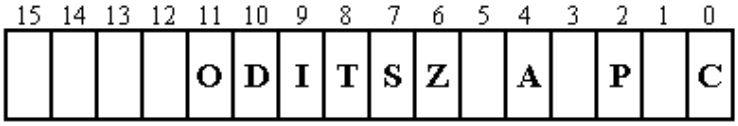
SS (Pila): Tiene la dirección lógica del segmento en que se encuentran la pila del sistema. La pila no puede ser mayor a un segmento.

Para calcular una dirección efectiva o real de un dato o una instrucción, el microprocesador toma de uno de los registros de segmento el valor del segmento y de otro registro el valor del desplazamiento:

- En el cálculo de la dirección de una instrucción, el valor del segmento se encuentra en el registro de segmento de código, **CS**, y el desplazamiento en el registro apuntador de instrucciones, **IP**.
- En el cálculo de la dirección de un dato, el valor del segmento se encuentra en el registro de segmento de datos, **DS**, y el desplazamiento puede estar en los registros **BX**, **DI**, **SI**, o ser un número de 16 bits.
- En algunas instrucciones para cadenas, también se emplea el registro de segmento extra, **ES**, y el registro **DI** para el cálculo de la dirección de los datos. **ES** contiene, el valor del segmento y **DI** el valor del desplazamiento.
- En el cálculo de la dirección de los datos almacenados en la pila, el valor del segmento se encuentra en el registro de segmento de pila, **SS**, y el desplazamiento en el registro apuntador de pila, **SP** o en el registro apuntador de base, **BP**.

El Registro De Banderas

Cada bandera es un bit en el registro de banderas, también llamado registro de código de condiciones. En la figura 2-3 se ilustra el registro de banderas del microprocesador 8086. Aunque el registro de banderas es de 16 bits sólo hay nueve banderas: **O**, **D**, **I**, **T**, **S**, **Z**, **A**, **P**, **C**. Los otros 7 bits no son usados.



Registro de banderas del microprocesador 8086

Figura 2-3

De estas nueve banderas, seis de ellas cambian después de ejecutar muchas de las instrucciones aritméticas y lógicas. Estas seis banderas son:

C (Acarreo): Indica un acarreo después de una suma o un préstamo después de una resta. La bandera de acarreo también indica condiciones de error en ciertos programas y procedimientos.

P (Paridad): Es un cero para una paridad impar y un 1 para una paridad par. La paridad es un conteo de unos expresada como un número par e impar.

A (Acarreo auxiliar): Indica un acarreo después de una suma o un préstamo después de una resta del bit 3 al bit 4 en el resultado. Esta bandera sólo se utiliza en las operaciones **daa** y **das** para ajustar el valor de AL después de una suma o resta BCD.

Z (Cero): Indica que el resultado de una operación aritmética o lógica es cero. Si **Z** = 1, el resultado es cero y si **Z** = 0, el resultado no es cero.

S (Signo): Indica el signo aritmético del resultado después de una suma o una resta. Si **S** = 1, el resultado es negativo. Si **S** = 0, el resultado es positivo. Se debe tener en cuenta que el valor del bit más significativo aparece en el bit de signo en cualquier instrucción que afecte las banderas.

O (Sobreflujo): Es una condición que ocurre cuando se suman o se restan números con signo. Un sobreflujo indica que el resultado ha excedido la capacidad de la máquina. Por ejemplo si se suma 7FH + 01H (127 + 1) el resultado es 80H (-128). Este resultado representa una situación de sobreflujo señalado por la bandera para la suma con signo. Para operaciones sin signo esta bandera se ignora.

Las otras tres banderas se utilizan para controlar ciertas características del microprocesador. Esas tres banderas y su uso son:

T (Trampa): Activa, si se pone a 1, o desactiva, si se pone a 0, el modo de ejecución paso a paso. Este modo es utilizado por los depuradores para ejecutar las instrucciones una a la vez y permitir observar el efecto de la instrucción sobre los registros y la memoria.

I (Interrupción): Habilita, si se pone a 1, o deshabilita, si se pone a 0, las interrupciones al microprocesador. El estado de esta bandera se controla con las instrucciones **sti** (habilitar interrupciones) y **cli** (desactivar las interrupciones).

D (Dirección): Controla la selección de autoincremento o autodecremento de los registros **DI** o **SI** durante las instrucciones de cadenas y arreglos. Si **D** = 1 hay autodecremento en los registros y si **D** = 0 hay autoincremento. El estado de esta bandera se controla con las instrucciones **std** (habilitar dirección) y **cld** (desactivar dirección).

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

3. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

4. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documentos PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

5. Morgan, Christhopher L, y Waite Mitchel. Introducción al microprocesador 8086/8088 (16 bit). McGraw-Hill/Interamericana de México, S.A de C. V. México. 1988.

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

CAPÍTULO 3

ESTRUCTURA DE UN PROGRAMA EN ENSAMBLADOR

Programas En Ensamblador

El **lenguaje ensamblador** es un lenguaje que utiliza símbolos en lugar de instrucciones codificadas en lenguaje máquina. También permite hacer referencia a las localidades de memoria por nombres simbólicos en lugar de usar sus direcciones directamente. Un programa escrito en lenguaje ensamblador, llamado código fuente, es traducido al código máquina de un microprocesador mediante un programa llamado **ensamblador**.

Un programa en ensamblador consiste en una secuencia de caracteres almacenada en uno o más archivos que serán traducidos por el ensamblador al lenguaje máquina de una determinada computadora.

En este curso se estudiará el lenguaje ensamblador del procesador 8086 y se utilizará el ensamblador Turbo Assembler de la compañía Borland Int.

Palabras Reservadas

Las palabras reservadas son para uso estricto del ensamblador. Éstas incluyen a los operadores, directivas y símbolos predefinidos. Estas palabras no pueden usarse como identificadores. La tabla 3.1 muestra la lista de palabras reservadas de Turbo Assembler:

Tabla 3.1 Palabras reservadas de Turbo Assembler.

:	;	=	?	[]	/
()	+	-	*	.	.186
.286	.286c	.286p	.287	.386	.386c
.387	.8086	.8087	align	.alpha	and
arg	assume	%bin.	byte	catstr	@code
codeseg	@codesize	comm	comment	%conds	.const
const	@cpu	%cref	.cref	%crefall	%crefref

Tabla 3.1 Palabras reservadas de Turbo Assembler (Cont.)

%crefuref	%ctls	@curseg	@data	.data	.data?
dataptr	dataseg	@datasize	??date	db	dd
%depth	df	display	dosseg	dp	dq
dt	dup	dw	dword	else	elseif
Emul	End	Endif	Endm	Endp	Ends
Eq	Equ	Err	.err	.err1	.err2
.errb	.errdef	Errdif	errdifi	erre	erridn
erridni	errifnb	Errifndef	errnb	errndef	errnz
even	evendata	Exitm	extrn	far	fardata
@fardata	.fardata	@fardata?	.fardata?	@filename	??filename
fword	ge	Global	group	gt	high
ideal	if	If1	if2	ifb	ifdef
ifdif	ifdifi	Ife	ifidn	ifidni	ifnb
ifndef	%incl	Include	includelib	instr	irp
irpc	jumps	Label	.lall	large	le
length	.lfcond	%linum	%list	.list	local
locals	low	lt	macro	%macs	mask
masm	masm51	mod	model	.model	multerrs
name	ne	near	%newpage	%noconds	%nocref
%noctls	noemul	%noincl	nojumps	%nolist	nolocals
%nomacs	nomasm51	nomulterrs	%nosyms	not	nothing
%notrunc	nowarn	offset	or	org	%out
p186	p286	p286n	p287	p386	p386n
p386p	p387	p8086	p8087	page	%pagesize
%pcnt	pno87	%poplctl	proc	ptr	public
purge	%pushlctl	pwor	quirqs	qword	radix
.radix	record	rept	.sall	seg	segment
.seq	.sfcond	shl	short	shr	size
sizestr	small	stack	.stack	struc	substr
subttl	%subttl	%syms	symtype	%tabsize	tbyte
%text	.tfcond	this	??time	title	%title
%trunc	type	.type	udataseg	ufardata	union
unknown	uses	??version	warn	width	word
@wordsize	.xall	.xcref	.xlist	xor	

Conjunto De Instrucciones Del 8086

El conjunto de instrucciones del lenguaje ensamblador son mnemónicos de las instrucciones en lenguaje máquina del procesador. El conjunto completo de instrucciones de la familia de procesadores 8086 es:

Tabla 3.2 Conjunto de Instrucciones del 8086

aaa	aad	aam	aas	adc	add
cmc	cmp	cmps	cmpsb	cmpsw	cwd
daa	das	dec	div	esc	hlt
idiv	imul	in	inc	int	into
iret	ja	jae	jb	jbe	jc
jcxz	je	jg	jge	jl	jle
jmp	jna	jnae	jnb	jnbe	jnc
jne	jng	jnge	jnl	jnle	jno
jnp	jns	jnz	jo	jp	jpe
jpo	js	jz	lahf	lds	lea
les	lock	lods	lodsb	lodsw	loop
loope	loopne	loopnz	loopw	loopwe	loopwz
loopwne	loopwnz	loopz	mov	movs	movsb
movsw	mul	neg	nop	not	or
out	pop	popf	push	pushf	rcl
rcr	rep	repe	repne	repnz	repz
ret	retn	retf	rol	ror	sahf
sal	sar	sbb	scas	scasb	scasw
segcs	segds	segss	seges	shl	shr
stc	std	sti	stos	stosb	stosw
stosw	test	wait	xchg	xlat	xlatb
xor					

Estructura De Un Programa En Ensamblador

Un programa en el lenguaje ensamblador consiste de una secuencia de proposiciones, **una en cada línea** del código fuente.

La sintaxis de una proposición es la siguiente:

`[`*`etiqueta`*`] [`*`instrucción`*`|`*`directiva`* `[`*`operandos`*`]] [;`*`comentario`*`]`

donde *etiqueta, instrucción*|*directiva*, *operandos*, *comentario* son los campos de la proposición. Los campos se separan por caracteres blancos: caracteres de espacio y/o de tabulación. Todos los campos son opcionales, pero el campo *operandos* sólo pueden estar presente si existe el campo *instrucción*|*directiva.*

etiqueta es un nombre simbólico empleado para referirse a números, cadenas de caracteres o localidades de memoria dentro de un programa. Las etiquetas permiten darle nombre a las variables, constantes y localidades de una instrucción particular.

- Las etiquetas pueden contener los siguientes caracteres: A-Z a-z _ @ $? 0-9. Los dígitos 0-9 no pueden usarse como el primer carácter de una etiqueta. Un sólo carácter $ o ? no puede usarse como etiqueta ya que tienen un significado especial.

- Las etiquetas deben ser únicas. Con excepción de las definidas con la directiva =, y las etiquetas locales de macros y subrutinas. Las etiquetas pueden usarse como operandos tantas veces como se desee.

- Una etiqueta puede aparecer por sí sola en una línea. En este caso el valor asociado a la etiqueta es la dirección de la operación en la siguiente línea en el programa.

- Las palabras reservadas no pueden usarse como etiquetas.

- Las etiquetas que aparecen solas en una línea y las que van seguidas de una instrucción deben terminar en dos puntos (:). Las demás etiquetas por lo general no terminan en dos puntos.

instrucción es un mnemónico de una instrucción del procesador. Cada instrucción se traduce directamente a una instrucción del lenguaje máquina del microprocesador.

directiva es una instrucción para el ensamblador. No produce código ejecutable, sino que controla varios de los aspectos de cómo opera el ensamblador: el tipo de código generado (8086, 80286, 80386, etc.), los segmentos a usar, etc.

operandos consiste de cero, uno o más operandos. Un operando le especifica al ensamblador qué valor, registro, localidad de memoria, etc., asociar con cada instrucción.

- Hay varias clases de operandos: registros, constantes, etiquetas, variables y cadenas.

- El tipo y número de operandos depende de la instrucción o directiva.

- Si hay dos o más, van separados por comas (,)

comentario es cualquier secuencia de caracteres precedidos por un punto y coma (;) y que termina con el fin de la línea.

- Los comentarios son ignorados por el ensamblador

- Las líneas en blanco se tratan como comentarios.

Por ejemplo considere el programa de la siguiente página. Los números a la izquierda del recuadro no forman parte del programa y sólo sirven para numerar las líneas:

> Las líneas 1 a 7, 9, 16, 20, 28 y 45 son líneas con sólo comentarios.
>
> Las líneas 8, 10, 15, 17, 19, 21, 27, 29, 34, 39, 44 y 46 son líneas en blanco y son tratadas como comentarios. Se usan para separar las partes de un programa y darle mayor claridad.
>
> Las líneas 31 y 40 son líneas con sólo una etiqueta

Las líneas 32, 33, 35 a 38 y 41 a 43 contienen instrucciones.

Las líneas 11 a 14, 18, 22 a 26, 30 y 47 contienen directivas.

```
 1 ;*******************************************************
 2 ; PRIMER.ASM.
 3 ;
 4 ; Este programa ilustra la estructura de un programa en
 5 ; ensamblador. También muestra las principales directivas
 6 ; para El ensamblador.
 7 ;*******************************************************
 8
 9 ;****** CÓDIGO DE INICIO **********************************
10
11         ideal
12         dosseg
13         model small
14         stack 256
15
16 ;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************
17
18 cte      equ     10
19
20 ;****** VARIABLES DEL PROGRAMA ****************************
21
22         dataseg
23 codsal  db      0
24 dato1   db      ?
25 dato2   db      ?
26 resul   db      ?
27
28 ;****** CÓDIGO DEL PROGRAMA *******************************
29
30 codeseg
31 inicio:
32         mov     ax, @data    ; Inicializa el segmento de
33         mov     ds, ax       ; datos
34
35         mov     al, [dato1]  ; resul = dato1 + dato2 + cte
36         add     al, [dato2]
37         add     al, cte
38         mov     [resul], al
39
40 salir:
41         mov     ah, 04Ch
42         mov     al, [codsal]
43         int     21h
44
45 ;****** CÓDIGO DE TERMINACIÓN ******************************
46
47         end    inicio
```

Normalmente podemos considerar que un programa en ensamblador está formado de las siguientes secciones:

Código de inicio
Declaración de constantes simbólicas
Variables del programa
Código del programa
Código de terminación

Código De Inicio

El código de inicio de un programa en ensamblador para el 8086 tiene como propósito declarar los nombres de los segmentos en los que se almacenará el código del programa, los datos y la pila. También establece el modelo de memoria bajo el que se ensamblará el programa y el tamaño de la pila. El código de inicio consta de un conjunto de directivas. En el programa de ejemplo tenemos:

ideal

El modo **ideal** es una notación que emplea el ensamblador Turbo Assembler de Borland International para simplificar la sintaxis del código inicial de un programa. La directiva **ideal** le dice a Turbo Assembler que utilice el modo ideal. Si se omite se utiliza la notación empleada por la mayoría de los ensambladores, por ejemplo el de Microsoft.

dosseg

Esta directiva que forma parte del modo ideal, le instruye al ensamblador que nombre y ordene los segmentos del programa de la misma forma en que lo hacen los compiladores de alto nivel. Esto es adecuado para la mayoría de los programas en ensamblador y sobre todo en las rutinas de ensamblador que se van a ligar a un programa en alto nivel.

model

Esta directiva que debe ir después de la directiva **dosseg,** selecciona uno de los 6 modos diferentes que tiene Turbo Assenbler de generar el código de un programa dependiendo del tamaño que tiene el código y los datos. Estos modos llamados **modelos de memoria**, difieren en el tipo de apuntadores que el compilador genera por omisión y por lo tanto en la rapidez de ejecución del código. Los seis modelos de memoria son: Pequeñito (tiny), pequeño (small), medio (medium), compacto (compact), grande (large) y extenso (huge).

Modelo pequeñito (tiny)

En el modelo pequeñito, el código, los datos y la pila deben de estar en un solo segmento, esto es, el

programa completo no debe exceder a 64 KB. Todos los registros de segmento son inicializados al mismo valor y permanecen constantes durante la ejecución del programa. Bajo este modelo el compilador genera apuntadores cercanos. El código ejecutable producido es el más pequeño y rápido.

Modelo pequeño (small)

En el modelo pequeño, el código tiene su propio segmento y los datos y la pila están en otro segmento, esto es, el código no debe exceder a 64 KB ni tampoco los datos y pila. Los registros de segmento permanecen constantes durante la ejecución del programa. El código producido bajo este modelo utiliza también apuntadores cercanos y es igual de rápido que en el modelo pequeñito pero el tamaño del programa puede ser del doble. Normalmente debemos usar este modelo de memoria a menos que nuestro código o datos exceda los 64 KB.

Modelo medio (medium)

En el modelo medio, el código del programa puede ocupar varios segmentos pero los datos y la pila ocupan un solo segmento, esto es, el código puede exceder a 64 KB pero los datos y pila no. Por lo tanto, bajo este modelo el código utiliza apuntadores lejanos y los datos apuntadores cercanos. Debido a los apuntadores lejanos la velocidad de ejecución del programa se ve degradada. Los registros de los segmentos de datos y la pila se inicializan al mismo valor y permanecen constantes durante la ejecución del programa. Este modelo se debe usar cuando tengamos un programa de código extenso y pocos datos.

Modelo compacto (compact)

En el modelo compacto, el código del programa está limitado a un segmento, los datos pueden ocupar varios segmentos aunque el total de las variables estáticas no pueden exceder los 64 KB. Por lo tanto, bajo este modelo el código utiliza apuntadores cercanos y los datos apuntadores lejanos. Debido a los apuntadores lejanos también hay degradación en la velocidad de ejecución del programa. El registro del segmento de código permanece constante durante la ejecución del programa. Este modelo se debe usar cuando tengamos un programa de código pequeño y muchos datos.

Modelo grande (large)

En el modelo grande, tanto el código del programa como los datos puede ocupar varios segmentos, aunque el total de las variables estáticas no pueden exceder los 64 KB. Los apuntadores en este modelo son lejanos tanto para el código como para los datos. Esto hace que la velocidad de ejecución de un programa compilado bajo este modelo sea más lenta que cualquiera de las versiones anteriores. Este modelo se debe usar cuando tengamos un programa de código extenso y muchos datos.

Modelo extenso (huge).

En el modelo extenso, es igual que el modelo grande, sólo que el total de las variables estáticas si pueden exceder los 64 KB. Sin embargo cada variable estática en forma individual no puede exceder a 64 KB. Los apuntadores en este modelo son lejanos tanto para el código como para los datos. La velocidad de ejecución de un programa compilado bajo este modelo es más lenta que cualquiera de los otros modelos.

stack

Esta directiva reserva espacio de memoria para la pila del programa, un área de memoria que almacena valores temporales empleados por las subrutinas y las direcciones de regreso de las subrutinas.

Declaración de constantes simbólicas

Las constantes simbólicas son identificadores que le asociamos a constantes numéricas o a constantes cadenas. El uso de constantes simbólicas en un programa hace que sea más fácil de leer y modificar. Hay dos formas de declarar constantes simbólicas: mediante la directiva **equ** y mediante la directiva =. Por ejemplo:

```
filas   equ     10
cols    equ     5
tam     =       filas * cols
msj     equ     "Hola"
tam     =       0
```

Las directivas **equ** y = tienen las siguientes reglas:

- Las constantes simbólicas no son variables. Ni el nombre ni su valor asociado se almacena en el segmento de datos del programa.

- La directiva **equ** permite darle nombre a constantes numéricas, expresiones y cadenas. La directiva = sólo permite darle nombre a constantes numéricas y expresiones.
- No se le puede cambiar el valor asociado a un símbolo declarado con la directiva **equ**. En cambio a un símbolo declarado con la directiva = se le puede cambiar su valor tantas veces como se desee.

- Las directivas **equ** y = pueden aparecer en cualquier parte del programa. Pero es preferible colocarlas al principio ya que son más visibles.

- Las expresiones declaradas con = se evalúan inmediatamente, mientras que las expresiones declaradas con **equ** se evalúan hasta que el símbolo se usa.

Variables del programa

Las variables de un programa se declaran en el segmento de datos, por lo que la sección de declaración de variables empieza con la directiva **dataseg**. Esta directiva le indica a Turbo Assembler que almacene las variables en el segmento de datos del programa. Las variables pueden estar inicializadas o no. Los valores de las variables inicializadas se almacenan en el código del programa y se cargan en las variables al ejecutar el programa.

Para declarar variables podemos utilizar las directivas **db**, **dw**, **dd** o **dq**. La directiva **db** define una variable de un byte, la directiva **dw** define una variable de tipo palabra (dos bytes), la directiva **dd** define una variable de tipo palabra doble (cuatro bytes) y la directiva **dq** define una variable de tipo palabra cuádruple (ocho bytes). La sintaxis de estas directivas son:

```
nomVar    db   exp
nomVar    dw   exp
nomVar    dd   exp
nomVar    dq   exp
```

donde *nomVar* es el nombre de la variable y *exp* es una expresión constante cuyo valor se utiliza para inicializar la variable. Si la variable no se inicializa *exp* se substituye por un signo de interrogación, **?**. Por ejemplo

```
bdato   db      10h                ; Variable de un byte
                                   ; inicializada a 10h.

cresp   db      'A'                ; Variable de un byte
                                   ; inicializada a 65d.

bdatx   db      ?                  ; Variable de un byte
                                   ; no inicializada.

wdato   dw      0123h              ; Variable de tipo
                                   ; palabra inicializada
                                   ; a 0123h.

wdatx   dw      ?                  ; Variable de tipo
                                   ; palabra no
                                   ; inicializada.

ddato   dd      01234567h          ; Variable de tipo
                                   ; palabra doble
                                   ; inicializada a
                                   ; 01234567h.

ddatx   dd      ?                  ; Variable de tipo
                                   ; palabra doble
                                   ; no inicializada

qdato   dq      0123456789ABCDEFh  ; Variable de tipo
                                   ; palabra cuadruple
                                   ; inicializada a
```

```
                                        ; 0123456789abcdefh.

qdatx   dq      ?                       ; Variable de tipo
                                        ; palabra cuádruple
                                        ; no inicializada
```

Código del programa

En esta sección van las instrucciones que el microprocesador ejecutará al correr el programa. Todas las instrucciones van en el segmento de código, por lo que la sección de código del programa empieza con la directiva **codeseg**. Esta directiva le dice a Turbo Assembler que almacene las instrucciones en el segmento de código del programa.

El código del programa en la mayoría de los casos empieza con las siguientes líneas:

```
        codeseg
inicio:
        mov     ax, @data       ; Inicializa el segmento
        mov     ds, ax          ; de datos
```

La primera línea contiene la directiva **codeseg** mencionada anteriormente.

La segunda línea establece que *inicio* es una etiqueta asociada a la dirección de la primera instrucción del programa, que en este ejemplo aparece en la siguiente línea. Esta etiqueta es usada en la directiva **end** explicada en la siguiente sección.

En la tercera y cuarta línea se carga en el registro DS la dirección de inicio del segmento de datos del programa. El símbolo predefinido **@data** tiene el valor de esa dirección.
La instrucción **mov** mueve datos entre registros o entre registros y localidades de memoria. En este ejemplo la primera instrucción **mov** almacena la dirección del inicio del segmento de datos en el registro **AX** y la segunda instrucción **mov** mueve esa dirección del registro **AX** al registro de segmento **DS**. No se puede cargar una dirección directamente a uno de los registros de segmento, por lo que la instrucción

```
        mov     ds, @data        ; No válido
```

no es válida. La dirección debe cargarse primero a un registro de propósito general y luego de ahí moverse al registro de segmento.

También la mayoría de los programas que se van a ejecutar bajo el sistema operativo MSDOS deben terminar con las siguientes líneas de código:

```
salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h
```

El propósito de este código es llamar a una de las rutinas del sistema operativo que termina la ejecución

del programa y nos regresa al sistema operativo. Esta rutina que se conoce como una rutina de servicio a interrupción se verá en el Capítulo 8: INTERRUPCIONES. La llamada a esa rutina se hace mediante la instrucción:

```
int 21
```

Previamente se cargó en el registro **AH** el valor 04Ch que especifica que se desea terminar el programa y regresar al sistema operativo y en el registro **AL** el valor de la variable *codsal*, que es el código de salida del programa. El programa le regresa al sistema operativo el valor del código de salida en la variable de ambiente **errorlevel**. Por convención, un código de salida de cero denota que el programa terminó con éxito y un código diferente de cero indica que hubo un error durante la ejecución del programa. El valor del código identifica al error ocurrido.

En los programas en C también se acostumbra regresarle un código de salida al sistema operativo. esto se hace mediante la proposición return en la función *main*():

```
int main(void)
{
   ...

   return 0;
}
```

Código de terminación

Todo programa en ensamblador debe contener una directiva **end**, la cual marca el final del código fuente. Turbo Assembler ignora cualquier línea después de la directiva **end**. La sintaxis de la directiva **end** es la siguiente:

```
end  [etiqueta]
```

etiqueta es el nombre simbólico de la dirección de la instrucción por la que empezará a ejecutarse el programa. Si se omite, el programa empezará su ejecución en la primera instrucción del código fuente.

En un programa de un sólo módulo, es decir con un archivo con código fuente, la directiva **end** debe siempre especificar el inicio del programa. En un programa que consiste de más de un módulo, sólo la directiva **end** en el módulo que contiene la primera instrucción a ejecutar debe llevar una etiqueta. En los otros módulos la directiva **end** debe aparecer por sí sola.

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documentod PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

CAPÍTULO 4

INSTRUCCIONES DE TRANSFERENCIA BÁSICAS, ARITMÉTICAS Y LÓGICAS

Operandos Y Modos De Direccionamiento

Los operandos de una instrucción especifican los datos con los que va a trabajar la instrucción. Esos datos se encuentran en registros o en localidades de memoria y los operandos le dicen al ensamblador en qué registro o localidad de memoria está el dato. En otras palabras los operandos o la ausencia de ellos establece el mecanismo para calcular las direcciones donde se encuentran los datos. A ese mecanismo se le conoce como **modo de direccionamiento.**

Instrucciones Sin Operandos O Modo De Direccionamiento Inherente

Algunas instrucciones no tienen operandos. Esto se debe a que la instrucción no necesita datos o a que los datos con los que opera la instrucción están en registros predefinidos. Ejemplos:

```
clc     ; Pone la bandera de acarreo en 0. No hay
        ; dato.

cbw     ; Extiende el valor signado del registro
        ; de 8 bits AL al registro de 16 bits AX.
        ; En este caso el dato se encuentra en el
        ; registro AL antes de la operación y en
        ; el registro AX después de la operación.
```

Operandos Constantes O Modo De Direccionamiento Inmediato

Muchas de las instrucciones permiten que uno de los operandos sea una constante. La constante puede ser una constante numérica, una expresión constante o una etiqueta.

- Una **constante numérica** es un número de uno o dos bytes expresado en decimal, hexadecimal o binario. Por ejemplo:

```
mov     al, 65          ; Constante decimal.
mov     al, 41h         ; Constante hexadecimal.
mov     al, 01000001b   ; Constante binaria.
mov     al, 'A'         ; Constante de carácter.
                        ; Se almacena su código
                        ; ASCII.
add     ax, 1234        ; Constante de dos bytes.
```

- Una **expresión constante** es una expresión donde todos sus operandos son constantes. Las expresiones que Turbo Assembler puede evaluar incluyen paréntesis anidados, operadores aritméticos, lógicos y relacionales y otros operadores que nos dan la dirección de segmento y desplazamiento de etiquetas y que determinan el tamaño de las variables. La evaluación de estas expresiones ocurre al tiempo de ensamblado. Algunos de los operadores que pueden usarse en expresiones se muestran en la tabla 4-1. La precedencia de los operadores va de mayor a menor con los operadores en la misma celda con la misma precedencia:

Tabla 4-1 Algunos operadores usados en expresiones constantes

Operador	Descripción	Ejemplos
() **[]** **offset** **seg** **size**	Cambia precedencia Indirección Desplazamiento Dirección de segmento Tamaño de una variable	mov ax, 2*(3 + 2)/3 add bx, [12h] mov ax, offset dato1 mov ax, seg dato1 mov ah, size dato1
-	Negativo	mov ah, -2
* / **mod** **shl** **shr**	Multiplicación División Módulo Corrimiento a la izquierda Corrimiento a la derecha	mov ax, 2*(3 + 2)/3 mov cx, 15 mod 4 mov ax, 3 shr 4 and 0Ah
+ -	Suma Resta	mov ax, 2*(3 + 2)/3
not	Complemento de bits	mov ax, not 3
and	Intersección de bits	mov ax, 3 shr 4 and 0Ah
or **xor**	Unión inclusiva de bits Unión exclusiva de bits	mov ax, 3 shl 4 or 3Ah

- Una **etiqueta** puede generar una constante de dos formas:

- Si se utiliza como operando de ciertos operadores en expresiones constantes. Por ejemplo, en las siguientes instrucciones la etiqueta dato1, que es el nombre de una variable, genera una constante.

```
mov     ax, offset dato1
mov     ax, seg dato1
mov     ah, size dato1
```

- Si se utilizan como el destino de instrucciones de salto y llamada a subrutina. Por ejemplo, la etiqueta ciclo que es el operando de la operación de salto jb, genera una constante: La dirección a la que salta el programa.

```
ciclo:
        ...
        cmp     ax, 10
        jb      ciclo
```

Operandos Registros O Modo De Direccionamiento Por Registro

Los registros son, tal vez, los operandos más usados en las instrucciones. Algunos ejemplos son:

```
mov     di, ax
push    di,
xchg    ah, dl
inc     si
mov     al, 2
```

Modo De Direccionamiento Directo

En este modo de direccionamiento el dato está en una localidad de memoria. El operando es una expresión constante sobre la que está operando el operador de indirección, []. Su sintaxis tiene la forma:

[*expCte***]**

donde *expCte* representa el desplazamiento del dato con respecto al segmento en que está contenido, normalmente el segmento de datos. Por ejemplo:

```
dato1   dw      0123h
dato2   dd      456789ABh
        ...

        mov     ax, [0AF3h]     ; Guarda en el registro
                                ; AX la palabra
                                ; almacenada en la
```

```
                                ; dirección 0AF3h.

mov     bx, [dato1]             ; Guarda en el registro
                                ; BX el valor de la
                                ; variable dato1

mov     cl, [dato1]             ; Guarda en el registro
                                ; CL el byte menos
                                 ; significativo del valor
                                ; de la variable dato1,
                                ; esto es, 23h.

mov     dh, [dato1+1]           ; Guarda en el registro
                                ; DH el byte más
                                ; significativo del valor
                                ; de la variable dato1,
                                ; esto es, 01h.

mov     ax, [dato2]             ; Guarda en el registro
                                ; AX la palabra menos
                                ; significativa del valor
                                ; de la variable dato2,
                                ; esto es, 89ABh.

mov     bx, [dato2+2]           ; Guarda en el registro
                                ; BX la palabra más
                                ; significativa del valor
                                ; de la variable dato2,
                                ; esto es, 4567h.

mov     cx, [dato2+1]           ; Guarda en el registro
                                ; CX la palabra que se
                                ; encuentra un byte más
                                ; adelante del inicio de
                                ; la variable dato2, esto
                                ; es, 6789h.
```

Instrucciones

Las instrucciones del lenguaje ensamblador se subdividen por su función en 6 grupos:

1. Instrucciones de transferencia de datos
2. Instrucciones aritméticas
3. Instrucciones lógicas
4. Instrucciones de control de flujo

5. Instrucciones de control del procesador

6. Instrucciones de cadenas.

Cabe mencionar que esta clasificación no es única y que existen otras formas de clasificar las instrucciones. En este tema se estudiarán algunas de las instrucciones en los primeros tres grupos. En otros temas se estudiarán algunas otras funciones de estos grupos así como de los grupos restantes.

Para describir la sintaxis de las instrucciones se usará la notación mostrada en la tabla 4-2.

Tabla 4-2. Notación empleada en la sintaxis de las instrucciones

Símbolo	Significado
|	Una de dos o más opciones mutuamente excluyentes
[]	Los términos entre corchetes son opcionales
	Si la instrucción aparece sola es que no tiene operandos.
inmB	Operando de un byte con direccionamiento inmediato
inmW	Operando de una palabra con direccionamiento inmediato
memB	Operando de un byte que se encuentra en una localidad de memoria.
memW	Operando de una palabra que se encuentra en una localidad de memoria.
regB	Registro de propósito general de un byte.
regW	Registro de propósito general de una palabra.

NOTA: A menos que se especifique lo contrario, los operandos que se encuentra en una localidad de memoria incluyen a todos los modos de direccionamiento que hacen referencia a una localidad de memoria. Los operandos con direccionamiento directo son de este tipo pero no son los únicos. Los restantes se verán en los siguientes temas.

Instrucciones De Transferencia

mov *destino, fuente*

Copia un valor de *fuente* a *destino*.

Sintaxis:

```
mov  regB, inmB|regB|memB
mov  memB, inmB|regB
mov  regW, inmW|regW|memW
mov  memW, inmW|regW
```

El valor de *fuente* no se modifica. Sólo uno de los operandos puede ser una localidad de memoria. Si

uno de los operandos es un registro de segmento, el otro registro debe ser un registro de propósito general.

La instrucción **mov** no afecta el estado de las banderas.

Cuando uno de los registros es el registro **AX** o **AL**, Turbo Assembler genera código más rápido.

xchg *destino, fuente*

Intercambia los valores en *destino* y *fuente.*

Sintaxis:

```
xchg regB, regB|memB
xchg memB, regB
xchg regW, regW|memW
xchg memW, regW
```

Sólo uno de los operandos puede ser una localidad de memoria.

La instrucción **xchg** no afecta el estado de las banderas.

Cuando los valores a intercambiar están en registros, Turbo Assembler genera código más rápido.

Instrucciones Aritméticas

add *destino, fuente*

Suma los contenidos de *fuente* y *destino* y guarda el resultado en *destino*.

Sintaxis:

```
add  regB, inmB|regB|memB
add  memB, inmB|regB
add  regW, inmB|inmW|regW|memW
add  memW, inmB|inmW|regW
```

La instrucción **add** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

adc *destino, fuente*

Suma los contenidos de *fuente, destino* y el valor de la bandera de acarreo y guarda el resultado en

destino.

Sintaxis:

```
adc  regB, inmB|regB|memB
adc  memB, inmB|regB
adc  regW, inmB|inmW|regW|memW
adc  memW, inmB|inmW|regW
```

La instrucción **adc** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

Al sumar valores multibytes o multipalabras, use **add** para sumar los bytes o palabras menos significativos y después use la instrucción **adc** para sumar los bytes o palabras más significativos y los posibles acarreos generados por las sumas de los bytes o palabras menos significativos.

inc *destino*

Incrementa en uno el contenido de *destino*.

Sintaxis:

```
inc  regB|memB
inc  regW|memW
```

La instrucción **inc** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A** y paridad **P**.

sub *destino, fuente*

Resta al valor de *destino* el valor de *fuente* y guarda el resultado en *destino*.

Sintaxis:

```
sub  regB, inmB|regB|memB
sub  memB, inmB|regB
sub  regW, inmB|inmW|regW|memW
sub  memW, inmB|inmW|regW
```

La instrucción **sub** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

sbb *destino, fuente*

Resta al valor de *destino* el valor de *fuente* tomando en consideración un posible préstamo de una instrucción **sbb** o **sub** previa y guarda el resultado en *destino*.

Sintaxis:

```
sbb  regB, inmB|regB|memB
sbb  memB, inmB|regB
sbb  regW, inmB|inmW|regW|memW
sbb  memW, inmB|inmW|regW
```

La instrucción **sbb** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

Al restar valores multibytes o multipalabras, use **sub** para restar los bytes o palabras menos significativos y después use la instrucción **sbb** para restar los bytes o palabras más significativos tomando en cuenta los posibles préstamos de las restas de los bytes o palabras menos significativos.

cmp *destino, fuente*

Compara los contenidos de *fuente*, y *destino*.

Sintaxis:

```
cmp  regB, inmB|regB|memB
cmp  memB, inmB|regB
cmp  regW, inmB|inmW|regW|memW
cmp  memW, inmB|inmW|regW
```

La instrucción **cmp** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

Sólo uno de los operandos puede ser una localidad de memoria. La instrucción **cmp** trabaja restando el valor de *fuente* de *destino* y tirando el resultado pero afectando las banderas.

dec *destino*

Decrementa en uno el contenido de *destino*.

Sintaxis:

```
dec  regB|memB
dec  regW|memW
```

La instrucción **dec** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A** y paridad **P**.

mul *fuente*

Multiplica dos operandos sin signo.

Sintaxis:

```
mul  regB|memB
mul  regW|memW
```

Si la multiplicación es de un byte por un byte, los operandos y el resultado están en:

Multiplicando: **AL**
Multiplicador: *fuente* (Un byte)
Producto: **AX**

Si la multiplicación es de una palabra por una palabra, los operandos y el resultado están en:

Multiplicando: **AX**
Multiplicador: *fuente* (Una palabra)
Producto: **DX:AX** (La palabra menos significativa en **AX**)

Después de la instrucción **mul** las siguientes banderas quedan en un estado indefinido: signo **S**, cero **Z**, acarreo auxiliar **A** y paridad **P**.

Si las banderas de acarreo **C** y de sobreflujo **O** son ambas 0 después de la operación entonces la parte más significativa del resultado es 0. Si las banderas de acarreo **C** y de sobreflujo **O** son ambas 1 después de la operación entonces el resultado ocupa todo el (los) registro(s) del producto.

imul *fuente*

Multiplica dos operandos con signo.

Sintaxis:

```
imul regB|memB
imul regW|memW
```

Si la multiplicación es de un byte por un byte, los operandos y el resultado están en:

Multiplicando: **AL**
Multiplicador: *fuente* (Un byte)
Producto: **AX**

Si la multiplicación es de una palabra por una palabra, los operandos y el resultado están en:

Multiplicando: **AX**
Multiplicador: *fuente* (Una palabra)
Producto: **DX:AX** (La palabra menos significativa en **AX**)

Después de la instrucción **imul** las siguientes banderas quedan en un estado indefinido: signo **S**, cero **Z**, acarreo auxiliar **A** y paridad **P**.

Si las banderas de acarreo **C** y de sobreflujo **O** son ambas 0 después de la operación entonces la parte más significativa del resultado es simplemente la extensión del signo de la parte menos significativa. Si las banderas de acarreo **C** y de sobreflujo **O** son ambas 1 después de la operación entonces el resultado ocupa todo el (los) registro(s) del producto.

div *fuente*

Divide dos operandos sin signo.

Sintaxis:

```
div  regB|memB
div  regW|memW
```

Si la división es de una palabra entre un byte, los operandos y los resultados están en:

Dividendo: **AX**
Divisor:*fuente* (Un byte)
Cociente: **AL**
Residuo: **AH**

Si la división es de una palabra doble entre una palabra, los operandos y los resultados están en:

Dividendo: **DX:AX** (La palabra menos significativa en **AX**)
Divisor:*fuente* (Una palabra)
Cociente: **AX**
Residuo: **DX**

Después de la instrucción **div** las siguientes banderas quedan en un estado indefinido: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A** y paridad **P** y acarreo **C**.

Si el resultado de la división es mayor que el valor máximo permitido en el registro donde se almacena el cociente o si el divisor vale cero, se genera una interrupción tipo 0: división entre 0. El programa se detiene y despliega el mensaje de división entre 0.

idiv *fuente*

Divide dos operandos con signo.

Sintaxis:

```
idiv regB|memB
idiv regW|memW
```

Si la división es de una palabra entre un byte, los operandos y los resultados están en:

Dividendo: **AX**
Divisor:*fuente* (Un byte)
Cociente: **AL**
Residuo: **AH**

Si la división es de una palabra doble entre una palabra, los operandos y los resultados están en:

Dividendo: **DX:AX** (La palabra menos significativa en **AX**)
Divisor:*fuente* (Una palabra)
Cociente: **AX**
Residuo: **DX**

Después de la instrucción **idiv** las siguientes banderas quedan en un estado indefinido: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A** y paridad **P** y acarreo **C**.

Si el resultado de la división es mayor que el valor máximo permitido en el registro donde se almacena el cociente o si el divisor vale cero, se genera una interrupción tipo 0: división entre 0. El programa se detiene y despliega el mensaje de división entre 0.

cbw

Convierte un byte con signo a una palabra por extensión de signo.

Sintaxis:

```
cbw
```

La instrucción **cbw** no afecta el estado de las banderas.

Utilice **cbw** para extender un valor de 8 bits con signo en el registro **AL** a un valor de 16 bits con signo de la misma magnitud en **AX**. La instrucción trabaja copiando el bit más significativo de **AL** a todos los bits de **AH**, esto es, colocando 0FFh en **AH** si **AL** es negativo o colocando 0h en **AH** si **AL** es positivo.

cwd

Convierte una palabra con signo a una palabra doble por extensión de signo.

Sintaxis:

```
cwd
```

La instrucción **cwd** no afecta el estado de las banderas.

Utilice **cwd** para extender un valor de 16 bits con signo en el registro **AX** a un valor de 32 bits con signo de la misma magnitud en **DX:AX**. La instrucción trabaja copiando el bit más significativo de **AX** a todos los bits de **DX**, esto es, colocando 0FFFFh en **DX** si **AX** es negativo o colocando 0h en **DX** si **AX** es positivo.

neg *destino*

Obtiene el negativo de *destino* (forma el complemento de 2).

Sintaxis:

```
neg  regB|memB
neg  regW|memW
```

La instrucción **neg** modifica las siguientes banderas: sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

Ejemplos Sobre Instrucciones Aritméticas

1. El siguiente programa suma una variable de 1 byte y una variable de una palabra ambas signadas.

```
;*********************************************************
; SUMABYWS.ASM
;
; Este programa suma una variable de 1 byte con una
; variable de una palabra, ambas signadas. El pseudocódigo
; de este programa es el siguiente:
;
;   AL = dato1
;   AX = (int)AL
;   AX += dato2
;   resul = AX
;*********************************************************
;
```

```
;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
dato1   db      ?
dato2   dw      ?
resul   dw      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos

        mov     al, [dato1]     ; AL = dato1
        cbw                     ; AX = AL
        add     ax, [dato2]     ; AX += dato2
        mov     [resul], ax     ; resul = AX
salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

2. El siguiente programa suma dos variables de tipo palabra doble.

```
;*********************************************************
; SUMA2DW.ASM
;
; Este programa suma dos variables de tipo palabra doble.
; El resultado queda en una variable tipo palabra doble.
; El pseudocódigo de este programa es el siguiente:
;
;   DX:AX = dato1
;   DX:AX += dato2
;   resul = DX:AX
;*********************************************************

;****** CÓDIGO DE INICIO **********************************
```

```
        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
dato1   dd      ?
dato2   dd      ?
resul   dd      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data               ; Inicializa el
        mov     ds, ax                  ; segmento de datos

        mov     ax, [word dato1]        ; DX:AX = dato1
        mov     dx, [word dato1+2]
        add     ax, [word dato2]        ; DX:AX += dato2
        adc     dx, [word dato2+2]
        mov     [word resull], ax       ; resul = DX:AX
        mov     [word resull+2], dx
salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

Ejercicios Sobre Instrucciones Aritméticas

Escriba programas en ensamblador que implemente los siguientes códigos en C:

1.
```
char x, y, z;
int w;

w = x * y – z;
```

2.
```
char x;
int y, z, w;

w = (x + y)/z;
```

Instrucciones Lógicas

not *destino*

Obtiene el complemento de 1 de *destino*.

Sintaxis:

```
not  regB|memB
not  regW|memW
```

La instrucción **not** deja inalteradas a las banderas.

and *destino, fuente*

Calcula la intersección lógica entre los bits de *fuente* y *destino* y guarda el resultado en *destino*.

Sintaxis:

```
and  regB, inmB|regB|memB
and  memB, inmB|regB
and  regW, inmB|inmW|regW|memW
and  memW, inmB|inmW|regW
```

La instrucción **and** modifica las siguientes banderas: sobreflujo **O** = 0, signo **S**, cero **Z**, paridad **P** y acarreo **C** = 0. La bandera de acarreo auxiliar **A** queda en un estado indefinido.

test *destino, fuente*

Compara valores mediante la intersección lógica entre los bits de *fuente* y *destino*.

Sintaxis:

```
test regB, inmB|regB|memB
test memB, inmB|regB
test regW, inmW|regW|memW
test memW, inmW|regW
```

La instrucción **test** trabaja en forma idéntica a la instrucción **and** excepto que el resultado de la operación se descarta y sólo se afectan las banderas.

La instrucción **test** modifica las siguientes banderas: sobreflujo **O** = 0, signo **S**, cero **Z**, paridad **P** y acarreo **C** = 0. La bandera de acarreo auxiliar **A** queda en un estado indefinido.

or *destino, fuente*

Calcula la unión inclusiva lógica entre los bits de *fuente* y *destino* y guarda el resultado en *destino*.
Sintaxis:

```
or   regB, inmB|regB|memB
or   memB, inmB|regB
or   regW, inmB|inmW|regW|memW
or   memW, inmB|inmW|regW
```

La instrucción **or** modifica las siguientes banderas: sobreflujo **O** = 0, signo **S**, cero **Z**, paridad **P** y acarreo **C** = 0. La bandera de acarreo auxiliar **A** queda en un estado indefinido.

xor *destino, fuente*

Calcula la unión exclusiva lógica entre los bits de *fuente* y *destino* y guarda el resultado en *destino*.

Sintaxis:

```
xor  regB, inmB|regB|memB
xor  memB, inmB|regB
xor  regW, inmB|inmW|regW|memW
xor  memW, inmB|inmW|regW
```

La instrucción **xor** modifica las siguientes banderas: sobreflujo **O** = 0, signo **S**, cero **Z**, paridad **P** y acarreo **C** = 0. La bandera de acarreo auxiliar **A** queda en un estado indefinido.

sal | shl *destino, cuenta*

Corre hacia la izquierda los bits de *destino* un número de veces dado por *cuenta.*

Sintaxis:

```
sal|shl   regB|memB, 1|cl
sal|shl   regW|memW, 1|cl
```

Los mnemónicos **sal** y **shl** son sinónimos. **sal|shl** corre hacia la izquierda los bits de *destino*. Esto es, en cada corrimiento el bit más significativo de *destino* se mueve a la bandera de acarreo y se inserta un cero como el bit menos significativo de *destino*, mientras todos los demás bits se mueven una posición a la izquierda. El mnemónico **sal** se emplea para multiplicar *destino* por potencias de 2, mientras que el mnemónico **shl** se emplea para correr los bits de *destino* a la izquierda. Aunque ambos son intercambiables.

Si el número de corrimientos deseado es de uno, *cuenta* puede ser el valor inmediato 1. Para otros valores, el número de corrimientos debe asignarse previamente al registro **CL** y especificar éste como

cuenta.

La instrucción **sal|shl** modifica la banderas de signo **S**, cero **Z**, paridad **P** y acarreo **C**. La bandera de acarreo auxiliar queda en un estado indefinido. Si *cuenta* es un uno inmediato, la instrucción también modifica a la bandera de sobreflujo **O**, si *cuenta* es un valor en el registro **CL**, la bandera de sobreflujo **O** queda en un estado indefinido.

sar *destino, cuenta*

Corre hacia la derecha los bits de *destino,* preservando el signo, un número de veces dado por *cuenta.*

Sintaxis:

```
sar  regB|memB, 1|cl
sar  regW|memW, 1|cl
```

A diferencia de los mnemónicos **sal** y **shl**, **sar** y **shr** no son sinónimos. **sar** corre hacia la derecha los bits de *destino* preservando el signo. Esto es, en cada corrimiento una copia del bit más significativo de *destino* se inserta en el bit más significativo. El bit menos significativo se mueve a la bandera de acarreo, mientras todos los demás bits se mueven una posición a la derecha. La instrucción **sar** se emplea para dividir un valor signado en *destino* entre potencias de 2.

Si el número de corrimientos deseado es de uno, *cuenta* puede ser el valor inmediato 1. Para otros valores, el número de corrimientos debe asignarse previamente al registro **CL** y especificar éste como *cuenta.*

La instrucción **sar** modifica la banderas de signo **S**, cero **Z**, paridad **P** y acarreo **C**. La bandera de acarreo auxiliar queda en un estado indefinido. Si *cuenta* es un uno inmediato, la instrucción también modifica a la bandera de sobreflujo **O**, si *cuenta* es un valor en el registro **CL**, la bandera de sobreflujo **O** queda en un estado indefinido.

shr *destino, cuenta*

Corre hacia la derecha los bits de *destino* un número de veces dado por *cuenta.*

Sintaxis:

```
shr  regB|memB, 1|cl
shr  regW|memW, 1|cl
```

shr corre hacia la derecha los bits de *destino*. Esto es, en cada corrimiento se inserta un cero en el bit más significativo de *destino*. El bit menos significativo se mueve a la bandera de acarreo, mientras todos los demás bits se mueven una posición a la derecha. La instrucción **shr** se emplea para dividir un valor sin signo en *destino* entre potencias de 2.

Si el número de corrimientos deseado es de uno, *cuenta* puede ser el valor inmediato 1. Para otros valores, el número de corrimientos debe asignarse previamente al registro **CL** y especificar éste como *cuenta*.

La instrucción **shr** modifica la banderas de signo **S**, cero **Z**, paridad **P** y acarreo **C**. La bandera de acarreo auxiliar queda en un estado indefinido. Si *cuenta* es un uno inmediato, la instrucción también modifica a la bandera de sobreflujo **O**, si *cuenta* es un valor en el registro **CL**, la bandera de sobreflujo **O** queda en un estado indefinido.

rcl *destino, cuenta*

Rota hacia la izquierda los bits de *destino* a través de la bandera de acarreo un número de veces dado por *cuenta.*

Sintaxis:

```
rcl  regB|memB, 1|cl
rcl  regW|memW, 1|cl
```

rcl rota hacia la izquierda los bits de *destino* incluyendo a la bandera de acarreo **C** como parte del valor original. Esto es, en cada rotación el bit más significativo de *destino* se mueve a la bandera de acarreo y el valor en la bandera de acarreo se inserta como el bit menos significativo de *destino*, mientras todos los demás bits se mueven una posición a la izquierda.

Si el número de rotaciones deseado es de uno, *cuenta* puede ser el valor inmediato 1. Para otros valores, el número de rotaciones debe asignarse previamente al registro **CL** y especificar éste como *cuenta.*

La instrucción **rcl** modifica la bandera de acarreo **C**. Si *cuenta* es un uno inmediato, la instrucción también modifica a la bandera de sobreflujo **O**, si *cuenta* es un valor en el registro **CL**, la bandera de sobreflujo **O** queda en un estado indefinido.

rcr *destino, cuenta*

Rota hacia la derecha los bits de *destino* a través de la bandera de acarreo un número de veces dado por *cuenta.*

Sintaxis:

```
rcr  regB|memB, 1|cl
rcr  regW|memW, 1|cl
```

rcr rota hacia la derecha los bits de *destino* incluyendo a la bandera de acarreo **C** como parte del valor original. Esto es, en cada rotación el bit menos significativo de *destino* se mueve a la bandera de acarreo y el valor en la bandera de acarreo se inserta como el bit más significativo de *destino*, mientras

todos los demás bits se mueven una posición a la derecha.

Si el número de rotaciones deseado es de uno, *cuenta* puede ser el valor inmediato 1. Para otros valores, el número de rotaciones debe asignarse previamente al registro **CL** y especificar éste como *cuenta.*

La instrucción **rcr** modifica la bandera de acarreo **C**. Si *cuenta* es un uno inmediato, la instrucción también modifica a la bandera de sobreflujo **O**, si *cuenta* es un valor en el registro **CL**, la bandera de sobreflujo **O** queda en un estado indefinido.

rol *destino, cuenta*

Rota hacia la izquierda los bits de *destino* un número de veces dado por *cuenta.*

Sintaxis:

```
rol  regB|memB, 1|cl
rol  regW|memW, 1|cl
```

rol rota hacia la izquierda los bits de *destino*. Esto es, en cada rotación el bit más significativo de *destino* se inserta como el bit menos significativo de *destino*, mientras todos los demás bits se mueven una posición a la izquierda. Adicionalmente el bit más significativo del valor original de *destino* se copia a la bandera de acarreo **C**.

Si el número de rotaciones deseado es de uno, *cuenta* puede ser el valor inmediato 1. Para otros valores, el número de rotaciones debe asignarse previamente al registro **CL** y especificar éste como *cuenta.*

La instrucción **rol** modifica la bandera de acarreo **C**. Si *cuenta* es un uno inmediato, la instrucción también modifica a la bandera de sobreflujo **O**, si *cuenta* es un valor en el registro **CL**, la bandera de sobreflujo **O** queda en un estado indefinido.

ror *destino, cuenta*

Rota hacia la derecha los bits de *destino* un número de veces dado por *cuenta.*

Sintaxis:

```
ror  regB|memB, 1|cl
ror  regW|memW, 1|cl
```

ror rota hacia la derecha los bits de *destino*. Esto es, en cada rotación el bit menos significativo de *destino* se inserta como el bit más significativo de *destino*, mientras todos los demás bits se mueven una posición a la derecha. Adicionalmente el bit menos significativo del valor original de *destino* se copia a la bandera de acarreo **C**.

Si el número de rotaciones deseado es de uno, *cuenta* puede ser el valor inmediato 1. Para otros valores, el número de rotaciones debe asignarse previamente al registro **CL** y especificar éste como *cuenta.*

La instrucción **ror** modifica la bandera de acarreo **C**. Si *cuenta* es un uno inmediato, la instrucción también modifica a la bandera de sobreflujo **O**, si *cuenta* es un valor en el registro **CL**, la bandera de sobreflujo **O** queda en un estado indefinido.

Ejemplos sobre instrucciones lógicas

1. El siguiente programa determina el valor del i-ésimo bit de una variable de un byte. Si el bit vale 0 deja un cero en la variable resul. Si el bit vale 1 deja un valor diferente de cero.

```
;*********************************************************
; BIT.ASM
;
; Este programa determina el valor del i-ésimo bit de una
; variable de un byte. Si el bit vale 0 deja un cero en la
; variable resul. Si el bit vale 1 deja un valor diferente
; de cero. El pseudocódigo de este programa es el
; siguiente:
;
;   AH = dato
;   AL = 1 << CL
;   AH = AH & AL
;   resul = AH
;*********************************************************

;****** CÓDIGO DE INICIO ***********************************

        ideal
        dosseg
        model small
        stack   256

;****** VARIABLES DEL PROGRAMA *****************************

        dataseg
codsal  db      0
dato    db      ?
bit     db      ?
resul   db      ?

;****** CÓDIGO DEL PROGRAMA ********************************

        codeseg
inicio:
        mov     ax, @data               ; Inicializa el
```

```
        mov     ds, ax                      ; segmento de datos

        mov     ah, [dato]                  ; AH = dato
        mov     al, 1                       ; Al = 1
        mov     cl, [bit]                   ; CL = bit
        shl     al, cl                      ; AL <= CL
        and     ah, al                      ; AH &= Al
        mov     [resul], ah                 ; resul = AH

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

2. El siguiente programa implementa todas las rotaciones con una variable de tipo palabra doble.

```
;*************************************************************
; ROTACION.ASM
;
; Este programa implementa todas las rotaciones con una
; variable de tipo palabra doble. El pseudocódigo de este
; programa es el siguiente:
;
;   DX:AX = dato
;   DX:AX = rcl DX:AX
;   r_rcl = DX:AX
;
;   DX:AX = dato
;   DX:AX = rcr DX:AX
;   r_rcr = DX:AX
;
;   DX:AX = dato
;   DX:AX = rol DX:AX
;   r_rol = DX:AX
;
;   DX:AX = dato
;   DX:AX = ror DX:AX
;   r_ror = DX:AX
;*************************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256
```

```
;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
dato    dd      ?
r_rcl   dd      ?
r_rcr   dd      ?
r_rol   dd      ?
r_ror   dd      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data               ; Inicializa el
        mov     ds, ax                  ; segmento de datos

; Rotación a la izquierda a través de la bandera de acarreo
        mov     ax, [word dato]         ; DX:AX = dato
        mov     dx, [word dato+2]

        rcl     ax, 1                   ; rcl DX:AX, 1
        rcl     dx, 1

        mov     [word r_rcl], ax        ; r_rcl = DX:AX
        mov     [word r_rcl+2], dx

; Rotación a la derecha a través de la bandera de acarreo
        mov     ax, [word dato]         ; DX:AX = dato
        mov     dx, [word dato+2]

        rcr     dx, 1                   ; rcr DX:AX, 1
        rcr     ax, 1

        mov     [word r_rcr], ax        ; r_rcr = DX:AX
        mov     [word r_rcr+2], dx

; Rotación a la izquierda
        mov     ax, [word dato]         ; DX:AX = dato
        mov     dx, [word dato+2]

        mov     bx, dx                  ; rol DX:AX, 1
        shl     bx, 1
        rcl     ax, 1
        rcl     dx, 1

        mov     [word r_rol], ax        ; r_rol = DX:AX
        mov     [word r_rol+2], dx

; Rotación a la derecha
        mov     ax, [word dato]         ; DX:AX = dato
        mov     dx, [word dato+2]
```

```
        mov     bx, ax                  ; ror DX:AX, 1
        shr     bx, 1
        rcr     dx, 1
        rcr     ax, 1

        mov     [word r_ror], ax        ; r_ror = DX:AX
        mov     [word r_ror+2], dx

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

Ejercicios sobre instrucciones lógicas

1. Escriba un programa que implemente todas los corrimientos con una variable de tipo palabra doble.

2. Escriba un programa que intercambie los nibles más significativo y menos significativo de una variable de un byte. El resultado queda en la misma variable.

3. Escriba un programa que intercambie los bits de una variable de tipo palabra. El valor del bit 15 debe quedar en el bit 0, el valor del bit 14 debe quedar en el bit 1, etc. El resultado queda en la misma variable.

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documento PDF en el siguiente servidor FTP:

ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

Problemas

1. Haga un programa que sume 4 variables de tipo palabra doble, llamadas **dato1**, **dato2**, **dato3** y **dato4**. El resultado deberá quedar en la variable de tipo palabra doble **resul**. El programa deberá llamarse **TAREA401.**

2. Haga un programa que multiplique dos variables de tipo palabra doble sin signo llamadas **dato1** y **dato2** y escriba el resultado en variable de tipo palabra cuádruple, llamada **resul.** El programa deberá llamarse **TAREA402.**

3. Haga un programa para convertir una temperatura en grados Centígrados a Fahrenheit. La temperatura en grados Centígrados está en la variable **CENTI**, y la temperatura en grados Fahrenheit está en la variable **FAHR**. El rango de temperaturas que el programa debe manejar es de -200 a 200 grados centígrados. El programa deberá llamarse **TAREA403**.

4. Haga un programa que complemente el valor del i-ésimo bit de una variable de un byte, esto es, si el bit tiene el valor de uno deberá cambiarlo a cero y viceversa. La variable deberá llamarse **dato**, el bit a complementar estará en la variable **bit** y el resultado en la variable **resul**. El programa deberá llamarse **TAREA404.**

5. Haga un programa que sume los nibles (4 bits) más significativo y menos significativo de una variable de un byte llamada **dato** y escriba el resultado en una variable de un byte llamada **resul.** El programa deberá llamarse **TAREA405.**

6. Haga un programa que separe los bits pares y nones de una variable de tipo palabra llamada **dato**. Los bits pares quedarán en una variable de un byte llamada **pares** y los bits nones en otra variable de un byte llamada **nones**. El programa deberá llamarse **TAREA406**.

7. Haga un programa que establezca el bit de paridad par para una localidad de memoria de un byte llamada **dato**. Si la suma de unos de los bits 0 a 6 es par pondrá el bit 7 en cero y si la suma es non pondrá el bit 7 en uno. El programa deberá llamarse **TAREA407.**

CAPÍTULO 5

INSTRUCCIONES DE CONTROL DE FLUJO

Los programas que se han desarrollado hasta ahora se ejecutan en forma secuencial, esto es, el programa inicia su ejecución con la primera instrucción y continúa de arriba hacia abajo ejecutando cada una de las instrucciones hasta llegar a la última. En la mayoría de los casos, sin embargo, se requiere que el programa ejecute una serie de instrucciones dependiendo de sí una condición se cumple o de que ejecute una serie de instrucciones en forma repetitiva. Este tipo de programas se puede implementar mediante las instrucciones de control de flujo. Las instrucciones de control de flujo se clasifican en instrucciones de salto incondicional, salto condicional y repetitivas.

Instrucción De Salto Incondicional

La instrucción de salto incondicional hace que el control de flujo del programa salte a la instrucción cuya dirección está especificada por su operando.

jmp *dirección*

La instrucción **jmp** hace que la ejecución del programa continué en la dirección especificada por el operando.

Sintaxis:

jmp *etiqueta*
jmp *regW|memW*
jmp *memDW*

La dirección puede ser una etiqueta o estar en un registro o localidad de memoria. En el caso de usar un registro o localidad de memoria de una palabra, el salto es a una instrucción que se encuentra en el mismo segmento y el valor representa el desplazamiento de la instrucción con respecto al segmento. En el caso de que la dirección esté en una localidad de memoria de tipo palabra doble, el salto es a otro segmento y el valor es el segmento:desplazamiento de la instrucción.
La instrucción **jmp** no afecta a las banderas.

Instrucciones De Salto Condicional

Las instrucciones de salto condicional hacen que el control de flujo del programa salte a la instrucción cuya dirección está especificada por su operando si se cumple una condición dada.

***j**cond* *dirección*

Cambian la dirección de la siguiente instrucción a ser ejecutada dependiendo del valor de ciertas banderas .

Sintaxis:

j*cond* *etiqueta*

Todas las instrucciones de salto condicional operan en forma similar. **j***cond* es el mnemónico de la instrucción y *etiqueta* es la etiqueta de la dirección donde está la instrucción a la que va a saltar el programa. Los mnemónicos de las instrucciones de salto condicional pueden clasificarse en tres grupos: En la tabla 5-1a están los saltos cuya condición de salto es el resultado de una comparación de dos números no signados; en la tabla 5-1b se muestran los saltos cuya condición de salto es el resultado de una comparación de dos números signados; mientras que en la tabla 5-1c están los saltos cuya condición de salto es el resultado de una bandera o el registro **CX**. Aunque en las instrucciones de salto de las tablas 5-1a y 5-1b se dice que la condición de salto es el resultado de una comparación, la instrucción utiliza el valor de una o dos banderas para ejecutar o no el salto. Los valores de las banderas que producen el salto se muestran en la tercera columna en las tablas.

Tabla 5-1a. Saltos si la condición es una comparación no signada

Instrucción	Salta si...	Banderas
ja | **jnbe**	mayor | no menor o igual	**C** = 0 & **Z** = 0
jae | **jnb**	mayor o igual | no menor	**C** = 0
jb | **jnae**	menor | no mayor o igual	**C** = 1
jbe | **jna**	menor o igual | no mayor	**C** = 1 | **Z** = 1
je	igual	**Z** = 1
jne	diferente	**Z** = 0

Tabla 5.1b. Saltos si la condición es una comparación signada

Instrucción	Salta sí ...	Banderas
jg | **jnle**	mayor | no menor o igual	**S** = **O** & **Z** = 0
jge | **jnl**	mayor o igual | no menor	**S** = **O**
jl | **jnge**	menor | no mayor o igual	**S** ≠ **O**
jle | **jng**	menor o igual | no mayor	**S** ≠ **O** | **Z** = 1
je	igual	**Z** = 1
jne	diferente	**Z** = 0

Tabla 5.1c. Saltos si la condición es un estado de una bandera o el registro CX.

Instrucción	Salta sí ...	Banderas
jc	acarreo	**C** = 1
jo	sobreflujo	**O** = 1
jp | **jpe**	paridad | paridad par	**P** = 1
jpo | **jnp**	paridad non | no paridad	**P** = 0
js	signo	**S** = 1
jz	cero	**Z** = 1
jnc	no acarreo	**C** = 0
jno	no sobreflujo	**O** = 0
jns	no signo	**S** = 0
jnz	no cero	**Z** = 0
jcxz	si **CX** es cero	-

Algunos mnemónicos generan el mismo código máquina como **ja** y **jnbe** o **je** y **jz**.

Una restricción que tienen todos los saltos condicionales es de que la dirección a la que saltan debe de estar como máximo 128 bytes hacia atrás o 127 bytes hacia adelante del primer byte de la siguiente instrucción a la instrucción de salto. Por ejemplo, suponga que en el siguiente código, la instrucción a la que se desea saltar se encuentra más allá del rango de los -128 a 127 bytes:

```
        cmp     dx,1            ; if(DX == 1)
        je      error           ; goto error
        ...
error:                          ; Esta dirección está más
                                ; allá de los 127 bytes
```

El ensamblador nos desplegará el siguiente mensaje:

```
**Error** Relative jump out of range by n bytes
(**Error** Salto relativo fuera de rango por n bytes)
```

Una solución al problema anterior es recodificar el código anterior como:

```
        cmp     dx,1            ; if(DX != 1)
        jne     continua        ; goto continua
                                ; else
        jmp     error           ; goto error
continua:
        ...
error:                          ; Esta dirección está más
                                ; allá de los 127 bytes
```

Note que en este caso la condición de salto es la contraria a la del código anterior.

Ejemplos sobre instrucciones de salto

1. El siguiente programa encuentra el mayor de tres números signados almacenados en variables de tipo palabra.

```
;*********************************************************
; MAYOR3.ASM
;
; Este programa encuentra el mayor de tres datos signados
; almacenados en variables de una palabra. El pseudocódigo
; de este programa es:
;
;   AX = dato1
;   if(AX > dato2) goto sigcmp
;   AX = dato2
;
;sigcmp:
;   if(AX > dato3) goto fincmp
;   AX = dato3
;
;   mayor = AX
```

```
;*******************************************************
;****** CÓDIGO DE INICIO ***********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA *****************************

        dataseg
codsal  db      0
dato1   dw      ?
dato2   dw      ?
dato3   dw      ?
mayor   dw      ?

;****** CÓDIGO DEL PROGRAMA ********************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos

        mov     ax, [dato1]     ; AX = dato1
        cmp     ax, [dato2]     ; if(AX > dato2)
        jg      sigcmp          ;    goto sigcmp
        mov     ax, [dato2]     ; AX = dato2

sigcmp: cmp     ax, [dato3]     ; if(AX > dato3)
        jg      fincmp          ;    goto fincmp
        mov     ax, [dato3]     ; AX = dato3

fincmp: mov     [mayor], ax     ; mayor = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ******************************

        end     inicio
```

2. El siguiente programa suma los números enteros desde 1 hasta nfinal.

```
;*********************************************************
; SERIE1.ASM
;
; Este programa suma los números enteros de 1 hasta nfinal.
; El pseudocódigo de este programa es:
;
;   AX = 0
;   CX = nfinal
;
;   while(CX > 0)
;   {
;      AX += CX
;      CX–
;   }
;
;   suma = AX
;*********************************************************

;****** CÓDIGO DE INICIO ***********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA *****************************

        dataseg
codsal  db      0
nfinal  dw      ?
suma    dw      ?

;****** CÓDIGO DEL PROGRAMA ********************************

        codeseg
inicio:
        mov     ax, @data       ; inicializa el
        mov     ds, ax          ; segmento de datos

        xor     ax, ax          ; AX = 0
        mov     cx, [nfinal]    ; CX = nfinal

while:  jcxz    endwhi          ; while(CX > 0)
                                ; {
        add     ax, cx          ;    AX += CX
        dec     cx              ;    CX--
        jmp     while           ; }

endwhi:
        mov     [suma], ax      ; suma = AX
```

```
salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ******************************

        end     inicio
```

Ejercicios sobre instrucciones de salto

1. Cree un programa que obtenga el mayor de dos números signados del tipo palabra doble.

2. Cree un programa que multiplique dos variables tipo palabra no signadas. El resultado será una variable de tipo palabra doble. Utilice el algoritmo de sumas y corrimientos.

Instrucciones Repetitivas

El ensamblador del 8086 posee tres instrucciones especiales que permiten la construcción de ciclos.

loop *dirección*

Decrementa **CX** y luego salta si **CX** no es 0.

Sintaxis:

loop *etiqueta*

Esta instrucción decrementa el contenido del registro **CX** en uno, si el valor que queda en **CX** es diferente de cero, entonces la instrucción salta a la dirección especificada por *etiqueta*, la cual no debe encontrarse más allá de los 126 bytes hacia atrás o de los 127 bytes hacia adelante.

Esta instrucción se utiliza para crear ciclos que se repiten el número de veces especificado por el registro **CX**. Como la instrucción **loop** decrementa **CX** antes de probar si vale cero. Si **CX** vale cero al principio, al decrementar **CX** tomará el valor de 65535 y por lo tanto el ciclo ejecutará 65536 veces. Para prevenir esto preceda el ciclo con la instrucción **jcxz** como se muestra en el siguiente código.

```
        jcxz    enddo           ; if(CX == 0) goto enddo

do:                             ; do
                                ; {
        ...                     ;    instrucciones dentro
del
                                ;    ciclo
        loop    do              ; }
                                ; while(CX > 0)

enddo:
```

la instrucción **loop** no modifica las banderas.

loope | loopz *dirección*

Decrementa **CX** y luego salta si **CX** no es 0 y la bandera de cero **Z** vale 1.

Sintaxis:

loope | **loopz** *etiqueta*

Los mnemónicos **loope** y **loopz** representan la misma instrucción. Esta instrucción decrementa el contenido del registro **CX** en uno, si el valor que queda en **CX** es diferente de cero y la bandera de cero **Z** vale 1, presumiblemente puesta por una comparación previa, entonces la instrucción salta a la dirección especificada por *etiqueta*, la cual no debe encontrarse más allá de los 126 bytes hacia atrás o 127 bytes hacia adelante.

loopne | **loopnz** *dirección*

Decrementa **CX** y luego salta si **CX** no es 0 y la bandera de cero **Z** vale 0.

Sintaxis:

loopne | **loopnz** *etiqueta*

Los mnemónicos **loope** y **loopz** representan la misma instrucción. Esta instrucción decrementa el contenido del registro **CX** en uno, si el valor que queda en **CX** es diferente de cero y la bandera de cero **Z** vale 0, presumiblemente puesta por una comparación previa, entonces la instrucción salta a la dirección especificada por *etiqueta*, la cual no debe encontrarse más allá de los 126 bytes hacia atrás o 127 bytes hacia adelante.

Ejemplo sobre instrucciones repetitivas

El siguiente programa es una modificación del ejemplo 2 sobre las instrucciones de salto. Suma los números enteros desde 1 hasta nfinal.

```
;*************************************************************
; SERIE2.ASM
;
; Este programa suma los números enteros de 1 hasta nfinal.
; Esta versión utiliza la instrucción repetitiva loop. El
; pseudocódigo de este programa es:
;
;   AX = 0
;   CX = nfinal
;
;   if(CX == 0) goto endo
;
;   do
;   {
;      AX += CX
;   }
;   while(--CX > 0)

; enddo:
;   suma = AX
;*************************************************************

;****** CÓDIGO DE INICIO ************************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ******************************

        dataseg
codsal  db      0
nfinal  dw      ?
suma    dw      ?

;****** CÓDIGO DEL PROGRAMA *********************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos
```

```
        xor     ax, ax          ; AX = 0
        mov     cx, [nfinal]    ; CX = nfinal

        jcxz    endo            ; if(CX == 0) goto endo

do:                             ; do
                                ; {
        add     ax, cx          ;    AX += CX
        loop    do              ; }
                                  while(--CX > 0)
enddo:
        mov     [suma], ax      ; suma = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ********************************

        end     inicio
```

Ejercicio sobre instrucciones repetitivas

Modificar el ejercicio 3 sobre instrucciones lógicas para utilizar la instrucción repetitiva **loop**. El ejercicio 3 es hacer un programa que intercambie los bits de una variable de tipo palabra. El valor del bit 15 debe quedar en el bit 0, el valor del bit 14 debe quedar en el bit 1, etc. El resultado queda en la misma variable.

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documento PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

Problemas

1. Un año bisiesto es aquel año que es divisible entre 4 pero no es divisible entre 100 a menos que sea divisible entre 400 en cuyo caso si es bisiesto. Crea un programa que determine si un año almacenado en la variable **anho** es un año bisiesto o no. La respuesta 0 para no y 1 para sí quedará en la variable **resul**. Llame este programa **TAREA501**.

2. La secuencia de Fibonacci, es la secuencia de enteros

   0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

   llamados números de Fibonacci. Podemos observar que a partir del tercer número de Fibonacci, estos están formados por la suma de los dos anteriores. El n-ésimo número de Fibonacci está dado por:

   Fib(n) = n sí n = 0, 1
   Fib(n) = Fib(n-1) + Fib(n-2) sí n >= 2

   hacer un programa que calcule el e-nesimo número de Fibonacci. El valor de n está en una variable de un byte llamada **n** y su Fibonacci quedará en la variable de tipo palabra **fibn**. Llame este programa **TAREA502**.

3. Hacer un programa que divida una variable de tipo palabra doble entre una variable de tipo palabra. Utilice el algoritmo de restas repetidas. El dividendo estará en la variable **dvdndo**, el divisor en la variable **divsor**, el cociente y el residuo en las variables de tipo palabra **cocien** y **resduo**. Llame este programa **TAREA503**.

4. Hacer un programa que encuentre el valor de n tal que:

   $1^2 + 2^2 + 3^2 + ... + n^2 \leq m$

El valor de m estará en la variable de tipo palabra doble **limite**. El valor de n quedará en la variable de tipo palabra **raiz.** El programa deberá llamarse **TAREA504**.

5. Haga un programa que separe los bits pares y nones de una variable de tipo palabra llamada **dato**. Los bits pares quedarán en una variable de un byte llamada **pares** y los bits nones en otra variable de un byte llamada **nones**. Utilice la instrucción repetitiva **loop**. El programa deberá llamarse **TAREA505**.

6. Haga un programa que establezca el bit de paridad par para una localidad de memoria de un byte llamada **dato**. Si la suma de unos de los bits 0 a 6 es par pondrá el bit 7 en cero y si la suma es non pondrá el bit 7 en uno. Utilice la instrucción repetitiva **loop**. El programa deberá llamarse **TAREA506**.

CAPÍTULO 6

PROCEDIMIENTOS

Los programas que se han desarrollado hasta ahora contienen pocas líneas de código, una o dos páginas a lo sumo. Conforme los programas crecen en tamaño y complejidad se vuelve difícil y tedioso depurarlos. En este caso, resulta conveniente dividir el programa en varias partes llamadas procedimientos. Un **procedimiento** es una colección de instrucciones relacionadas que realiza una tarea específica. También un procedimiento puede contener un conjunto de instrucciones que deseamos que se ejecuten en varias partes del programa. Los procedimientos del lenguaje ensamblador tienen su contraparte en los lenguajes de alto nivel, por ejemplo, en el lenguaje C estos procedimientos se llaman funciones. Aunque en los lenguajes de alto nivel, el mecanismo empleado por los procedimientos para implementar su llamada, ejecución y regreso es transparente para el usuario, en ensamblador se requiere entender ese mecanismo. Un componente indispensable empleado por ese mecanismo es la pila del programa. La pila de un programa también se emplea para implementar el paso de parámetros a los procedimientos así como para crear las variables locales al procedimiento.

La Pila De Un Programa

Todo programa escrito en el lenguaje ensamblador del 8086 requiere de una pila. La pila se emplea para almacenar temporalmente direcciones y datos. En los programas hechos para el microprocesador 8086, una pila es un arreglo de palabras. El tamaño de la pila lo establece el programador mediante la directiva **stack** en el **código de inicio** de un programa. Tal como se estudió en el tema 3, la directiva **stack** cuya sintaxis es:

stack *tamPila*

crea una pila para el programa de tamaño *tampila*. El cargador del programa inicializa en forma automática el registro de segmento de pila **SS** a la dirección del segmento en que se creó la pila y el registro apuntador de pila **SP** al valor de *tamPila*. La figura 6.1 muestra el estado inicial de la pila de un programa al ser cargado a memoria para su ejecución (la pila se encuentra vacía).

Note que el registro apuntador de pila **SP** apunta a la palabra que está pasando el final de la pila. La pila difiere de los otros segmentos en la manera en que almacena los datos: empieza en la localidad más alta (la que tiene la dirección dada por **SS:*tamPila*-2**) y almacena los datos hacia las localidades más bajas de la pila (hacia **SS:0000**).

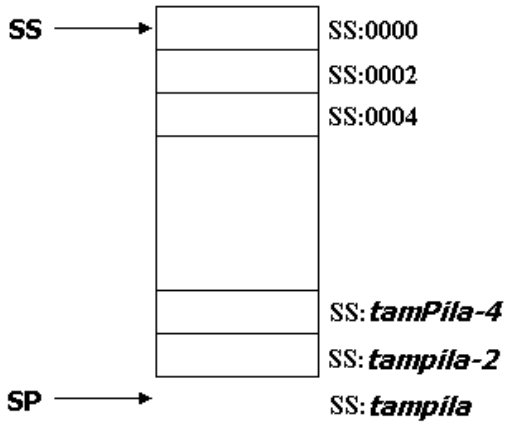


Figura 6-1

Instrucciones para el manejo de la pila

El ensamblador del 8086 posee un conjunto de instrucciones especiales para insertar y extraer datos en la pila:

push *fuente*

Transfiere el valor dado por *fuente* a la pila del programa.

Sintaxis:

push *regW|memW*

Donde *regW* puede ser cualquiera de los registros de propósito general o de segmento.

La instrucción **push** transfiere una palabra a la pila del programa. La instrucción **push** primero decrementa el valor del registro apuntador de pila **SP** en dos. A continuación copia el valor especificado por su operando a la localidad cuya dirección está dada por **SS:SP**. Note que esto hace que la pila crezca hacia las direcciones más pequeñas de memoria.

La instrucción **push** no afecta a las banderas.

pop *destino*

Extrae un valor del tope de la pila del programa y lo almacena en *destino*.

Sintaxis:

pop *regW|memW*

Donde *regW* puede ser cualquiera de los registros de propósito general o de segmento excepto el registro de segmento de código CS, que no está permitido. La instrucción **pop** extrae una palabra de la localidad cuya dirección está dada por **SS:SP** (el tope de la pila del programa) y lo almacena en la localidad especificada por el operando. A continuación incrementa el valor del registro apuntador de pila **SP** en dos.

La instrucción **pop** no afecta a las banderas.

En la figura 6.2 se muestran los estados de la pila después de insertar los valores de los registros **AX** y **BX** mediante las instrucción **push** y en la figura 6.3 se muestran los estados de la pila después de extraer esos valores de la pila mediante la instrucción **pop** al ejecutar el siguiente fragmento de código:

```
mov    ax, 0123h
mov    bx, 4567h
push   ax
push   bx
pop    bx
pop    ax
```

Note en la figura 6.3a que al extraer el valor 4567h de la pila, éste no se elimina físicamente de la pila. Sin embargo no hay una garantía de que ese valor permanezca en la pila. Si en ese momento insertamos un nuevo valor en la pila, éste se escribirá en la localidad **SS**:*tamPila-4* sobrescribiendo el valor de 4567h.

pushf

Transfiere el registro de banderas a la pila del programa.

Sintaxis:

pushf

La instrucción **pushf** transfiere el contenido del registro de banderas a la pila del programa de la misma manera en la que opera la instrucción **push**.

La instrucción **pushf** no afecta a las banderas.

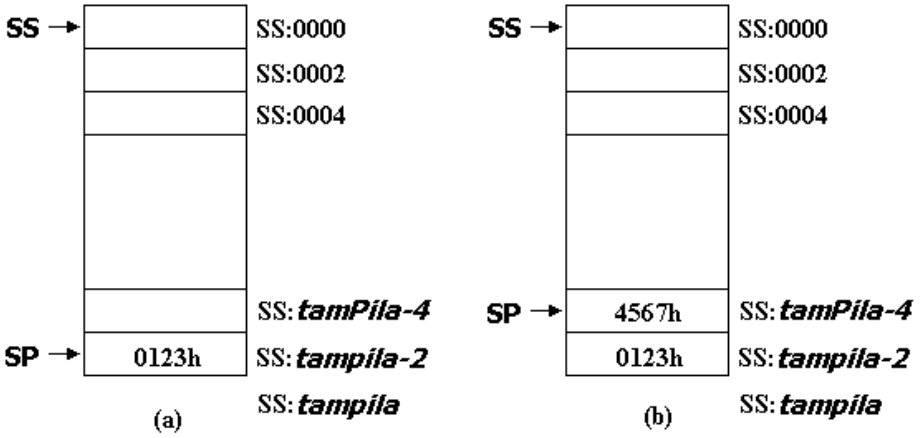


Figura 6-2

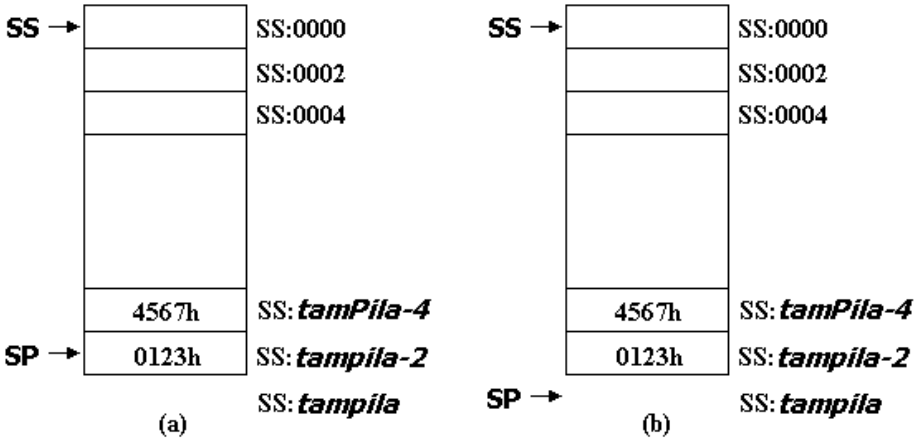


Figura 6-3

popf

Extrae un valor del tope de la pila del programa y lo almacena en registro de banderas.

Sintaxis:

popf

La instrucción **popf** extrae una palabra del tope de la pila y la inserta en el registro de banderas de la misma manera en que trabaja la instrucción **pop**. Normalmente se hace esto para extraer el estado de las banderas almacenado previamente por una instrucción **pushf**, o para fijar las banderas a nuevos valores.

La instrucción **popf** afecta todas las banderas.

Procedimientos

Como ya se mencionó anteriormente, un procedimiento es una colección de instrucciones que realizan una tarea específica: Sumar dos valores, desplegar un carácter en la pantalla, leer un carácter del teclado, inicializar un puerto, etc.

Dependiendo de su extensión y complejidad, un programa puede contener uno, algunos o inclusive cientos de procedimientos. Para emplear un procedimiento en un programa se requiere definir el procedimiento y llamarlo. Al definir a un procedimiento escribimos las instrucciones que contiene. Al llamar al procedimiento transferimos el control del flujo del programa al procedimiento para que sus instrucciones se ejecuten.

Definición de un procedimiento

La sintaxis de la definición de un procedimiento es la siguiente:

proc *nomProc*
proposición
[*proposición*]
...
endp [*nomProc*]

Las directivas **proc** y **endp** marcan el inicio y el final del procedimiento. No generan código.

nomProc es el nombre del procedimiento y etiqueta la primera instrucción del procedimiento.

Al menos una de las proposiciones de un procedimiento es la instrucción **ret**, la cual se describe más adelante.

Llamada a un procedimiento

La llamada a un procedimiento normalmente tiene la siguiente forma:

call *nomProc*

La instrucción **call** se describe más adelante. *nomProc* es el nombre que se le dio al procedimiento al definirlo.

Ejemplo sobre procedimientos

El siguiente programa suma dos variables de tipo palabra doble y guarda el resultado en una variable de tipo palabra doble. Este programa utiliza un procedimiento para efectuar la suma.

```
;*********************************************************
; SUM2DW_P.ASM
;
; Este programa suma dos variables de tipo palabra doble
; y guarda el resultado en una variable de tipo palabra
; doble. Este programa utiliza un procedimiento para
; efectuar la suma.
;
; El pseudocódigo de este programa es:
;
;    DX:AX = dato1
;    CX:BX = dato2
;
;    suma2dw()
;
;    resul = DX:AX
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
dato1   dd      ?
dato2   dd      ?
resul   dd      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data               ; Inicializa el
        mov     ds, ax                  ; segmento de datos

        mov     ax, [word dato1]        ; DX:AX = dato1
        mov     dx, [word dato1+2]
        mov     bx, [word dato2]        ; CX:BX = dato2
        mov     cx, [word dato2+2]
```

```
; Llama al procedimiento para efectuar la suma
        call    suma2dw                 ; suma2dw()

        mov     [word resul], ax        ; resul = DX:AX
        mov     [word resul+2], dx

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h


;****** PROCEDIMIENTOS ************************************

;********************************************************
; SUMA2DW
;
; Este procedimiento suma dos variables de tipo
; palabra doble.
;
; Parámetros:
;
;       DX:AX = Primer sumando
;       CX:BX = Segundo sumando
;
; Regresa:
;
;       DX:AX = Suma
;
; El pseudocódigo de este procedimiento es:
;
;   DX:AX += CX:BX
;********************************************************
proc    suma2dw
        add     ax, bx                  ; DX:AX += CX:BX
        adc     dx, cx

        ret                             ; Regresa del
                                        ; procedimiento
endp    suma2dw

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

Mecánica de la llamada a un procedimiento

Para implementar la llamada a un procedimiento y el regreso de éste, se tiene el siguiente mecanismo:

1. Cuando el programa lee de memoria y decodifica la instrucción que corresponde a la llamada a un procedimiento, el registro apuntador de instrucciones **IP** tiene el desplazamiento de la siguiente instrucción a ejecutar. Por ejemplo, en el listado parcial del archivo SUM2DW_P.LST que se muestra a continuación se puede ver que el desplazamiento de la siguiente instrucción a la llamada al procedimiento **suma2dw** es de 0017h.

```
31
32 0005  A1 0001r              mov     ax, [word dato1]     ; DX:AX = dato1
33 0008  8B 16 0003r           mov     dx, [word dato1+2]
34 000C  8B 1E 0005r           mov     bx, [word dato2]     ; CX:BX = dato2
35 0010  8B 0E 0007r           mov     cx, [word dato2+2]
36
37 0014  E8 000E               call    suma2dw              ; Llama al procedimiento
38                                                          ; para efectuar la suma
39
40 0017  A3 0009r              mov     [word resul], ax     ; resul = DX:AX
41 001A  89 16 000Br           mov     [word resul+2], dx


64 0025                 proc    suma2dw
65 0025  03 C3                  add     ax, bx               ; DX:AX += CX:BX
66 0027  13 D1                  adc     dx, cx
67
68 0029  C3                     ret                          ; Regresa del
69                                                          ; procedimiento
70 002A                 endp    suma2dw
```

2. La instrucción **call** inserta el valor de este desplazamiento en la pila del programa y carga en el registro apuntador de instrucciones el valor de la dirección etiquetada con el nombre del procedimiento. En este ejemplo cargaría a **IP** con el valor de 0025h. Esto hace que el control de flujo del programa se transfiera a la primera instrucción del procedimiento.

3. Al ejecutar una instrucción **ret**, ésta extrae del tope de la pila el valor previamente almacenado por la instrucción **call** y lo carga en el registro apuntador de instrucciones. Como este valor es el desplazamiento de la instrucción que va después de la llamada al procedimiento, ésta será la siguiente instrucción a ejecutarse. Esto es lo que se conoce como el regreso del procedimiento. En este ejemplo **IP** se carga con el valor de 0017h.

La descripción de las instrucciones **call** y **ret**, se muestran a continuación.

call *destino*

Llama a un procedimiento.

Sintaxis:

call *etiqueta*
call *regW|memW*
call *memDW*

Si el procedimiento a llamar se encuentra en el mismo segmento en el que se encuentra su llamada, la instrucción **call** inserta en la pila el desplazamiento de la siguiente instrucción después de la llamada al procedimiento y luego carga en el registro apuntador de instrucciones el desplazamiento dado por *destino.* Si el procedimiento está en un segmento diferente al en que está su llamada, la instrucción **call** inserta en la pila el segmento:desplazamiento de la siguiente instrucción después de la llamada al procedimiento y luego carga en el registro apuntador de instrucciones y en el registro de segmento de código el segmento:desplazamiento dado por *destino.*

En la mayoría de los programas, *destino* es una etiqueta que marca el inicio del procedimiento. Sin embargo también puede ser un registro o localidad de memoria que contenga la dirección del procedimiento.

ret

Regresa de un procedimiento

Sintaxis:

ret

Si el procedimiento del que se va a regresar se encuentra en el mismo segmento en el que se encuentra su llamada, la instrucción **ret** extrae de la pila el desplazamiento de la siguiente instrucción después de la llamada al procedimiento, el cual fue insertado en la pila por la llamada al procedimiento y lo carga en el registro apuntador de instrucciones **IP**. Si el procedimiento está en un segmento diferente al en que está su llamada, la instrucción **ret** extrae de la pila el segmento:desplazamiento de la siguiente instrucción después de la llamada al procedimiento y lo carga en los registros de segmento de código **CS** y apuntador de instrucciones **IP**, respectivamente.

Consideraciones sobre el diseño de un procedimiento

Un procedimiento bien diseñado debe llenar los siguientes requisitos:

- Debe hacer una sola tarea.
- Debe ser tan pequeño como sea posible y tan largo como sea necesario. Su listado no debiera de exceder una o dos páginas.

- Debe contener un comentario describiendo su propósito, sus datos de entrada y de salida.

- Debe entenderse por sí solo sin necesidad de saber que hace el programa completo.

- El comportamiento interno del procedimiento debe ser transparente para el usuario. Esto se llama **encapsulamiento de la información**. El usuario sólo debe saber cómo llamar al procedimiento, esto es, qué datos se le deben suministrar y cómo regresa el resultado. A esto se le llama la **interfase del procedimiento**.

  Para lograr esa transparencia se requiere que:

  - Un procedimiento, desde el punto de vista del usuario, funcione como lo haría una instrucción del microprocesador. Por lo tanto un procedimiento sólo puede alterar los registros en los que recibe los datos, los registros en los que regresa el resultado o el registro de banderas.

  - Un procedimiento tampoco debe utilizar variables globales ni para recibir datos o regresar un resultado, ni para almacenar temporalmente resultados intermedios.

Uso de los registros en un procedimiento

Un procedimiento puede utilizar los registros de propósito general de tres formas diferentes:

- Para recibir valores o direcciones del código que lo llama.

- Para regresar valores o direcciones al código que lo llama.

- Como variables locales del procedimiento o como registros de trabajo de las instrucciones del procedimiento.

En la sección anterior se mencionó que para que un procedimiento sea transparente al usuario, sólo debe modificar los registros que emplea para recibir datos o regresar resultados. ¿Cómo puede entonces emplear los registros como variables locales o registros de trabajo sin modificar su valor? La respuesta es que, si un procedimiento emplea un registro como variable local o registro de trabajo, debe guardar el valor original de ese registro en la pila del programa al entrar al procedimiento y recuperar su valor antes de salir del procedimiento.

Ejemplo sobre procedimientos que usan registros como variables locales o registros de trabajo

El siguiente programa intercambia los nibles más significativo y menos significativo de una variable de un byte. El programa utiliza un procedimiento para intercambiar los nibles.

Como el procedimiento **swapnibl** empleado para intercambiar los nibles requiere usar el registro **CX** como registro de trabajo de la instrucción **rol** el valor inicial de ese registro al entrar al procedimiento se almacena en la pila con una instrucción **push**. Antes de regresar del procedimiento, se restaura el valor original del registro **CX** mediante la instrucción **pop**.

```
;*********************************************************
; SWAPNIBL2.ASM
;
; Este programa intercambia los MSN y LSN de una variable
; de un byte. El resultado queda en la misma variable.
; Este programa utiliza un procedimiento para intercambiar
; los nibles.
;
; El pseudocódigo de este programa es:
;
;    AL = dato
;    swapnibl()
;    dato = AL
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
dato    db      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos

        mov     al, [dato]      ; AL = dato

; Llama al procedimiento para intercambiar los nibles
        call    swapnibl        ; swapnibl()

        mov     [dato], al      ; dato = AL

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h
```

```
;****** PROCEDIMIENTOS ************************************

;*************************************************************
; SWAPNIBL
;
; Este procedimiento intercambia los MSN y LSN de una
; variable de un byte.
;
; Parámetro:
;
;       AL = dato
;
; Regresa:
;
;       AL = dato con los nibles intercambiados.
;
; El pseudocódigo de este procedimiento es:
;
;     CL = 4
;     rol al, 4
;*************************************************************
proc    swapnibl
        push    cx              ; Preserva CX

        mov     cl, 4
        rol     al, cl

        pop     cx              ; Recupera CX
        ret
endp    swapnibl

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

Etiquetas locales a un procedimiento

Los identificadores que hemos empleado para etiquetar las direcciones de las instrucciones son etiquetas globales. Una **etiqueta global** es conocida en todo el programa, incluyendo en los procedimientos. Si un procedimiento definiera una etiqueta global, ésta también sería conocida por todo el programa. Como las etiquetas globales deben ser únicas, los nombres de las etiquetas de un procedimiento deben ser diferentes entre sí y diferentes a las de otros procedimientos y de las del resto del programa. Lo anterior viola el principio transparencia mencionado en la sección pasada ya que el programador debería conocer los nombres de las etiquetas empleadas en los diferentes procedimientos para no repetirlos.

Para evitar este problema Turbo Assembler nos permite definir etiquetas locales. El ámbito de una etiqueta local se extiende desde su definición hacia adelante y hacia atrás hasta la siguiente etiqueta

no local. Como las etiquetas definidas con la directiva **proc** son globales, los procedimientos están limitados por etiquetas globales, la del inicio del procedimiento y la de del inicio del siguiente procedimiento. Por lo tanto, las etiquetas locales son visibles sólo dentro del código del procedimiento y podemos reutilizarlas en otro lado sin problema.

La sintaxis de una etiqueta local es similar a la de las etiquetas globales sólo que empieza con dos arrobas, por ejemplo @@while, @@endwhi, etc.

Ejemplo sobre procedimientos que usan etiquetas locales

El siguiente programa suma los números enteros desde 1 hasta nfinal. El programa utiliza un procedimiento para obtener la suma.

```
;*********************************************************
; SERIE3.ASM
;
; Este programa suma los números enteros de 1 hasta nfinal
; Versión que utiliza un procedimiento para obtener la
; suma.
;
; El pseudocódigo de este programa es:
;
;   AX = nfinal
;
;   serie()
;
;   suma = AX
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
nfinal  dw      ?
suma    dw      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos
```

```
        mov     ax, [nfinal]    ; AX = nfinal

        call    serie

        mov     [suma], ax      ; suma = AX
salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** PROCEDIMIENTOS ************************************

;*******************************************************
; SERIE
;
; Este procedimiento suma los números de 1 hasta nfinal
;
; Parámetro:
;
;        AX = nfinal
;
; Regresa:
;
;        AX = suma
;
; El pseudocódigo de este procedimiento es:
;
;   CX = nFinal
;   if(CX == 0) goto enddo
;   AX = 0
;
;   do
;   {
;      AX+= CX
;   }
;   while(--CX > 0)
;*******************************************************
proc    serie
        push    cx              ; Preserva CX
        mov     cx, ax          ; CX = nfinal

        jcxz    @@enddo         ; if(CX == 0) goto enddo
        xor     ax, ax          ; AX = 0

@@do:                           ; do {
        add     ax, cx          ;    AX += CX
        loop    @@do            ; } while(--CX > 0)
@@enddo:

        pop     cx              ; Recupera CX
        ret
endp    serie
```

```
;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

Ejercicios sobre procedimientos

1. Cree un procedimiento que obtenga el máximo común divisor de dos números no signados de tipo palabra. El procedimiento recibe los dos números en los registros **AX** y **DX** y regresa el máximo común divisor de los números en **AX.**

2. Cree un procedimiento que obtenga el mínimo común múltiplo de dos números no signados de tipo palabra. El procedimiento recibe los dos números en los registros **AX** y **DX** y regresa el mínimo común múltiplo de los números en **DX:AX.**

Modo de direccionamiento base

En este modo de direccionamiento el cálculo de la dirección efectiva del dato emplea uno de los registros **BX** o **BP**. Las referencias a **BX** son desplazamientos con respecto al registro de segmento de datos **DS**, mientras que las referencias a **BP** son desplazamientos con respecto al registro de segmento de pila **SS**.

El modo de direccionamiento base tiene la siguiente sintaxis:

[bx+*n*]
[bx-*n*]
[bp+*n*]
[bp-*n*]

En los dos primeros casos el desplazamiento del dato con respecto a **DS** está dado por el valor de **BX** más o menos *n* bytes. En los dos últimos casos el desplazamiento del dato con respecto a **SS** está dado por el valor de **BP** más o menos *n* bytes.

El direccionamiento base utilizando el registro **BX** se emplea normalmente para localizar campos dentro de una estructura de datos. Por ejemplo:

```
        mov     bx, offset dato ; Apunta a la estructura
                                ; dato
        mov     ax, [bx+5]      ; Obtén el valor que se
                                ; encuentra a 5 bytes
                                ; del inicio de dato
        inc     [word bx+3]     ; Incrementa el valor de
                                ; tipo palabra que se
                                ; encuentra a 3 bytes del
                                ; inicio de dato
```

En la última instrucción del ejemplo anterior fue necesario indicar que la localidad de memoria apuntada por bx+3 es una localidad de tipo palabra. De otra manera la instrucción **inc** no podría determinar si la localidad de memoria era de un byte o de una palabra.

El direccionamiento base utilizando el registro **BP** se emplea normalmente para referenciar a variable locales en la pila del programa como se verá en la siguiente sección.

Variables locales en la pila

Los procedimientos en ensamblador utilizan por lo general los registros de propósito general para variables locales. En la mayoría de los casos los registros con que disponemos: **AX**, **BX**, **CX**, **DX**, **SI**, **DI**, son suficientes para nuestras necesidades de almacenamiento temporal. Sin embargo si llegáramos a necesitar de más espacio de almacenamiento temporal podemos emplear el mecanismo usado por los lenguajes de alto nivel para ubicar las variables locales de un procedimiento. Los lenguajes de alto nivel crean a las variables locales de un procedimiento en la pila del programa.

El mecanismo empleado para crear las variables en la pila es el siguiente:

1. En la figura 6-4, se muestra el estado de la pila inmediatamente después de entrar al procedimiento. Note que en la localidad apuntada por el tope de la pila se encuentra la dirección de regreso del procedimiento *dirRegProc*.

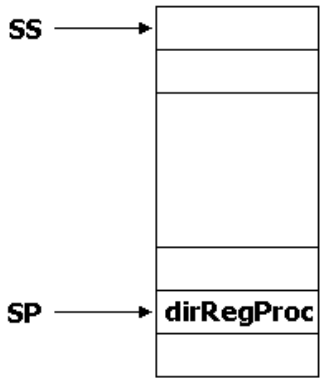


Figura 6-4

2. Una vez dentro del procedimiento, lo primero que se hace es preservar el valor del registro apuntador de base, **BP**, insertándolo en la pila. A continuación se hace que **BP** apunte al tope de la pila, ya que a partir de esa posición se ubicarán las variables locales. Las instrucciones para obtener lo anterior son:

```
push    bp
mov     bp, sp
```

El estado de la pila, después de estas operaciones se muestra en la figura 6-5.

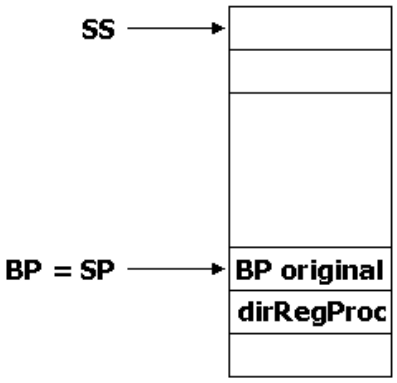


Figura 6-5

3. Se crean las variables locales decrementando **SP** en el número de bytes requeridos por las variables. Por ejemplo, para crear dos variables locales de tipo palabra, la instrucción sería:

```
sub sp, 4                ; 2 variables de 2 bytes
                         ; c/u = 4 bytes
```

El estado de la pila después de la instrucción se muestra en la figura 6-6. Las variables locales se identifican en la figura con los nombres: **varLoc1** y **varLoc2.** Sin embargo el acceso a las variables es mediante su dirección las cuales son: **BP**-2 y **BP**-4, respectivamente.

El mecanismo empleado para destruir las variables es el siguiente:

1. Se incrementa **SP** en el número de bytes requeridos por las variables locales. Esto se puede hacer mediante la instrucción:

```
mov     sp, bp           ; Equivale en este caso
                         ; a: add sp, 4
```

Esto nos deja la pila como en la figura 6-5.

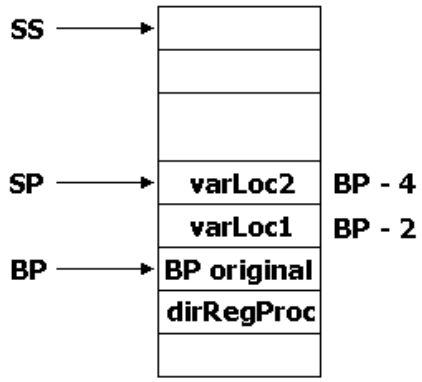


Figura 6-6

2. Se recupera el valor de BP de la pila, haciendo que **SP** apunte de nuevo a la dirección de regreso del procedimiento, mediante la instrucción:

```
pop     bp
```

   El estado final de la pila se muestra en la figura 6-4.

Ejemplo sobre variables locales en la pila

El siguiente procedimiento reduce una fracción a su expresión más simple. Por ejemplo el número 12/60 se reduce a 1/5, dividiendo el numerador y denominador entre el máximo común divisor. El procedimiento llama al procedimiento desarrollado en el ejercicio 1 de la sección anterior para encontrar el máximo común divisor.

```
;*********************************************************
; REDFRAC
;
; Este procedimiento reduce una fracción a su mínima
; expresión
;
; Parámetros:
;
;       AX = numerador
;       DX = denominador
;
; Regresa:
;
;       AX = numerador reducido
;       DX = denominador reducido
;
; El pseudocódigo para este procedimiento es:
;
```

```
;   BX = mcd(numerador, denominador)
;
;   numerador /= BX
;   denominador /= BX
;
;*******************************************************
proc    redfrac
        push    bp              ; Preserva BP
        mov     bp, sp          ; Crea variables
        sub     sp, 4           ; locales en la pila
        push    bx              ; Preserva BX

;num = [BP-2], den = [BP-4]
        mov     [bp-2], ax      ; num = numerador
        mov     [bp-4], dx      ; den = denominador

        call    mcd             ; BX = mcd(numerador,
                                ;          denominador)
        mov     bx, ax
        xor     dx, dx          ; num = numerador/BX
        mov     ax, [bp-2]
        div     bx
        mov     [bp-2], ax

        mov     ax, [bp-4]      ; DX = denominador/BX
        div     bx
        mov     dx, ax
        mov     ax, [bp-2]      ; AX = num

        pop     bx              ; Restaura BX
        mov     sp, bp          ; Elimina variables locales
        pop     bp              ; Restaura BP
        ret
endp    redfrac
```

Variables locales en la pila usando la directiva local

Turbo Assembler permite simplificar un poco el manejo de las variables locales permitiendo emplear identificadores locales para referenciar a las variables en lugar de emplear el direccionamiento base. Para ello se emplea la directiva **local**. La directiva **local** le asigna a cada variable local creada en la pila del programa un identificador local. La sintaxis de esta directiva es:

local *nomVar*: *tipo*[, *nomVar*: *tipo*]... [**=** *tamVarsLoc*]

donde *nomVar* es el nombre de la variable y *tipo* es su tipo: **byte**, **word**, **dword**, **qword**, etc. *tamVarsLoc* es un símbolo que toma el valor del número de bytes ocupado por las variables locales. Una vez que se ha empleado la directiva **local** se puede hacer referencia a las variables locales mediante sus nombres como si fueran variables globales.

Ejemplo sobre variables locales en la pila usando la directiva local

El siguiente procedimiento es una modificación del ejemplo anterior en el que se utiliza la directiva **local** para darle nombres locales a las variables locales creadas en la pila.

```
;*********************************************************
; REDFRAC2
;
; Este procedimiento reduce una fracción a su mínima
; expresión
;
; Parámetros:
;
;       AX = numerador
;       DX = denominador
;
; Regresa:
;
;       AX = numerador reducido
;       DX = denominador reducido
;
; El pseudocódigo para este procedimiento es:
;
;   numerador /= BX
;   denominador /= BX
;*********************************************************
proc    redfrac
; Declara las variables locales en la pila
        local   num: word, den: word = tamVarsLoc

        push    bp              ; Preserva BP
        mov     bp, sp          ; Crea variables
        sub     sp, tamVarsLoc  ; locales en la pila
        push    bx              ; Preserva BX

        mov     [num], ax       ; num = numerador
        mov     [den], dx       ; den = denominador

        call    mcd             ; BX = mcd(numerador,
        mov     bx, ax          ;          denominador)

        xor     dx, dx          ; num = numerador/BX
        mov     ax, [num]
        div     bx
        mov     [num], ax

        mov     ax, [den]       ; DX = denominador/BX
        div     bx

        mov     dx, ax
        mov     ax, [num]       ; AX = num
```

```
        pop     bx              ; Restaura BX
        mov     sp, bp          ; Elimina variables
                                ; locales

        pop     bp              ; Restaura BP
        ret
endp    redfrac
```

Ejercicios sobre variables locales en la pila

1. Modifique el procedimiento que encuentra el mínimo común múltiplo de la sección anterior para que el valor que se almacena en los registros **SI:DI** se guarde en una variable local en la pila. No utilice la directiva **local**

2. Modifique el procedimiento del ejercicio anterior para que emplee la directiva **local**.

Programación Modular

En los programas en ensamblador hechos hasta ahora, todo el código del programa reside en un sólo archivo. Sin embargo es posible y en muchos casos deseable dividir en código de un programa en varios archivos llamados módulos. Hay varias ventajas de emplear esta técnica llamada programación modular:

- Hay un límite en el tamaño de un archivo que un programa ensamblador puede manejar. Si el código del programa crece mucho, al dividirlo en módulos, tendremos archivos de menor tamaño. Como cada módulo puede ensamblarse por separado, esta técnica nos permite ensamblar programas más grandes.

- El proceso de ensamblado de un módulo es menor que el de un programa completo. Cuando se está en la etapa de construcción / depuración de un programa, por lo general uno escribe y depura un procedimiento a la vez, por lo que sólo se está haciendo cambios a un módulo, por lo que sólo ese módulo requiere de volver a ensamblarlo.

- Para obtener un programa ejecutable, los códigos objeto obtenidos al ensamblar cada módulo deben ligarse. Normalmente, cada módulo contiene un conjunto de procedimientos relacionados. Estos procedimientos pueden emplearse para formar diferentes programas, simplemente ligándolos con otros módulos.

- Podemos utilizar otros módulos escritos por terceros para resolver problemas específicos.

- Podemos escribir programas escritos en lenguaje de alto nivel y que utilicen módulos escritos en ensamblador.

Cada módulo en ensamblador puede contener declaraciones de constantes, declaraciones de variables y definiciones de procedimientos. La estructura de cada módulo es similar a la estructura de un programa sin módulos con las siguientes diferencias:

- Sólo uno de los módulos contiene las instrucciones que inicializan el segmento de datos y las instrucciones que terminan el programa y regresan el control al sistema operativo. A este módulo se le conoce como el módulo principal.

- Todos los módulos tienen código de inicio con las directivas: **ideal**, **dosseg**, **model**, pero sólo el módulo principal tiene la directiva **stack**.

- Todos los módulos tiene la directiva **end** que indica el final del código de ese módulo. Sin embargo sólo en el módulo principal la directiva **end** va seguida de la etiqueta que apunta al punto de entrada del programa. En los otros módulos la directiva **end** aparece por sí sola.

- Las constantes, variables y procedimientos declaradas y definidos en un módulo pueden ser conocidas sólo en el módulo donde fueron declaradas o pueden ser conocidas en otros módulos también. Los símbolos de las constantes, variables y procedimientos de un módulo que deseamos que se conozcan en otros módulos deben de exportarse utilizando la directiva **public**. La sintaxis de la directiva public es:

  ```
  public nomSimb[, nomSimb] ...
  ```

  donde *nomSimb* es el nombre de la constante, variable o procedimiento cuyo símbolo se va exportar. Sólo los símbolos de las constantes numéricas declaradas con la directiva = pueden exportarse. Los símbolos de las constantes declaradas con la directiva **equ** no pueden exportarse.

- Los símbolos de las constantes, variables y procedimientos declaradas y definidos en otro módulo y que fueron exportados con la directiva **public** tienen que importarse en el módulo en que se desea que sean conocidos también. Para importar un símbolo se utiliza la directiva **extrn**, cuya sintaxis es:

  ```
  extrn nomSimb: tipo[, nomSimb: tipo] ...
  ```

  donde *nomSimb* es el nombre de la constante, variable o procedimiento cuyo símbolo se va importar.

  tipo especifica el tipo de símbolo. Para los símbolos de las constantes numéricas declaradas con la directiva = el tipo es **abs**; para los símbolos de las variables a importar los tipos son: **byte**, **word**, **dword**, **qword**, etc. y para los procedimientos el tipo es **proc**.

En el siguiente ejemplo se muestra parte del código de un programa formado por dos módulos: MODULO.ASM y PRINCIPA.ASM. Este último contiene el módulo principal.

```
;*********************************************************
; MÓDULO.ASM
;
; Este módulo exporta algunos de sus símbolos e importa
; algunos símbolos del módulo con el programa principal.
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

        extrn   cte4: abs       ; importa cte4
        public  cte2            ; exporta cte2

cte1   equ    10               ; constante local
cte2   =      20               ; constante global

;****** VARIABLES DEL MÓDULO ******************************

        dataseg

        extrn   dato4: byte     ; importa dato4
        public  dato2           ; exporta dato2

dato1   db      ?               ; variable local
dato2   dw      ?               ; variable global

;****** CÓDIGO DEL MÓDULO *********************************

        codeseg

        extrn   proc4: proc     ; importa proc4
        public  proc2           ; exporta proc2

;****** PROCEDIMIENTOS ************************************

;*********************************************************
; PROC1
;
; Este procedimiento es local a este módulo.
;*********************************************************
proc    proc1
        ...
        ret
endp    proc1
```

```
;*********************************************************
; PROC2
;
; Este procedimiento es global.
;*********************************************************
proc    proc2
        ...
        ret
endp    proc2

;****** CÓDIGO DE TERMINACIÓN ***************************

        end
```

```
;*********************************************************
; PRINCIPA.ASM
;
; Este módulo que contiene el programa principal, también
; exporta algunos de sus símbolos e importa algunos
; símbolos del otro módulo.
;*********************************************************

;****** CÓDIGO DE INICIO *********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

        extrn   cte2: abs       ; importa cte4
        public  cte4            ; exporta cte2

cte3   equ    10               ; constante local
cte4   =      20               ; constante global

;****** VARIABLES DEL MÓDULO *****************************

        dataseg

        extrn   dato2: word     ; importa dato4
        public  dato4           ; exporta dato2

dato3   db      ?               ; variable local
dato4   db      ?               ; variable global

;****** CÓDIGO DEL MÓDULO ********************************

        codeseg

        extrn   proc2: proc     ; importa proc2
```

```
        public  proc4           ; exporta proc4

inicio:
        mov     ax, @data       ; Inicializa el segmento
        mov     ds, ax          ; de datos
        ...

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** PROCEDIMIENTOS ************************************

;*********************************************************
; PROC3
;
; Este procedimiento es local a este módulo.
;*********************************************************
proc    proc3
        ...
        ret
endp    proc3

;*********************************************************
; PROC4
;
; Este procedimiento es global.
;*********************************************************
proc    proc4
        ...
        ret
endp    proc4

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

Macros

Una **macro** es un conjunto de instrucciones asociadas a un identificador: el nombre de la macro. Este conjunto de instrucciones es invocado como una sola instrucción o **macroinstrucción**. Normalmente las instrucciones de una macro se repiten varias veces en un programa o aparecen frecuentemente en los programas. Para emplear una macro en un programa debemos de hacer dos cosas: Definir la macro e invocar la macro.

La **definición de una macro** establece el nombre al que se asocia la macro, el número y nombre de sus parámetros formales y qué instrucciones contiene la macroinstrucción. La sintaxis de la definición de una macro es la siguiente:

```
macro nomMacro [parForm[, parForm]...]
        proposición
       [proposición]
       ...
endm [nomMacro]
```

donde las directivas **macro** y **endm** marcan el inicio y el final de la definición de la macro. No generan código.

nomMacro es el nombre de la macro y se emplea al invocar la macro.

parForm es cada uno de los parámetros formales de la macro. Los parámetros permiten que una misma macro opere sobre datos distintos.

proposición son cada una de las instrucciones que forman la macroinstrucción.

Aunque la definición de una macro puede ir en cualquier parte de un programa, el lugar más recomendable para su localización es al principio de un archivo, antes de los segmentos de datos y de código.

Al encontrar una **invocación de una macro** el macroensamblador, Turbo Assembler por ejemplo, substituye la línea con la invocación por las proposiciones que contiene la definición de la macro. Este proceso de substitución se conoce **expansión de la macro**. La sintaxis de la invocación de la macro es similar a cualquier instrucción:

```
nomMacro [parReal[, parReal]...]
```

donde cada *parReal* es conocido como un **parámetro real** de la macro. Al expandirse la macro cada una de las ocurrencias de un parámetro formal en la definición de la macro se substituye por su correspondiente parámetro real.

Los parámetros reales pueden ser símbolos, mnemónicos, directivas, palabras reservadas, expresiones y números.

Ejemplo sobre macros

El siguiente programa ejecuta todas las rotaciones de un bit con una variable de tipo palabra doble. Las operaciones de rotación se implementan mediante macros. El parámetro formal **dest** de cada macro representa la localidad de memoria que contiene el dato a rotar. Al invocar a cada una de las macros el parámetro real de cada macro es el nombre de la variable cuyo dato se va a rotar.

```
;*********************************************************
; ROTACIO2.ASM
;
```

```
; Este programa ejecuta todas las rotaciones de un bit con
; una variable de tipo palabra doble. Las operaciones de
; rotación se implementan mediante macros que rotan el
; contenido de la localidad de memoria dada por el
; parámetro de la macro un bit.
;*********************************************************
;
;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** MACROS ********************************************

;*********************************************************
;
; RCLDW1
;
; Esta macro rota el contenido de dest un bit a la
; izquierda a través de la bandera de acarreo.
;*********************************************************
;
macro   rcldw1  dest
        rcl     [word dest], 1          ;; rcl dest, 1
        rcl     [word dest+2], 1
endm    rcldw1

;*********************************************************
;
; RCRDW1
;
; Esta macro rota el contenido de dest un bit a la
; derecha a través de la bandera de acarreo.
;*********************************************************
;
macro   rcrdw1  dest
        rcr     [word dest+2], 1        ;; rcr dest, 1
        rcr     [word dest], 1
endm    rcrdw1

;*********************************************************
;
; ROLDW1
;
; Esta macro rota el contenido de dest un bit a la izquierda
;*********************************************************
;
macro   roldw1  dest
        push    bx                      ;; Preserva BX
        mov     bx, [word dest+2]       ;; rol dest, 1
        shl     bx, 1
        rcl     [word dest], 1
        rcl     [word dest+2], 1
        pop     bx                      ;; Restaura BX
endm    roldw1
```

```
;*********************************************************
; RORDW1
;
; Esta macro rota el contenido de dest un bit a la derecha.
;*********************************************************
macro   rordw1  dest
        push    bx                      ;; Preserva BX
        mov     bx, [word dest]         ;; ror dest, 1
        shr     bx, 1
        rcr     [word dest+2], 1
        rcr     [word dest], 1
        pop     bx                      ;; Restaura BX
endm    rordw1

;*** VARIABLES DEL PROGRAMA ******************************

        dataseg
codsal  db      0
r_rcl   dd      ?
r_rcr   dd      ?
r_rol   dd      ?
r_ror   dd      ?

;****** CÓDIGO DEL PROGRAMA ******************************

        codeseg
inicio:
        mov     ax, @data               ; Inicializa el
        mov     ds, ax                  ; segmento de datos

; Rotación a la izquierda a través de la bandera de acarreo.

        rcldw1  r_rcl                   ; rcl r_rcl, 1

; Rotación a la derecha a través de la bandera de acarreo
        rcrdw1  r_rcr                   ; rcr r_rcr, 1

; Rotación a la izquierda
        roldw1  r_rol                   ; rol r_rol, 1

; Rotación a la derecha
        rordw1  r_ror                   ; ror r_ror, 1

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

En el ejemplo anterior podemos notar que los comentarios dentro de la definición de una macro empiezan con doble punto y coma. Esto se hace para que en el archivo con el listado del programa generado por el ensamblador no aparezcan los comentarios cada vez que se expande la macro.

También podemos notar que las macros **roldw1** y **rordw1** que implementan las rotaciones a la izquierda y a la derecha, respectivamente, utilizan el registro **BX** y por lo tanto se preserva su valor al inicio de la macro y se restaura al salir. Esto se hace con el fin de que el uso de la macro sea transparente para el usuario.

Ejercicio sobre macros

Escriba un programa que ejecute todos los corrimientos de un bit con una variable de tipo palabra doble. Las operaciones de corrimiento se implementan mediante macros. Cada macro tiene un parámetro formal **dest** que representa la localidad de memoria que contiene el dato a rotar.

Macros con etiquetas locales

Al igual que con los procedimientos, si una macro emplea identificadores para etiquetar las direcciones de instrucciones que componen la macro, estas etiquetas deben ser etiquetas locales, para evitar la duplicidad de etiquetas si la macro se invoca más de una vez en un programa. Para crear etiquetas locales en Turbo Assembler se emplea la directiva **local**. La directiva **local** en una macro hace que las etiquetas declaradas en la directiva tengan un alcance restringido a la macro. La sintaxis de esta directiva es:

```
local etiqueta [, etiqueta]...
```

En una macro, la directiva local debe ir inmediatamente de la directiva macro. Pueden tenerse varias directivas local.

Ejemplo sobre macros con etiquetas locales

El programa siguiente encuentra el mayor de tres datos signados de tipo palabra. El programa utiliza la macro **mayor3** para encontrar el mayor de los tres datos. La macro tiene dos etiquetas locales: **sigcmp** y **fincmp**.

```
;*********************************************************
; MAYOR3_2.ASM
;
; Este programa encuentra el mayor de tres datos signados
; almacenados en variables de una palabra. El programa
; utiliza una macro para encontrar al mayor de los tres
; datos
;*********************************************************
```

```
;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256


;****** MACROS ********************************************

;*****************************************************
; MAYOR3
;
; Esta macro encuentra el mayor de tres datos signados
; dados por los parámetros dato1, dato2 y dato3. El mayor
; de los tres datos queda en el registro AX.
;*****************************************************
macro   mayor3  dato1, dato2, dato3
        local   sigcmp, fincmp

        mov     ax, [dato1]     ; AX = dato1
        cmp     ax, [dato2]     ; if(AX > dato2)
        jg      sigcmp          ;    goto sigcmp
        mov     ax, [dato2]     ; else AX = dato2
sigcmp:
        cmp     ax, [dato3]     ; if(AX > dato3)
        jg      fincmp          ;    goto fincmp
        mov     ax, [dato3]     ; else AX = dato3
fincmp:
endm    mayor3

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
dato1   dw      ?
dato2   dw      ?
dato3   dw      ?
mayor   dw      ?

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov      ax, @data      ; Inicializa el
        mov      ds, ax         ; segmento de datos

        mayor3  dato1, dato2, dato3

        mov     [mayor], ax     ; mayor = AX

salir:
```

```
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

Ejercicio sobre macros con etiquetas locales

Escribe una macro que determine el mayor de dos datos signados de tipo palabra doble. La macro recibe los datos en parámetros y regresa el mayor de los dos datos en los registros **DX:AX**.

Directiva de repetición rept

Los macroensambladores del 8086 poseen varias directivas que permiten repetir instrucciones. Una de esas es la directiva **rept**. La directiva **rept** permite repetir un grupo de proposiciones varias veces. Su sintaxis es:

```
rept    expresión
        proposición
        [proposición]
        ...
endm
```

donde las directivas **rept** y **endm** delimitan la directiva de repetición **rept**.

expresión es una expresión constante entera y es el número de veces que se repiten las instrucciones dadas por *proposición*.

La directiva de repetición **rept** puede utilizarse por sí sola o dentro de macros para crear instrucciones que repitan instrucciones.

Ejemplo sobre macros que usan la directiva de repetición rept

El archivo siguiente muestra una modificación de las macros que rotan variables de tipo palabra doble para que en lugar de rotar las variables un solo bit, las roten varios bits. Las macros contienen la directiva de rotación **rept**. Las macros se encuentran en un archivo con el nombre ROTA_N.INC el cual puede incluirse en otros programas usando la directiva **include** la cual se verá en la siguiente sección. La extensión .INC no es obligatoria para los archivos de inclusión pero si es una práctica común.

```
;*********************************************************
; ROTA_N.INC
;
; Este archivo contiene una serie de macros que pueden
; incluirse en otros programas con la directiva include e
; implementan las operaciones de rotación del contenido de
; la localidad de memoria dada por el parámetro dest un
; número de bits dado por el parámetro cnta.
;*********************************************************


;*********************************************************
; RCLDWN
;
; Esta macro rota el contenido de dest cnta bits a la
; izquierda a través de la bandera de acarreo.
;*********************************************************
macro   rcldwn  dest, cnta
        rept    cnta
         rcl    [word dest], 1          ;; rcl dest, 1
         rcl    [word dest+2], 1
        endm
endm    rcldwn

;*********************************************************
; RCRDWN
;
; Esta macro rota el contenido de dest cnta bits a la
; derecha a través de la bandera de acarreo.
;*********************************************************
macro   rcrdwn  dest, cnta
        rept    cnta
         rcr    [word dest+2], 1        ;; rcr dest, 1
         rcr    [word dest], 1
        endm
endm    rcrdwn

;*********************************************************
; ROLDWN
;
; Esta macro rota el contenido de dest cnta bits a la
; izquierda.
;*********************************************************
macro   roldwn  dest, cnta
        push    bx                      ;; Preserva BX
        rept    cnta
         mov    bx, [word dest+2]       ;; rol dest, 1
         shl    bx, 1
         rcl    [word dest], 1
         rcl    [word dest+2], 1
        endm
        pop     bx                      ;; Restaura BX
endm    roldwn
```

```
;*********************************************************
; RORDWN
;
; Esta macro rota el contenido de dest cnta bits a la
; derecha.
;*********************************************************
macro   rordwn  dest, cnta
        push    bx                      ;; Preserva BX
        rept    cnta
         mov    bx, [word dest]         ;; ror dest, 1
         shr    bx, 1
         rcr    [word dest+2], 1
         rcr    [word dest], 1
        endm
        pop     bx                      ;; Restaura BX
endm    rordwn
```

Ejercicio sobre macros que usan la directiva de repetición rept

Modifica el programa de las macros que corren variables de tipo palabra doble para que en lugar de correr las variables un solo bit, las corran varios bits. Las macros deben usar la directiva de rotación **rept**.

Inclusión de archivos con la directiva include

La directiva **include** permite la inclusión de un archivo texto dentro de otro. La sintaxis de esta directiva es:

include "*nomArch*"

donde *nomArh* es el nombre del archivo cuyo contenido será substituido en lugar de la directiva **include**.

A continuación se muestra un ejemplo que ilustra el uso de la directiva **include** para incluir dentro del código el contenido del archivo ROTA_N.INC del ejemplo anterior. Este programa ejecuta todas las rotaciones de un número de bits con una variable de tipo palabra doble.

```
;*********************************************************
; ROTACIO3.ASM
;
; Este programa ejecuta todas las rotaciones de un número
; de bits con una variable de tipo palabra doble. Las
; operaciones de rotación están implementadas mediante
; macros y se encuentran en el archivo ROTA_N.INC. Este
```

```
; programa ilustra el uso de la directiva include.
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** INCLUSIÓN DE ARCHIVOS *****************************

include "rota_n.inc"

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
r_rcl   dd      ?
r_rcr   dd      ?
r_rol   dd      ?
r_ror   dd      ?

;****** CÓDIGO DEL PROGRAMA *******************************

   codeseg
inicio:
        mov     ax, @data               ; Inicializa el
        mov     ds, ax                  ; segmento de datos

; Rotación a la izquierda a través de la bandera de acarreo
        rcldwn  r_rcl, 2                ; rcl r_rcl, 2

; Rotación a la derecha a través de la bandera de acarreo
        rcrdwn  r_rcr, 2                ; rcr r_rcr, 2

; Rotación a la izquierda
        roldwn  r_rol, 2                ; rol r_rol, 2

; Rotación a la derecha
        rordwn  r_ror, 2                ; ror r_ror, 2

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documento PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

Problemas

Escribe un programa que implemente las operaciones de suma, resta, multiplicación y división de números fraccionarios. Cada número fraccionario se representará mediante un par de números enteros (de tipo palabra). El programa constará de dos módulos.

El primer módulo llamado **ARITFRAC.ASM** contiene los siguientes procedimientos:

mcd.- que calcula el máximo común divisor de dos números enteros de tipo palabra. El procedimiento recibe los dos números en los registros AX y DX y regresa el máximo común divisor en el registro AX.

mcm.- que calcula el mínimo común múltiplo de dos números enteros de tipo palabra. El procedimiento recibe los dos números en los registros AX y DX y regresa el mínimo común múltiplo en los registros DX:AX.

redfrac.- que reduce una fracción a su mínima expresión. El procedimiento recibe los dos números que representan el numerador y el denominador en los registros AX y DX, respectivamente y regresa la fracción reducida en los mismos registros.

addfrac.- que suma dos números fraccionarios y regresa la suma como una fracción reducida a su mínima expresión. El procedimiento recibe los dos números fraccionarios en los registros AX y DX para el numerador y denominador del primer número y BX y CX para el numerador, denominador del segundo número. El procedimiento regresa el resultado en los registros AX y DX para el numerador y denominador, respectivamente.

subfrac.- que resta dos números fraccionarios y regresa la resta como una fracción reducida a su mínima expresión. El procedimiento recibe los dos números fraccionarios en los registros AX y DX para el numerador y denominador del primer número y BX y CX para el numerador, denominador del segundo número. El procedimiento regresa el resultado en los registros AX y DX para el numerador y denominador, respectivamente.

mulfrac.- que multiplica dos números fraccionarios y regresa el producto como una fracción reducida a su mínima expresión. El procedimiento recibe los dos números fraccionarios en los registros AX y DX para el numerador y denominador del primer número y BX y CX para el numerador, denominador del segundo número. El procedimiento regresa el resultado en los registros AX y DX para el numerador y denominador, respectivamente.

divfrac.- que divide dos números fraccionarios y regresa el cociente como una fracción reducida a su mínima expresión. El procedimiento recibe los dos números fraccionarios en los registros AX y DX para el numerador y denominador del primer número y BX y CX para el numerador, denominador del segundo número. El procedimiento regresa el resultado en los registros AX y DX para el numerador y denominador, respectivamente.

El segundo módulo llamado **DEMOFRAC.ASM** contiene un programa para probar los procedimientos del módulo anterior. En este módulo se declaran 8 variables: **numA**, **denA**, **numB**, **denB**, **numC**, **denC**, **numR**, **denR**, para almacenar cuatro números fraccionarios:

A = numA/denA

B = numB/denB

C = numC/denC

R = numR/denR

El programa deberá efectuar la siguiente operación:

R = ((A + B) * (C - A))/(B + C).

El programa ejecutable deberá llamarse **DEMOFRAC.EXE**

CAPÍTULO 7

ARREGLOS Y CADENAS

Arreglos

Un arreglo es una lista de variables del mismo tipo. A diferencia de los lenguajes de alto nivel donde existen mecanismos para declarar y usar arreglos, en ensamblador el programador tiene que crear el código para declarar y usar los arreglos.

Declaración de arreglos

Para declarar arreglos se utilizan las mismas directivas empleadas para declarar variables sencillas: **db** para declarar arreglos de bytes, **dw** para arreglos de palabras, **dd** para arreglos de palabras dobles y **dq** para arreglos de palabras cuádruples. La sintaxis de estas directivas, en el caso de declaraciones de arreglos, son:

[*nomArreg*] **db** *exp*[**,** *exp*]...
[*nomArreg*] **dw** *exp*[**,** *exp*]...
[*nomArreg*] **dd** *exp*[**,** *exp*]...
[*nomArreg*] **dq** *exp*[**,** *exp*]...

donde *nomArreg* es el nombre del arreglo y *exp* son expresiones constantes. El número de expresiones determina el tamaño del arreglo y sus valores se utilizan para inicializar el arreglo. Por ejemplo

```
bdtos   db   0, 1, 2, 3, 4      ; Arreglo de cinco
                                ; variables de tipo byte
                                ; inicializadas a los
                                ; valores 0, 1, 2, 3 y 4.
cresps  db   'A', 'B', 'C'      ; Arreglo de tres
                                ; variables de tipo byte
                                ; inicializadas a los
                                ; a los valores 65d, 66d
                                ; y 67d.
wdtos   dw   0123h, 2345h       ; Arreglo de cinco
        dw   4567h, 6789h       ; variables de tipo
        dw   89abh              ; palabra inicializadas
```

```
                                    ; a los valores 0123h,
                                    ; 2345h, 4567h, 6789h y
                                    ; 89abh.
```

Declaración de arreglos usando el operador dup

En la declaración de arreglos se puede utilizar el operador **dup** para inicializar el arreglo a valores duplicados. La sintaxis del operador **dup** es la siguiente:

```
cnta dup(exp[, exp]... )
```

donde *cnta* define el número de veces que el dato definido por *exp* será repetido. El operador **dup** aparece después de una directiva para declarar variables: **db**, **dw**, etc.

Se puede anidar un operador **dup** dentro de *exp* de otro **dup**, hasta 17 niveles.

Ejemplos:

```
bdtos   db   5 dup(0)          ; Arreglo de cinco
                               ; variables de tipo byte
                               ; inicializadas a 0.

bdtosx  db   10 dup(?)         ; Arreglo de diez
                               ; variables de tipo byte
                               ; no inicializadas.

bdtos   db   5 dup(1), 0       ; Arreglo de seis
                               ; variables de tipo byte.
                               ; Las primeras cinco
                               ; inicializadas a 1 y la
                               ; última a 0.

wdtos   dw   5 dup(0, 1)       ; Arreglo de 10 variables
                               ; de tipo palabra
                               ; inicializadas a los
                               ; valores 0, 1, 0, 1,
                               ; etc.

wdtosx  dw   5 dup(4 dup(?))   ; Arreglo de 5 × 4
                               ; variables de tipo
                               ; palabra no
                               ; inicializadas.
```

Declaración de arreglos usando la directiva de repetición rept

La directiva de repetición **rept** nos permite también declarar e inicializar arreglos como se muestra en el siguiente ejemplo:

```
label   bArreg  byte            ; Arreglo de 100 arreglos
rept    100                     ; de 21 bytes cada uno.
        db   20 dup(' '), 0     ; Cada arreglo está
endm                            ; inicializado a veinte
                                ; caracteres de espacio
                                ; seguidos de un cero
```

La primera línea de la declaración anterior utiliza la directiva **label** para declarar la etiqueta **bArreg** de tipo byte. La directiva **label** le indica al ensamblador cómo accesar a los datos que se declaran enseguida. En este caso **bArreg** será tratado como un arreglo de bytes, no reserva espacio de memoria. La sintaxis de la directiva **label** es la siguiente:

label *etiq tipo*

donde *etiq* es el identificador de la etiqueta y *tipo* es el tipo de los datos declarados a continuación. *tipo* puede ser: **byte**, **word**, **dword**, etc.

Como un segundo ejemplo, se tiene la siguiente declaración:

```
val = 0
label   wArreg  word            ; Arreglo de 10 variables
rept    10                      ; de tipo palabra
        dw   val                ; inicializadas a 0, 1,
        val = val + 1           ; 2, ..., 9.
endm
```

En este caso el símbolo **val** se inicializa a 0 usando la directiva =. En cada repetición el valor de **val** se ve incrementando en uno.

Acceso a los elementos de un arreglo

El nombre de un arreglo etiqueta la dirección donde se encuentra el primer elemento de un arreglo. Esto es, el nombre del arreglo es la dirección base del arreglo. La dirección de cualquier elemento del arreglo se puede expresar como la suma de la dirección base del arreglo más la distancia de ese elemento con respecto al primer elemento del arreglo. Esa distancia puede medirse de dos formas: La distancia expresada en número de elementos, llamada también **índice**, o la distancia medida en bytes, llamada también **desplazamiento**.

El primer elemento de un arreglo tiene un índice de cero. También su desplazamiento es de cero. Si el arreglo es de bytes el índice y el desplazamiento son iguales. Para los arreglos de otros tipos el desplazamiento es un múltiplo del índice y está dado por:

desplazamiento = tamDato * índice

donde tamDato es el tamaño en bytes de cada elemento del arreglo. A diferencia de los lenguajes de alto nivel donde se accesa a los elementos de un arreglo mediante su índice, en ensamblador se accesa a los elementos de un arreglo mediante su desplazamiento. Por ejemplo sean, las siguientes declaraciones:

```
bdtosx  db      10 dup(?)
wdtosx  dw      5 dup(?)
```

entonces, las siguientes instrucciones almacenan el valor de 5 en el primer elemento, índice 0, y el valor de 6 en el tercer elemento, índice 2, de **bdtosx**.

```
mov     [bdtosx], 5     ; bdtosx[0] = 5
mov     [bdtosx+2], 6   ; bdtosx[2] = 6
```

y la siguientes instrucciones almacenan el valor de 500 en el primer elemento, índice 0, y el valor de 300 en el cuarto elemento, índice 3, de **wdtosx**:

```
mov     [wdtosx], 500           ; wdtosx[0] = 500
mov     [wdtosx+2*3], 300       ; wdtosx[3] = 300
```

En muchas ocasiones deseamos accesar a los elementos de un arreglo en una secuencia determinada por lo que sería conveniente que el índice o el desplazamiento de un elemento del arreglo estuviera en una variable o en un registro. En este caso, se puede utilizar el direccionamiento base para accesar a los elementos del arreglo. Por ejemplo, las siguientes instrucciones almacenan el valor de 6 en el elemento del arreglo **bdtosx** cuyo índice está en la variable **indice**:

```
mov     bx, [indice]    ; bdtosx[indice] = 6
mov     [bx+bdtosx], 6
```

Por otro lado, las siguientes instrucciones almacenan el valor de 300 en el elemento del arreglo **wdtosx** cuyo índice está en la variable **indice**:

```
mov     bx, [indice]    ; wdtosx[indice] = 300
sal     bx, 1
mov     [bx+wdtosx], 300
```

Otros modos de direccionamiento que se pueden emplearse para accesar a los elementos de un arreglo son el direccionamiento indirecto, el direccionamiento indexado y el direccionamiento base-indexado.

Modo de direccionamiento registro indirecto

En el modo de direccionamiento registro indirecto, la dirección efectiva del dato se calcula tomando como desplazamiento el contenido de uno de tres registros: **BX**, **SI** o **DI**. El segmento en los tres casos es el valor del registro de segmento de datos **DS**.

El modo de direccionamiento registro indirecto tiene la siguiente sintaxis:

[bx]
[si]
[di]

El direccionamiento registro indirecto se emplea normalmente para permitir que una instrucción opere en múltiples variables o elementos de un arreglo. Por ejemplo:

```
mov     ax, [bx]
inc     [word si]
dec     [byte di]
```

Modo de direccionamiento indexado

El modo de direccionamiento indexado es similar al modo de direccionamiento **base**, sólo que utiliza los registros **SI** o **DI** en lugar de los registros **BX** o **BP**. Las referencias a **SI** o **DI** son desplazamientos con respecto al registro de segmento de datos **DS**.

El modo de direccionamiento base tiene la siguiente sintaxis:

[si+*n*]
[si-*n*]
[di+*n*]
[di-*n*]

En los dos primeros casos el desplazamiento del dato con respecto a **DS** está dado por el valor de **SI** más o menos *n* bytes. En los dos últimos casos el desplazamiento del dato con respecto a **DS** está dado por el valor de **DI** más o menos *n* bytes.

El direccionamiento indexado se emplea normalmente para accesar a los elementos de un arreglo. Por ejemplo, las siguientes instrucciones almacenan el valor de 6 en el elemento del arreglo **bdtosx** cuyo índice está en la variable **indice**:

```
mov     si, [indice]    ; bdtosx[indice] = 6
mov     [si+bdtosx], 6
```

Por otro lado, las siguientes instrucciones almacenan el valor de 300 en el elemento del arreglo **wdtosx** cuyo índice está en la variable **indice**:

```
mov     di, [indice]    ; wdtosx[indice] = 300
sal     di, 1
mov     [di+wdtosx], 300
```

Modo de direccionamiento base-indexado

En el modo de direccionamiento base-indexado, el cálculo de la dirección efectiva del dato emplea dos registros: un registro base: **BX** o **BP** y un registro índice: **SI** o **DI**. Las referencias a **BX** son desplazamientos con respecto al registro de segmento de datos **DS** mientras que las referencias a **BP** son desplazamientos con respecto al registro de segmento de pila **SS**.

El modo de direccionamiento base-indexado tiene la siguiente sintaxis:

[bx+si[+*n*]**]**
[bx+di[+*n*]**]**
[bp+si[+*n*]**]**
[bp+di[+*n*]**]**

En los dos primeros casos el desplazamiento del dato con respecto a **DS** está dado por el valor de **BX** más **SI** o **DI** más opcionalmente *n* bytes. En los dos últimos casos el desplazamiento del dato con respecto a **SS** está dado por el valor de **BP** más **SI** o **DI** más opcionalmente *n* bytes.

El direccionamiento base-indexado se emplea normalmente para accesar a los elementos de un arreglo. Los registros **BX** o **BP** pueden contener la dirección base de un arreglo, los registros **SI** o **DI** el desplazamiento del elemento con respecto a la dirección base del arreglo y luego agregarle un posible valor n para localizar un campo en este elemento específico. Por ejemplo, las siguientes instrucciones almacenan el valor de 6 en el elemento del arreglo **bdtosx** cuyo índice está en la variable **indice**:

```
mov     bx, offset bdtosx       ; bdtosx[indice]
mov     si, [indice]            ;    = 6
mov     [bx+si], 6
```

Por otro lado, las siguientes instrucciones almacenan el valor de 300 en el elemento del arreglo **wdtosx** cuyo índice está en la variable **indice**:

```
mov     bx, offset wdtosx       ; wdtosx[indice]
mov     si, [indice]            ;    = 300
sal     si, 1
mov     [bx+si], 300
```

Ejemplos sobre arreglos

1. El siguiente programa encuentra el mayor de un conjunto de datos de tipo palabra almacenados en un arreglo. En este programa se utiliza el direccionamiento indexado para accesar a los elementos del arreglo.

```
;*********************************************************
; MAYORNW1.ASM
;
; Este programa encuentra el mayor de un conjunto de datos
; de tipo palabra almacenados en un arreglo. El
; pseudocódigo de este programa es:
;
;    mayor = datos[0]
;    i = 0;
;
;    while(i < nDatos-1)
;    {
;       i++
;       if(mayor >= datos[i]) continue
;       mayor = datos[i]
;   }
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
mayor   dw      ?
nDatos  dW      ?
datos   dw      TAMMAX dup(?)

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos

        mov     ax, [datos]     ; mayor = datos[0]
        mov     bx, [nDatos]    ; BX = 2 *(nDatos-1)
        dec     bx
        sal     bx, 1
        xor     si, si          ; SI = 0

while:  cmp     si, bx          ; while(SI < 2*nDatos)
        jae     endwhi          ; {
        inc     si              ;    SI++
```

```
        inc     si              ;    SI++
        cmp     ax, [si+datos]  ;    if(mayor >= datos[SI])
        jae     while           ;       continue
        mov     ax, [si+datos]  ;    mayor = datos[SI]

        jmp     while           ; }
endwhi:
        mov     [mayor], ax

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

2. El siguiente programa es una modificación del ejemplo anterior que utiliza el direccionamiento registro indirecto en lugar del direccionamiento indexado para accesar a los elementos del arreglo.

```
;*********************************************************
; MAYORNW2.ASM
;
; Este programa encuentra el mayor de un conjunto de datos
; de tipo palabra almacenados en un arreglo. El
; pseudocódigo de este programa es:
;
;   pDatos = datos
;   p = datos + nDatos
;   mayor = *pDatos
;
;   while(pDatos < p-1)
;   {
;      pDatos++
;      if(mayor >= *pDatos continue
;      mayor = *pDatos
;   }
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100
```

```
;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
mayor   dw      ?
nDatos  dW      ?
datos   dw      TAMMAX dup(?)

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data          ; Inicializa el
        mov     ds, ax             ; segmento de datos

        mov     bx, offset datos   ; pDatos = datos
        mov     cx, [nDatos]       ; p = datos + (2*nDatos
        dec     cx                 ;    -1)
        sal     cx, 1
        add     cx, bx
        mov     ax, [bx]           ; mayor = *pDatos

while:  cmp     bx, cx             ; while(pDatos < p)
        jae     endwhi             ; {
        inc     bx                 ;    pDatos++
        inc     bx                 ;    pDatos++

        cmp     ax, [bx]           ;    if(mayor >= *pDatos)
        jae     while              ;       continue
        mov     ax, [bx]           ;    mayor = *pDatos
        jmp     while              ; }
endwhi:
        mov     [mayor], ax

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

Ejercicios sobre arreglos

1. Cree un programa que sume un conjunto de datos de tipo palabra almacenados en un arreglo. Utilice el direccionamiento indexado para accesar a los elementos del arreglo.

2. Cree un programa que sume un conjunto de datos de tipo palabra almacenados en un arreglo. Utilice el direccionamiento registro indirecto para accesar a los elementos del arreglo.

Procedimientos y arreglos

En muchas ocasiones deseamos que un procedimiento opere sobre los elementos de un arreglo. La técnica más eficiente es la de pasarle al procedimiento la dirección del arreglo y así permitirle que accese a los elementos del arreglo.

Ejemplos sobre procedimientos y arreglos

1. El siguiente programa encuentra la primera ocurrencia de un dato dentro de un arreglo de datos de tipo palabra. El programa almacena en la variable **pos** la posición de la primera ocurrencia del valor dado por **llave**, -1 (0FFFFh) en caso contrario. En este programa se utiliza el direccionamiento base-indexado para accesar a los elementos del arreglo. Se utiliza el algoritmo de búsqueda lineal.

```
;*********************************************************
; BLINEAL1.ASM
;
; Este programa busca la primera ocurrencia del dato dado
; por llave dentro del arreglo de palabras dado por datos.
; El programa almacena en la variable pos la posición de
; llave dentro del arreglo, -1 (0FFFFh) en caso contrario.
; El pseudocódigo de este programa es:
;
;   pos = blineal(llave, datos, ndatos)
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
llave   dw      ?
nDatos  dW      ?
pos     dW      ?
datos   dw      TAMMAX dup(?)
```

```
;****** CÓDIGO DEL PROGRAMA ********************************

        codeseg
inicio:
        mov     ax, @data         ; Inicializa el
        mov     ds, ax            ; segmento de datos

        mov     ax, [llave]       ; AX = llave
        mov     bx, offset datos  ; BX = datos
        mov     cx, [nDatos]      ; CX = nDatos

        call    blineal

        mov     [pos], ax         ; pos = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** PROCEDIMIENTOS *************************************

;*******************************************************
; BLINEAL
;
; Este procedimiento utiliza direccionamiento base indexado
; para encontrar la primera ocurrencia de llave dentro del
; arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        BX = datos
;        CX = nDatos
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario
;
; El pseudocódigo para este procedimiento es:
;
;   int blineal(int llave, int *pdatos, int ndatos)
;   {
;      i = 0
;
;       while(i < nDatos)
;       {
;          if(llave == datos[i]) goto @@siesta
;          i++
;       }
;
;       return -1
;
```

```
;   @@siesta:
;       return I
;   }
;*************************************************************
proc    blineal
        push    si                  ; Preserva SI

        xor     si, si              ; SI = 0
        sal     cx, 1               ; CX = 2 * nDatos

@@whi:  cmp     si, cx              ; while(SI < 2 * nDatos)
        jae     @@endwhi            ; {
        cmp     ax, [bx+si]         ;   if(llave ==
                                    ;       datos[SI])
        je      @@siesta            ;       goto @@siesta
        inc     si                  ;    SI++
        inc     si                  ;    SI++
        jmp     @@whi               ; }
@@endwhi:
        mov     ax, 0FFFFh          ; AX = 0FFFFh
        jmp     @@fin               ; goto fin

@@siesta:
        sar     si, 1               ; SI /= 2
        mov     ax, si              ; AX = CX

@@fin:  pop     si                  ; Recupera SI
        ret
endp    blineal

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

2. El siguiente código muestra una segunda versión del procedimiento **blineal** del ejemplo anterior que utiliza el direccionamiento registro indirecto en lugar del direccionamiento base-indexado para accesar a los elementos del arreglo.

```
;*************************************************************
; BLINEAL
;
; Este procedimiento utiliza el direccionamiento registro
; indirecto para encontrar la primera ocurrencia de llave
; dentro del arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        BX = datos
;        CX = nDatos
```

```
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario.
;
; El pseudocódigo de este procedimiento es:
;
;   int blineal(int llave, int *pdatos, int ndatos)
;   {
;      pDatos = datos
;      p = pDatos + nDatos
;
;       while(pDatos < p)
;       {
;          if(llave == *pdatos) goto @@siesta
;          pDatos++
;       }
;
;       return –1
;
;   @@siesta:
;       return pDatos – datos
;   }
;*******************************************************
proc    blineal
        push    si              ; Preserva SI

        mov     si, bx          ; pDatos = SI = datos
        sal     cx, 1           ; p = pDatos + 2*nDatos
        add     cx, bx

@@whi:  cmp     bx, cx          ; while(pDatos < p)
        jae     @@endwhi        ; {
        cmp     ax, [bx]        ;    if(llave == *pDatos)
        je      @@siesta        ;       goto @@siesta
        inc     bx              ;    pDatos++
        inc     bx              ;    pDatos++
        jmp     @@whi           ; }
@@endwhi:

        mov     ax, 0FFFFh      ; AX = 0FFFFh
        jmp     @@fin           ; goto fin

@@siesta:
        mov     ax, bx          ; AX = (pDatos-datos)/2
        sub     ax, si
        sar     ax, 1

@@fin:  pop     si              ; Recupera SI
        ret
endp    blineal
```

3. El siguiente código muestra una tercera versión del procedimiento **blineal**. En el ejemplo anterior se utilizan tres apuntadores: **CX** para apuntar al final del arreglo, **SI** para apuntar al inicio del arreglo y **BX** para recorrer el arreglo. En esta versión sólo se utilizan dos apuntadores: **CX** que apunta al inicio del arreglo y **BX** que inicialmente apunta al final del arreglo y que se utiliza para recorrer el arreglo de atrás hacia adelante.

```
;*********************************************************
; BLINEAL
;
; Este procedimiento utiliza el direccionamiento registro
; indirecto para encontrar la primera ocurrencia de llave
; dentro del arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        BX = datos
;        CX = nDatos
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario
;
; El pseudocódigo de este procedimiento es:
;
;   int blineal(int llave, int *pdatos, int ndatos)
;   {
;      p = pDatos + nDatos
;
;       while(p > datos)
;       {
;          p--
;          if(llave == *p) goto @@siesta
;       }
;
;       return –1
;   @@siesta:
;       return p – datos
;   }
;*********************************************************
proc    blineal
        sal     cx, 1               ; p = BX = datos
        add     cx, bx              ;  + 2*nDatos
        xchg    cx, bx              ; CX = datos

@@whi:  cmp     bx, cx              ; while(p > datos)
        jbe     @@endwhi            ; {
        dec     bx                  ;    p--
        dec     bx                  ;    p--
        cmp     ax, [bx]            ;    if(llave == *p)
        je      @@siesta            ;       goto @@siesta
        jmp     @@whi               ; }
```

```
@@endwhi:

        mov     ax, 0FFFFh          ; AX = 0FFFFh
        ret

@@siesta:
        mov     ax, bx              ; AX = (p - datos)/2
        sub     ax, cx
        sar     ax, 1
        ret
endp    blineal
```

Ejercicios sobre procedimientos y arreglos

1. Escribe un procedimiento llamado **ppunto** que encuentre el producto punto entre dos vectores que se encuentran almacenados en dos arreglos unidimensionales. El procedimiento recibe como parámetros las direcciones de los dos arreglos en los registros **AX** y **DX** y el número de elementos en cada vector en el registro **CX**. El procedimiento regresa el producto punto en los registros **DX:AX**.

2. Escribe un procedimiento que ordene un arreglo en forma ascendente usando el algoritmo de selección. El procedimiento recibe como parámetro la direcciones del arreglo en el registro **AX** y el número de elementos en el arreglo en el registro **CX**.

Operaciones De Cadenas

El ensamblador del 8086 posee un conjunto de instrucciones especiales para procesar arreglos de bytes y de palabras. Estas instrucciones aunque reciben el nombre de **instrucciones de cadenas** operan sobre arreglos de bytes y palabras sin importar el contenido de los arreglos. Las instrucciones de cadenas se dividen en tres grupos: Instrucciones de transferencia de cadenas, instrucciones de inspección de cadenas y prefijos de repetición de instrucciones.

Todas las instrucciones de cadenas utilizan los registros **DS:SI** y **ES:DI** para realizar su trabajo. Ambas combinaciones **DS:SI** y **ES:DI** se utilizan como índices a los arreglos sobre los que se está operando. Al igual que como lo hemos hecho con el registro de segmento de datos **DS**, debemos inicializar el registro de segmento extra **ES** para que apunte al segmento que contiene el arreglo al que va hacerse referencia mediante **ES:DI**. Si el programa contiene un sólo segmento de datos o si las cadenas sobre las que se van a operar están todas en el mismo segmento de datos la inicialización puede hacerse de la siguiente manera:

```
mov ax, @data
mov ds, ax
mov es, ax
```

Si por el contrario, queremos que el registro de segmento extra **ES** apunte a otro segmento donde está definida la variable **edato**, podemos hacer lo siguiente:

```
mov  ax, seg edato
mov  es, ax
```

Las instrucciones de cadena además de realizar su trabajo, incrementan o decrementan en forma automáticamente los registros índice que usan. Las operaciones de byte incrementan o decrementan los registros **SI**, **DI**, o ambos en uno, mientras que las operaciones de palabras incrementan o decrementan los registros **SI**, **DI**, o ambos en dos. El que las instrucciones de cadenas incrementen o decrementen los registros de índice está controlado por la **bandera de dirección D**. Si **D** = 0, entonces los registros índice se incrementan y si **D** = 1, entonces los registros índice se decrementan. Para establecer el valor de la bandera de dirección se utilizan las instrucciones: **cld** y **std**.

cld

Coloca un cero en la bandera de dirección.

Sintaxis:

cld

Utilice **cld** siempre que se va a ejecutar una instrucción de cadena donde se desee que los registros **SI**, **DI**, o ambos se incrementen automáticamente.

La instrucción **cld** no afecta el resto de las banderas.

std

Coloca un uno en la bandera de dirección.

Sintaxis:

std

Utilice **std** siempre que se va a ejecutar una instrucción de cadena donde se desee que los registros **SI**, **DI**, o ambos se decrementen automáticamente.

La instrucción **cld** no afecta el resto de las banderas.

Instrucciones de transferencia de cadenas

Estas instrucciones permiten mover bytes y palabras de memoria a un registro, de un registro a memoria o directamente de memoria a memoria.

lods *origen*

Carga en el acumulador **AX** o **AL** el valor del elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **SI**.

Sintaxis:

lods [byte [**es:**]**si]**
lods [word [**es:**]**si]**

El operando de **lods** es siempre el registro **SI** que contiene el desplazamiento del dato con respecto al segmento dado por **DS**. Si el arreglo esta en el segmento apuntado por el registro de segmento extra se puede utilizar el operador **:** (dos puntos) que modifica el registro de segmento empleado por omisión. Para que **SI** contenga el desplazamiento con respecto al segmento extra en lugar del segmento de datos escribiremos **es:si** en lugar de **si**.

Cada vez que la instrucción **lods** se ejecuta, el registro **SI** se incrementa o decrementa en uno o dos para que apunte al siguiente o al anterior elemento del arreglo dependiendo de sí el valor de la bandera de dirección es cero o uno.

La instrucción **lods** no afecta las banderas.

lodsb | lodsw

lodsb es una abreviatura de **lods [byte si]** y **lodsw** es una abreviatura de **lods [word si].**

Sintaxis:

lodsb
lodsw

movs *destino***,** *origen*

Copia directamente el valor de un elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **SI** a otro elemento de un segundo arreglo cuyo desplazamiento con respecto del principio de este segundo arreglo está dado por **DI**.

Sintaxis:

movs [byte di], [[**es:**]**si]**
movs [word di], [[**es:**]**si]**

El primer operando de **movs** es siempre el registro **DI** que contiene el desplazamiento del destino con respecto al segmento dado por **ES**. El segundo operando de **movs** es siempre el registro **SI** que contiene el desplazamiento del origen con respecto al segmento dado por **DS**. Si el arreglo origen esta en el segmento apuntado por el registro de segmento extra se puede utilizar el operador **:** (dos puntos) que modifica el registro de segmento empleado por omisión. Para que **SI** contenga el desplazamiento con respecto al segmento extra en lugar del segmento de datos escribiremos **es:si** en lugar de **si**.

Cada vez que la instrucción **movs** se ejecuta, ambos los registros **SI** y **DI** se incrementan o decrementan en uno o dos dependiendo del número de bytes copiados. Si la bandera de dirección vale 0, los registros se incrementan y si la bandera de dirección vale 1 los registros se decrementan.

La instrucción **movs** no afecta las banderas.

movsb | movsw

movsb es una abreviatura de **movs [byte di], [si]** y **movsw** es una abreviatura de **movsw [word di], [si].**

Sintaxis:

movsb
movsw

stos *destino*

Carga el valor en el acumulador **AX** o **AL** al elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **DI**.

Sintaxis:

stos [byte di]
stos [word di]

El operando de **stos** es siempre el registro **DI** que contiene el desplazamiento del destino con respecto al segmento dado por **ES**.

Cada vez que la instrucción **stos** se ejecuta, el registro **DI** se incrementa o decrementa en uno o dos para que apunte al siguiente o al anterior elemento del arreglo dependiendo de si el valor de la bandera de dirección es cero o uno.

La instrucción **stos** no afecta las banderas.

stosb | stosw

stosb es una abreviatura de **stos [byte di]** y **stosw** es una abreviatura de **stos [word di].**

Sintaxis:

stosb
stosw

Instrucciones de inspección de cadenas

Estas instrucciones permiten comparar y revisar bytes y palabras buscando valores específicos.

cmps *origen*, *destino*

Compara el valor del elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **SI** con el valor del elemento de un segundo arreglo cuyo desplazamiento con respecto del principio de este segundo arreglo está dado por **DI**.

Sintaxis:

cmps [byte [**es:**]**si], [di]**
cmps [word [**es:**]**si], [di]**

El primer operando de **cmps** es siempre el registro **SI** que contiene el desplazamiento de *origen* con respecto al segmento dado por **DS**. El segundo operando de **cmps** es siempre el registro **DI** que contiene el desplazamiento de *destino* con respecto al segmento dado por **DS**. Si el arreglo *origen* está en el segmento apuntado por el registro de segmento extra se puede utilizar el operador **:** (dos puntos) que modifica el registro de segmento empleado por omisión. Para que **SI** contenga el desplazamiento con respecto al segmento extra en lugar del segmento de datos escribiremos **es:si** en lugar de **si**.

La instrucción **cmps** efectúa la resta [*origen*] - [*destino*], tira el resultado y almacena las bandera en la misma forma en que trabaja la instrucción **cmp**.

Cada vez que la instrucción **cmps** se ejecuta, ambos los registros **SI** y **DI** se incrementan o decrementan en uno o dos dependiendo del número de bytes copiados. Si la bandera de dirección vale 0, los registros se incrementan y si la bandera de dirección vale 1 los registros se decrementan.

La instrucción **cmps** afecta las banderas de sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

cmpsb | cmpsw

cmpsb es una abreviatura de **cmps [byte si], [di]** y **cmpsw** es una abreviatura de **cmpsw [word si], [di].**

Sintaxis:

cmpsb
cmpsw

scas *destino*

Compara el valor en el acumulador **AX** o **AL** con el elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **DI**.

Sintaxis:

scas [byte di]
scas [word di]

El operando de **scas** es siempre el registro **DI** que contiene el desplazamiento de *destino* con respecto al segmento dado por **ES**.

La instrucción **scas** efectúa la resta **AX**|**AL** - [*destino*], tira el resultado y almacena las bandera en la misma forma en que trabaja la instrucción **cmp**.

Cada vez que la instrucción **scas** se ejecuta, el registro **DI** se incrementa o decrementa en uno o dos para que apunte al siguiente o al anterior elemento del arreglo dependiendo de si el valor de la bandera de dirección es cero o uno.

La instrucción **scas** afecta las banderas de sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

scasb | scasw

scasb es una abreviatura de **scas [byte di]** y **scasw** es una abreviatura de **scas [word di].**

Sintaxis:

scasb
scasw

Prefijos de repetición de instrucciones

Los prefijos de repetición de instrucciones son mnemónicos que preceden a las instrucciones de cadenas para crear comandos que se repitan un número de veces o hasta que se cumpla una condición.

rep

Este prefijo repite la instrucción de cadena que le sigue un número de veces especificado por el registro **CX**.

Sintaxis:

rep movs [byte di], [[es:]si]
rep movs [word di], [[es:]si]
rep movsb
rep movsw
rep stos [byte di]
rep stos [word di]
rep stosb
rep stosw

repe | repz

Los dos mnemónicos representan el mismo prefijo de repetición de instrucciones. Este prefijo repite la instrucción de cadena que le sigue un número de veces especificado por el registro **CX** o hasta que después de una iteración el valor de la bandera de cero **Z** valga 1.

Sintaxis:

repe|repz cmps [byte [es:]si], [di]
repe|repz cmps [word [es:]si], [di]
repe|repz cmpsb
repe|repz cmpsw
repe|repz scas [byte di]
repe|repz scas [word di]
repe|repz scasb
repe|repz scasw

repne | repnz

Los dos mnemónicos representan el mismo prefijo de repetición de instrucciones. Este prefijo repite la instrucción de cadena que le sigue un número de veces especificado por el registro **CX** o hasta que después de una iteración el valor de la bandera de cero **Z** valga 0.

Sintaxis:

repe|**repz cmps [byte** [**es:**]**si], [di]**
repe|**repz cmps [word** [**es:**]**si], [di]**
repe|**repz cmpsb**
repe|**repz cmpsw**
repe|**repz scas [byte di]**
repe|**repz scas [word di]**
repe|**repz scasb**
repe|**repz scasw**

Ejemplos sobre instrucciones de cadenas

1. El siguiente programa es otra variante del programa que busca un dato dentro de un arreglo de datos de tipo palabra utilizando el algoritmo de búsqueda lineal. El procedimiento empleado para hacer la búsqueda emplea la instrucción para cadenas **scasw** y el prefijo de repetición de instrucciones **repne**.

```
;*********************************************************
; BLINEAL2.ASM
;
; Este programa busca la primera ocurrencia del dato dado
; por llave dentro del arreglo de palabras dado por datos.
; El programa almacena en la variable pos la posición de
; llave dentro del arreglo, -1 (0FFFFh) en caso contrario.
; El pseudocódigo de este programa es:
;
;   pos = blineal(llave, datos, ndatos)
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
llave   dw      ?
nDatos  dW      ?
pos     dW      ?
datos   dw      TAMMAX dup(?)
```

```
;****** CÓDIGO DEL PROGRAMA ******************************

        codeseg
inicio:
        mov     ax, @data         ; Inicializa el
        mov     ds, ax            ; segmento de datos y
        mov     es, ax            ; el segmento extra

        mov     ax, [llave]       ; AX = llave
        mov     di, offset datos  ; DI = datos
        mov     cx, [nDatos]      ; CX = nDatos

        call    blineal

        mov     [pos], ax         ; pos = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** PROCEDIMIENTOS ************************************

;*************************************************************
; BLINEAL
;
; Este procedimiento utiliza la instrucción de cadenas
; scasw y el prefijo de repetición de instrucciones repne
; para encontrar la primera ocurrencia de llave dentro del
; arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        DI = datos
;        CX = nDatos
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario.
;
; El pseudocódigo de este procedimiento es:
;
;   int blineal(int llave, int *datos, int ndatos)
;   {
;      n = nDatos
;
;      while(n-- > 0 && llave != *(datos++));
;
;      if(llave == *(datos-1) goto @@siesta
;
;      return –1
```

```
;
;   @@siesta:
;       return nDatos – (n+1)
;   }
;*************************************************************
proc    blineal
        push    cx                  ; Preserva CX

        cld                         ; Autoincrementa DI
        repne   scasw               ; while(CX-- > 0 &&
                                    ;      llave != [DI++]);
        cmp     ax, [di-2]          ; if(llave==datos[DI-2])
        je      @@siesta            ;    goto @@siesta

        pop     cx                  ; Restaura CX
        mov     ax, 0FFFFh          ; AX = 0FFFFh
        ret

@@siesta:
        pop     ax                  ; AX = nDatos
        sub     ax, cx              ; AX -= CX + 1
        dec     ax
        ret
endp    blineal

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

2. El siguiente procedimiento regresa la longitud de una cadena terminada en 0.

```
;*************************************************************
; ASTRLEN
;
; Este procedimiento regresa la longitud de una cadena
; terminada en 0.
;
; Parámetros:
;
;        SI = cadena
;
; Regresa:
;
;        CX = strlen(cadena)
;
;
; El pseudocódigo de este procedimiento es:
;
;   int astrlen(char *cadena)
;   {
;      p = cadena
```

```
;
;       while(*p++);
;
;       return p – cadena - 1
;   }
;*************************************************************
proc    astrlen
        push   ax                   ; Preserva AX, DI
        push   di

        mov     di, si              ; DI = SI
        xor     al, al              ; AL = 0

        cld                         ; Autoincrementa DI
@@whi:  scasb                       ; while([DI++]);
        jnz     @@whi

        mov     cx, di              ; CX = DI - SI – 1
        sub     cx, si
        dec     cx

        popr    di                  ; Restaura DI, AX
        pop     ax
        ret
endp    astrlen
```

3. El siguiente procedimiento convierte una cadena terminada en 0 a mayúsculas.

```
;*************************************************************
; ASTRUPR
;
; Este procedimiento convierte una cadena terminada en 0 a
; mayúsculas.
;
; Parámetros:
;
;       SI = cadena
;
; Regresa:
;
;       SI = scadena
;
; El pseudocódigo de este procedimiento es:
;
;
;   char *astrupr(char *cadena)
;   {
;      n = astrlen(cadena)
;      if(¡n) goto @@fin
;
;      p = q = cadena
```

```
;       do
;       {
;          ch = *(p++)
;          if(ch >= ‘a’ && ch <= ‘z’) toupper(ch)
;          *(q++) = ch
;       }
;       while(--n > 0)
;
;       return cadena
;    }
;*************************************************************
proc    astrupr
        push    ax                  ; Preserva AX, CX, SI, DI
        push    cx
        push    si
        push    di

        call    astrlen             ; CX = strlen(cadena)

        jcxz    @@fin               ; if(!CX) goto @@fin

        mov     di, si              ; DI = SI

        cld                         ; Autoincrementa SI, DI
@@do:                               ; do
                                    ; {
        lodsb                       ;    AL = [SI++]
        cmp     al, 'a'             ;    if(AL < 'a' ||
        jb      @@sig               ;       AL > 'z')
        cmp     al, 'z'
        ja      @@sig
        sub     al, 'a'-'A'         ;    AL = toupper(AL)

@@sig:  stosb                       ;    [DI++] = AL
        loop    @@do                ; }
                                    ; while(--CX > 0)
@@fin:  pop     di                  ; Restaura DI, SI, CX, AX
        pop     si
        pop     cx
        pop     ax
        ret
endp    astrupr
```

4. El siguiente procedimiento convierte una cadena terminada en 0 que representa un entero con signo a binario.

```
;*************************************************************
; AATOI
;
; Este procedimiento convierte una cadena terminada en 0,
; que representa un número de tipo palabra con signo a
```

```
; binario. El signo si existe debe ser el primer carácter
; de la cadena. La cadena puede terminar en d, b, h
; indicando la base del número. No se permiten espacios en
; la cadena.
;
; Parámetros:
;
;       SI = cadena con el número
;
; Regresa:
;
;       AX = número en binario
;
; El pseudocódigo del procedimiento es:
;
;     int aatoi(char *s)
;     {
;        astrup(cadena)
;        l = astrlen(s)
;        signo = obtenSigno(&s, &l)
;        base = obtenBase(s, &l)
;        n = atou(s, base, l)
;        if(signo) n *= -1
;        return n
;      }
;*************************************************************
***
proc    aatoi
        push    bx              ; Preserva BX, CX, DX, SI
        push    cx
        push    dx
        push    si

        call    astrupr         ; strup(cadena)
        call    astrlen         ; CX = strlen(cadena)
        call    obtenSigno      ; DX = [SI] == '-',
                                ; SI++, CX--
        call    obtenBase       ; BX = base, CX--
        call    atou            ; AX = atou(cadena)
        cmp     dx, 0           ; if(dx == 0)
        je      @@sig           ;    goto @@sig
        neg     ax              ; ax = -ax

@@sig:  pop     si              ; Restaura SI, DX, CX, BX
        pop     dx
        pop     cx
        pop     bx
        ret
endp    aatoi
```

```
;*************************************************************
; OBTENSIGNO
;
; Este procedimiento que solo debe llamarlo aatoi, lee el
; primer carácter de una cadena que representa un número de
; tipo palabra con signo y determina si este carácter
; representa el signo del número.
;
; Parámetros:
;
;       SI = cadena con el número
;       CX = Longitud de la cadena
;
; Regresa:
;
;       CX : if([si] == '+' || [si] == '-') CX--
;       DX = [si] == '-'
;       SI : if([si] == '+' || [si] == '-') SI++
;
; El pseudocódigo de este procedimiento es:
;
;   int obtenSigno(char **s, int *l)
;   {
;      signo = 0
;      if(**s == ‘+’) goto @@pos
;      if(**s == ‘-’) goto @@neg
;      goto @@fin
;
;   @@neg:
;      signo = 1
;   @@pos:
;      (*s)++
;      *l--
;
;      return signo
;   }
;*************************************************************
proc    obtenSigno
        xor     dx, dx              ; dx = 0
        cmp     [byte si], '+'      ; if([si] == '+')
        je      @@pos               ;    goto @@pos
        cmp     [byte si], '-'      ; if([si] == '-')
        je      @@neg               ;    goto @@neg
        jmp     @@fin               ; goto @@fin

@@neg:  mov     dx, 1               ; Dx = 1
@@pos:  inc     si                  ; SI++
        dec     cx                  ; CX--
@@fin:  ret
endp    obtenSigno
```

```
;*************************************************************
; OBTENBASE
;
; Este procedimiento que solo debe llamarlo aatoi, lee el
; último carácter de una cadena que representa un número de
; tipo palabra sin signo y determina la base en que está
; representada el número. Por omisión la base es 10.
;
; Parámetros:
;
;       SI = cadena con el número
;       CX = Longitud de la cadena
;
; Regresa:
;
;       BX : if([si+cx-1] == 'B') BX = 2
;            else if([si+cx-1] == 'H') BX = 16
;            else BX = 10
;       CX : if([si+cx-1] == 'B' || [si+cx-1] == 'H' ||
;               [si+cx-1] == 'D') CX–
;
; El pseudocódigo de este procedimiento es:
;
;   int obtenBase(char *s, int *l)
;   {
;      p = s + astrlen(p) –1
;      base = 10
;      if(*p == ‘B’) base = goto @@bin
;      if(*p == ‘H’) base = goto @@hex
;      if(*p == ‘D’) base = goto @@dec
;      goto @@fin
;
;   @@bin:
;      base = 2
;      goto @@dec
;   @@hex:
;      base = 16
;   @@dec:
;      *l–
;   @@fin:
;      return base
;   }
;*************************************************************
proc    obtenBase
        push    si              ; Preserva SI
        add     si, cx          ; SI = cadena + strlen(
        dec     si              ;      cadena) – 1

        mov     bx, 10          ; base = 10


        cmp     [byte si], 'B'  ; if([si] == 'B')
        je      @@bin           ;    goto @@bin
```

```
        cmp     [byte si], 'H'    ; if([si] == 'H')
        je      @@hex             ;    goto @@hex
        cmp     [byte si], 'D'    ; if([si] == 'D')
        je      @@dec             ;    goto @@dec
        jmp     @@fin             ; goto @@fin

@@bin:  mov     bx, 2             ; base = 2
        jmp     @@dec             ; goto @@dec
@@hex:  mov     bx,16             ; Base = 16
@@dec:  dec     cx                ; CX--
@@fin:
        pop     si                ; Restaura SI
        ret
endp    obtenBase

;*************************************************************
; ATOU
;
; Este procedimiento que solo debe llamarlo aatoi,
; convierte una cadena que representa un número de tipo
; palabra sin signo a binario.
;
; Parámetros:
;
;       SI = cadena con el número
;       BX = 2, 10, 16, base del número
;       CX = strlen(cadena)
;
; Regresa:
;
;        AX = número en binario
;
; El pseudocódigo de este procedimiento es:
;
;   int atou (char *s, int base, int l)
;   {
;      n = 0
;      if(¡l) goto @@fin
;
;      do
;      {
;         n = base*n + valC(*s)
;         s++
;      }
;      while(--l > 0)
;      return n
;   }
;*************************************************************
proc    atou
        push   dx                 ; Preserva DX, DI
        push   di

        xor     ax, ax            ; n = 0
```

```
        jcxz    @@fin               ; if(!CX) goto @@fin

        xor     di, di              ; n = 0
@@do:                               ; do
                                    ; {
        mov     ax, di              ;    AX = base*n
        mul     bx
        mov     dl, [byte si]       ;    DX = [SI]
        xor     dh, dh
        call    valC                ;    DX = val([SI])
        add     ax, dx              ;    AX = base*n + DX
        mov     di, ax              ;    n = AX
        inc     si                  ;    SI++
        loop    @@do                ; }
                                    ; while(--CX > 0)

        mov     ax, di
@@fin:  pop     di                  ; Restaura DI, DX
        pop     dx
        ret
endp    atou

;*******************************************************
; VALC
;
; Este procedimiento que solo debe llamarlo atou, convierte
; un carácter que representa un número a su valor en
; binario.
;
; Parámetros:
;
;        DX = carácter
;
; Regresa:
;
;        DX = número
;
; El pseudocódigo de este procedimiento es:
;
;   int valC(char ch)
;   {
;      if(ch > '9') goto @@hex
;      return ch - '0'
;
;   @@hex:
;      return ch – ('A'- 10)
;   }
;*******************************************************
***

proc    valC
        cmp     dx, '9'
        ja      @@hex
```

```
        sub     dx, '0'
        ret

@@hex:  sub     dx, 'A' – 10
        ret
endp    valC
```

Ejercicios sobre instrucciones de cadenas

1. Crea un procedimiento llamado **astrcat** que concatena dos cadenas. El procedimiento recibe como parámetros las direcciones de las cadenas en los registros **SI** y **DI** y concatena la cadena apuntada por **SI** a la cadena apuntada por **DI**. Los registros **SI** y **DI** deben quedar sin modificación.

2. Crea un procedimiento llamado **astrrev** que invierta una cadena. El procedimiento recibe como parámetro la dirección de la cadena en el registro **SI**. El registro **SI** debe quedar sin modificación.

3. Crea un procedimiento llamado **aitoa** que convierta un número binario a una cadena con su representación en cualquiera de las tres bases: binario, decimal y hexadecimal. El procedimiento recibe como parámetros el número a convertir en **AX**, la base en que se deberá convertir el número en **BX** y la dirección de la cadena en donde quedará el número convertido en **SI**. El registro **SI** debe quedar sin modificación.

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documento PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

Problemas

1. Crea un programa que encuentre la primera ocurrencia de un dato dentro de un arreglo de datos de tipo palabra utilizando el algoritmo de búsqueda binaria.

   El programa estará formado de dos módulos:

   - El primer módulo llamado **DEMO_OBB** contiene el programa principal con las variables del programa: **datos** que contiene el arreglo de datos; **nDatos** que contiene el número de datos en el arreglo; **llave** que contiene el dato a buscar y **pos** donde queda la posición de la primera ocurrencia del valor buscado.

   - El segundo módulo llamado **ORD_BBIN** contiene dos procedimientos: El procedimiento para ordenar los elementos de un arreglo visto en el ejercicio sobre arreglos y procedimientos. El segundo procedimiento implementará el algoritmo de búsqueda binaria. El procedimiento llamado **bbinaria** recibe como parámetros el valor de **llave** en **AX**, la dirección del arreglo en **BX**, el valor de **nDatos** en **CX**. El procedimiento regresa en **AX** la posición de la primera ocurrencia de llave en el arreglo, -1 (0FFFFh) en caso contrario.

2. Crea un programa que realice las cuatro operaciones fundamentales con dos datos de tipo palabra sin signo. El programa recibe los datos y la operación en una cadena llamada **soper**, por ejemplo:

   "36h + 156d"

   El resultado deberá quedar en otra cadena llamada **sresul** expresado en las tres bases, por ejemplo:

   "210d = 11010010b = D2h"

   Todas las cadenas en este programa son cadenas terminadas en 0.

   El programa estará formado por tres módulos:

   El primer módulo llamado **CALCULA** contiene tres procedimientos:

   - El procedimiento principal del programa.

   - Un procedimiento llamado **sscan** que obtiene de la cadena **soper** los dos datos y el carácter que representa la operación. El procedimiento recibe como parámetro la dirección de la cadena **soper** en el registro **SI** y regresa el primer dato en el registro **AX**, el segundo dato en el registro **BX** y el carácter que indica la operación en el registro **CX**.

   - Un procedimiento llamado **sprint** que recibe el resultado de la operación en binario y forma la cadena con la representación del resultado de la operación en decimal, binario y

hexadecimal. El procedimiento recibe como parámetros el resultado de la operación en el registro **AX** y la dirección de la cadena **sresul** en el registro **SI**.

El segundo módulo llamado **ASCII_N**, contiene los procedimientos **aatoi** y **aitoa** vistos anteriormente.

El tercer módulo llamado **STRING** contiene los procedimientos: **astrlen**, **astrup**, **astrcat** y **astrrev** vistos anteriormente.

CAPÍTULO 8

INTERRUPCIONES

Interrupciones Por Hardware

Durante la ejecución de un programa, el procesador debe interactuar con una serie de dispositivos de entrada/salida: El teclado, las unidades de disco, los puertos serie y paralelo, la tarjeta de red, etc. Normalmente, la velocidad de procesamiento de estos dispositivos es inferior a la del procesador. Así que el procesador tendrá que esperar un tiempo para que el dispositivo se desocupe antes de hacerle una nueva solicitud. ¿Cómo sabe el procesador que el dispositivo ya está listo para la nueva petición?. También, algunos dispositivos requieren la atención del procesador en forma asíncrona, es decir, no en forma periódica. ¿Cómo sabe el procesador que el dispositivo requiere que se le atienda? Hay dos formas en que el procesador puede saber si el dispositivo está listo o si el dispositivo requiere atención: La primera forma es que el procesador esté periódicamente sondeando al dispositivo en cuestión. A fin de asegurarse de poder atender rápidamente a un dispositivo cuando se lo solicita, la frecuencia con el que procesador **sondea** a los dispositivos debe ser muy alta. Lo anterior produce que el procesador ocupa mucho tiempo interrogando a los dispositivos y en la mayoría de las veces éstos no están listos o requieren de la atención del procesador. La segunda forma, una más eficiente, es hacer que sea el dispositivo el que le avise al procesador cuando esté listo o requiera atención, interrumpiendo al procesador. El procesador dejará de hacer lo que está haciendo para atender al dispositivo para posteriormente regresar a la tarea que estaba ejecutando antes de ser interrumpido. Los dispositivos utilizan una de las líneas del procesador para interrumpirlo. A este tipo de interrupción se le conoce como **Interrupción por Hardware.** Para atender a un dispositivo el procesador ejecuta una rutina especial llamada **Rutina de Servicio a Interrupciones.** Las rutinas de servicio a interrupciones, por lo general, se cargan a la memoria RAM como parte del proceso de iniciación de la computadora.

Las interrupciones de cada dispositivo están priorizadas de tal manera que si dos dispositivos desean interrumpir al procesador al mismo tiempo, sólo la de mayor prioridad interrumpirá al procesador.

Interrupciones Por Software

Un programa escrito en un lenguaje de alto nivel utiliza funciones de la biblioteca estándar, provistas por el compilador empleado, para realizar la mayoría de las tareas relacionadas con el manejo de los dispositivos de entrada y salida, y la gestión de los archivos y memoria. La mayoría de dichas funciones hacen su trabajo mediante **llamadas al sistema**. Una llamada al sistema invoca a una rutina provista

por el sistema operativo para realizar una tarea. En algunas ocasiones la función de biblioteca accesa directamente al hardware para realizar su tarea si es que no existe una rutina del sistema operativo o el desempeño de ésta no es adecuado.

Si el programa escrito en un lenguaje de alto nivel necesita realizar una tarea para la cual no existe una función de biblioteca o si el desempeño de ésta no es adecuado, entonces puede ejecutar una llamada al sistema operativo para realizar dicha tarea. De igual manera, si el sistema operativo no provee de una rutina para ejecutar esa tarea o si el desempeño de ésta no es adecuado, entonces el programa puede accesar directamente al hardware.

Un programa escrito en ensamblador, normalmente no tiene a su disposición una biblioteca de funciones estándar y por lo tanto para realizar las tareas relacionadas con el manejo de los dispositivos de entrada y salida, y la gestión de los archivos y memoria deberá hacerlo mediante llamadas al sistema. También puede accesar directamente al hardware para realizar su tarea si es que no existe una rutina del sistema operativo o el desempeño de ésta no es adecuado.

En la figura 8-1 se muestra como un programa puede acceder a las funciones de biblioteca, rutinas del sistema operativo y hardware para realizar una tarea que implica el manejo del hardware. Las flechas representan las llamadas a funciones de la biblioteca, llamadas al sistema o acceso directo al hardware.

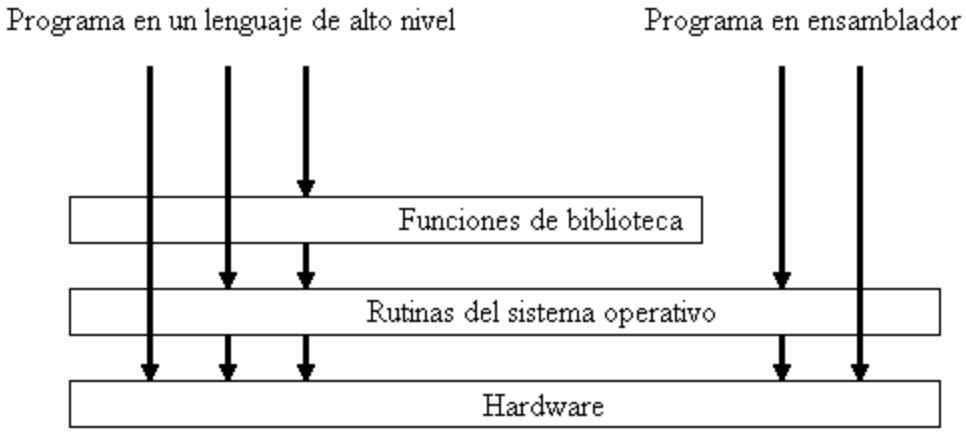


Figura 8-1

Cuando un programa ejecuta una llamada al sistema interrumpe al procesador. El procesador dejará de hacer lo que está haciendo para ejecutar la rutina del sistema deseada y posteriormente regresará a la tarea que estaba ejecutando antes de ser interrumpido. La estructura de una rutina del sistema es igual a la estructura de una rutina de servicio a interrupciones. Una llamada al sistema se genera mediante una instrucción del ensamblador llamada **Interrupción por Software**. Dado que el procesador responde en forma similar tanto las interrupciones por hardware como las interrupciones por software podemos simular una interrupción por hardware mediante una interrupción por software con el fin de depurar las rutinas de servicio a interrupciones.
Las rutinas de servicio a interrupciones y las rutinas del sistema son en esencia procedimientos, y difieren de los procedimientos normales en el mecanismo empleado para llamarlas, en el mecanismo de

retorno y en ciertas consideraciones que hay que tomar en cuenta si deseamos que el procesador acepte una interrupción mientras se está ejecutando una rutina de servicio a interrupción.

Rutinas Del Sistema Operativo MSDOS

Uno de los sistemas operativos que utilizan las computadoras personales IBM y compatibles es el sistema operativo MSDOS de la compañía Microsoft Corp. Dichas computadoras utilizan un procesador de la familia 80x86 de la compañía Intel Corporation o un procesador compatible con él. Las rutinas de las llamadas al sistema operativo MSDOS se encuentran divididas en dos capas: Las **rutinas del BIOS** y las **rutinas del DOS**.

Rutinas del BIOS

Las rutinas del BIOS (Basic Input/Output System) son específicas a cada computadora y las proporciona el fabricante del sistema. Contienen los manejadores por defecto para los siguientes dispositivos:

- Monitor y teclado
- Puerto serie
- Dispositivo auxiliar
- Reloj de tiempo real
- Unidades de disco

Las porciones más primitivas de estos manejadores se encuentran en memoria de sólo lectura (ROM) para que puedan ser utilizadas por las rutinas de diagnóstico y arranque.

Las rutinas del BIOS son cargadas a memoria RAM durante el proceso de iniciación del sistema y se encuentran en un archivo llamado IO.SYS o IBMBIO.COM.

Rutinas del DOS

Las rutinas del DOS (Disk Operating System), llamado también el núcleo del DOS, es un programa propietario de la compañía Microsoft Corp. Son una colección de rutinas independientes del hardware que incluyen los siguientes servicios:

- Gestión de archivos
- Gestión de la memoria
- Entrada y salida
- Ejecución de programas
- Acceso al reloj de tiempo real

Las rutinas del DOS utilizan a las rutinas del BIOS para hacer su trabajo y en algunas ocasiones proveen el mismo servicio que éstas. En la figura 8-2 se muestra como interactúan las funciones de biblioteca, las rutinas del BIOS y las rutinas del DOS.

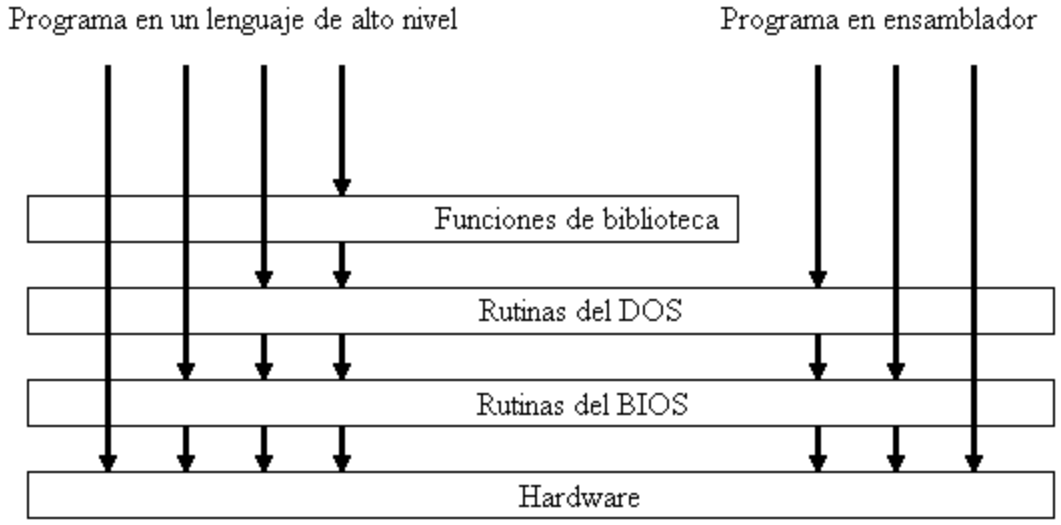


Figura 8-2

Las rutinas del DOS son cargadas a memoria RAM durante el proceso de iniciación del sistema y se encuentran en un archivo llamado MSDOS.SYS o IBMDOS.COM.

Vectores De Interrupción

En las secciones anteriores vimos que tanto las rutinas de servicio a interrupciones como las rutinas del sistema (BIOS y DOS) se cargan en la memoria RAM durante la inicialización del sistema, aunque también un programa puede instalar sus propias rutinas de servicio a interrupciones o modificar una ya existente.

Debido a la diversidad de dispositivos de entrada/salida y a las diferentes versiones de un sistema operativo, la posición en la memoria de las rutinas de servicio a interrupciones y de las rutinas del sistema cambia de una computadora a otra. A fin de que un programa pueda llamarlas, las direcciones de entrada a esas rutinas se almacenan, en una tabla llamada **Vectores de Interrupción**. La iniciación de esta tabla ocurre durante el proceso de arranque aunque también un programa puede modificar una entrada de esa tabla para que apunte a una rutina que el programa haya instalado. Cada vector de interrupción es una palabra doble que contiene la dirección, segmento:desplazamiento, donde inicia el código de la rutina de servicio de interrupción o rutina del sistema que será invocada cuando ocurra la interrupción.

Toda interrupción ya sea por hardware o software tiene asociado un número entero llamado **Número de Interrupción**. Cuando ocurre una interrupción, el procesador utiliza ese número de interrupción como un índice a la tabla de vectores de interrupción. En las computadoras personales IBM o

compatibles con un sistema operativo MSDOS, la tabla de vectores de interrupción ocupa los primeros 1000 bytes de la memoria: De la dirección 0000:0000 a la dirección 0000:0400, tal como se muestra en la figura 8-3.

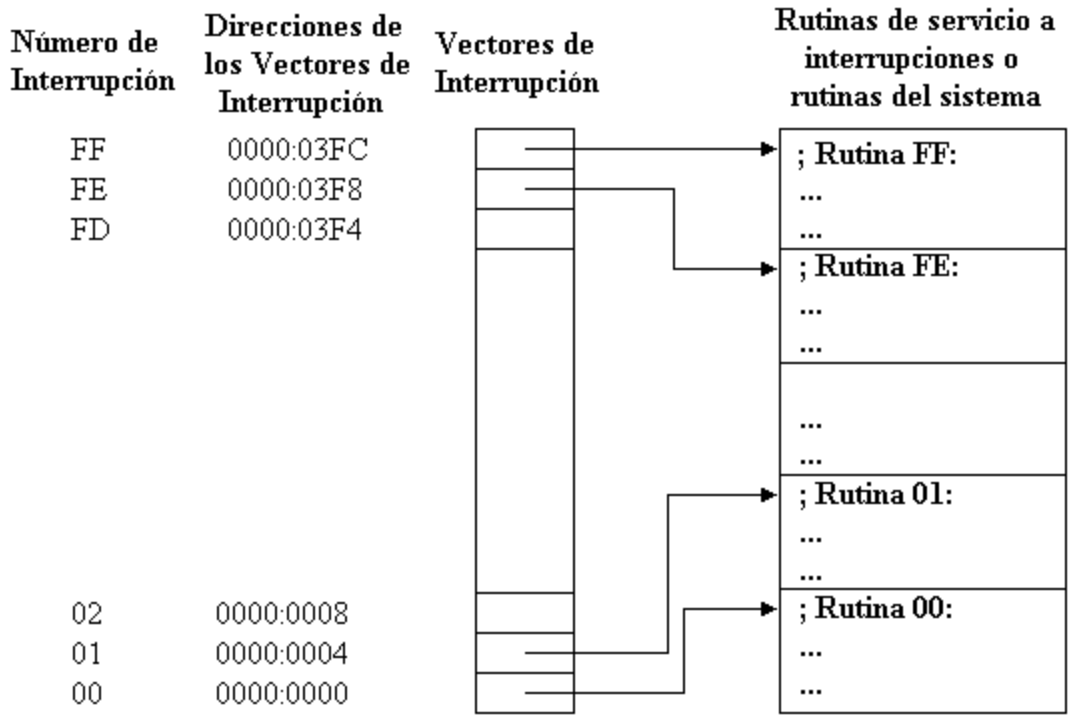


Figura 8-3

En la tabla 8.1 se lista para cada vector de interrupción su número de interrupción, la dirección en que se encuentra y el propósito de la rutina de servicio a interrupción o del sistema, para una computadora personal IBM o compatible con un sistema operativo MSDOS.

La mayoría de las rutinas del sistema están, cada una de ellas, formadas por un grupo de subrutinas llamadas **funciones** e inclusive, en algunos casos, las funciones están formadas por subfunciones. Tanto las funciones como sus subfunciones están numeradas. El vector asociado a la rutina apunta al inicio de este grupo de funciones. Antes de hacer la llamada al sistema se selecciona función y en su caso la subfunción cargando en los registros **AH** y **AL** los números de la función y subfunción deseadas.

Llamadas Al Sistema

Para llamar a una rutina del sistema se utiliza la instrucción llamada Interrupción por Software. La sintaxis de esta instrucción es:

int *numInt*

Tabla 8.1 Vectores de Interrupción

Número de Interrupción	Localización del vector	Propósito
000h	0000h	División por cero
001h	0004h	Ejecución paso a paso
002h	0008h	Interrupción no enmascarable
003h	000Ch	Punto de ruptura
004h	0010h	Sobreflujo
005h	0014h	Impresión de pantalla (ROM BIOS)
006h	0018h	*
007h	001Ch	*
008h	0020h	Temporizador (IRQ0)
009h	0024h	Teclado (IRQ1)
00Ah	0028h	* (IRQ2)
00Bh	002Ch	Entrada/Salida del Puerto serie COM2 (IRQ3)
00Ch	0030h	Entrada/Salida del Puerto serie COM1 (IRQ4)
00Dh	0034h	Disco duro (IRQ5)
00Eh	0038h	Disco flexible (IRQ6)
00Fh	003Ch	Puerto paralelo (IRQ7)
010h	0040h	Manejador de Video (ROM BIOS)
011h	0044h	Verificación de equipo (ROM BIOS)
012h	0048h	Verificación de memoria (ROM BIOS)
013h	004Ch	Manejador de Disco (ROM BIOS)
014h	0050h	Manejador del puerto serie (ROM BIOS)
015h	0054h	Manejador de cassette (ROM BIOS)
016h	0058h	Manejador del teclado (ROM BIOS)
017h	005Ch	Manejador del puerto paralelo (ROM BIOS)
018h	0060h	Basic en ROM

* Reservados o no en uso.
† No es una rutina de servicio de interrupción.
Las interrupciones sombreadas son interrupciones por hardware. Las demás son por software

Tabla 8.1 Vectores de Interrupción (Continuación).

Número de Interrupción	Localización del vector	Propósito
019h	0064h	Rutina de encendido (ROM BIOS Bootstrap)
01Ah	0068h	Hora del día (ROM BIOS)
01Bh	006Ch	Ctrl-Break del teclado (ROM BIOS)
01Ch	0070h	Temporizador (ROM BIOS)
01Dh	0074h	Tabla de parámetros del video (ROM BIOS)
01Eh	0078h	Parámetros del disco flexible (ROM BIOS)†
01Fh	007Ch	Tabla de caracteres 80h - FFh (ROM BIOS)†
020h – 03Fh	0080h - 00FCh	Reservadas para el DOS
040h – 06Fh	0100h - 01BCh	Varios
070h	01C0h	Reloj de tiempo real (IRQ8)
071h	01C4h	* (IRQ9)
072h	01C8h	* (IRQ10)
073h	01CCh	* (IRQ11)
074h	01D0h	* (IRQ12)
075h	01D4h	Coprocesador numérico (IRQ13)
076h	01D8h	Disco duro (IRQ14)
077h	01DCh	* (IRQ15)
078h – 0FFh	01E0h - 03FCh	Varios

* Reservados o no en uso.
† No es una rutina de servicio de interrupción.
Las interrupciones sombreadas son interrupciones por hardware. Las demás son por software

donde *numInt* es el número de interrupción asociada a la rutina del sistema deseada. Como ya se mencionó anteriormente, podemos utilizar la instrucción **int** para simular una interrupción por hardware. En este caso, *numInt* es el número de interrupción por hardware que deseamos simular. Las interrupciones por software no pueden deshabilitarse por lo que podemos utilizar instrucción **int** para simular una interrupción por hardware aún cuando la bandera de interrupción este deshabilitada (su valor sea cero).

El procedimiento para hacer una llamada a una rutina del sistema es el siguiente:

1. Cargar, si es necesario, en los registros **AH** y **AL** los números de las función y subfunción deseados.

2. Cargar, si es necesario, en los registros requeridos los parámetros requeridos por la rutina.

3. Llamar a la rutina usando la interrupción por software.

4. Si la rutina genera resultados, inspeccionar los registros, banderas o localidades de memoria en que se encuentran éstos.

Antes de iniciar la ejecución de rutina del sistema (o de una rutina de servicio a interrupción) el procesador guarda en la pila el registro de banderas y la dirección de regreso. Adicionalmente, el procesador pone en cero las banderas de interrupción, I, y de trampa, T. Antes de regresar de la rutina del sistema (o de una rutina de servicio a interrupción) el procesador extrae de la pila el registro de banderas dejando las banderas en las condiciones previas a la interrupción.

Algunas Rutinas del BIOS

Despliegue de Caracteres

En la tabla 8.1 podemos ver que la interrupción 10h es la interrupción por software cuyo vector de interrupción apunta al **Manejador de Video**. El manejador de video es un conjunto de funciones que nos permiten manejar la tarjeta de video de la computadora. Esas funciones forman parte del BIOS que se encuentra en memoria ROM y nos permiten seleccionar y obtener el modo de video, seleccionar el tamaño del cursor, posicionar el cursor, obtener la posición del cursor, escribir un carácter en la pantalla, dibujar un pixel, seleccionar la paleta de colores, el tipo de fuente, etc. La función 0Eh del manejador de video nos permite desplegar un carácter en el modo teletipo. El modo teletipo es la forma más primitiva de desplegar texto en la pantalla. Para desplegar un carácter debemos hacer lo siguiente:

1. Cargar los siguientes registros:

   AH = 0Eh
   AL = Carácter a desplegar
   BH = Página

2. Llamar a la interrupción 10h.

- La función reconoce los siguientes códigos ASCII como caracteres de control: 07h (alarma), 08h (borrado), 0Dh (retorno de carro) y 0Ah (salto de línea). El resto de los caracteres son enviados a la pantalla aún cuando sean caracteres de control. El cursor avanza a la siguiente posición.

- En los modos de video que soportan páginas múltiples, podemos seleccionar en que página queremos escribir sin importar cual página es la que se está desplegando actualmente. Sólo se despliega una página a la vez, las otras están ocultas. Otra de las funciones del manejador de vídeo nos permite cambiar de página. Las páginas están numeradas: 0, 1, .., n. Por defecto la página desplegada es la 0.

Lectura de un carácter del teclado

En la tabla 8.1 podemos ver que la interrupción 16h nos permite accesar al **Manejador del Teclado**, que es un conjunto de funciones que nos permiten manejar el teclado de la computadora. Esas funciones también forman parte del BIOS que se encuentra en memoria ROM y nos permiten leer un carácter del teclado, obtener el estado y las banderas del teclado. La función 10h de este manejador nos permite leer un carácter del teclado. Para leer un carácter del teclado debemos hacer lo siguiente:

1. Cargar el siguiente registro:

   AH = 10h

2. Llamar a la interrupción 16h.

3. El código ASCII del carácter tecleado y el código de la tecla presionada están en los registros:

   AH = Código de la tecla presionada.
   AL = Código ASCII del carácter tecleado.

- La función no despliega en la pantalla el carácter leído. Si se quiere que aparezca en la pantalla de puede usar la función 0Eh del manejador de video.

Ejemplo sobre Rutinas del BIOS

El siguiente listado que puede almacenarse en un archivo llamado BIOS_MAC.INC contiene macros para desplegar un carácter en la pantalla y para leer un carácter del teclado.

```
;*************************************************************
; BIOS_MAC.ASM
;
; Este archivo contiene macros permiten desplegar un carácter
; a pantalla y leer un carácter del teclado. Las macros
utilizan
; llamadas a las rutinas del BIOS
;*************************************************************

;*************************************************************
; APUTCH
;
; Esta macro despliega un carácter en la pantalla
;
; Parámetro:
;
;       AL = carácter
;
```

```
; Regresa:
;
;       Nada
;*************************************************************
macro   aputch
        mov     ah, 0Eh         ;; Despliega un carácter
        mov     bh, 0           ;; en la página 0
        int     10h             ;; Interrupción de video
endm    aputch


;*************************************************************
; AGETCH
;
; Esta macro lee un carácter del teclado
;
; Parámetro:
;
;       Nada
;
;
;
;
; Regresa:
;
;       AH = Código de la tecla
;       AL = Código ASCII del carácter
;*************************************************************
macro   agetch
        mov     ah, 10h         ;; Leer un carácter
        int     16h             ;; Interrupción del teclado
endm    agetch
```

Ejercicios sobre Rutinas del BIOS

1. Escriba un procedimiento llamado **aputs** que despliegue en la pantalla una cadena terminada en cero. El procedimiento recibe como parámetro la dirección de la cadena en el registro **SI**. El registro **SI** debe quedar sin modificación. El procedimiento debe llamar a la macro **aputch.**

2. Escriba un procedimiento llamado **agets** que lea del teclado una cadena. La cadena deberá irse desplegando en la pantalla conforme se vaya tecleando. La lectura de la cadena termina cuando se presiona la tecla [Enter]. El carácter de retorno de carro (el carácter que se obtiene cuando se presiona la tecla [Enter] no forma parte de la cadena. La cadena deberá terminar en cero. El procedimiento recibe como parámetro la dirección de la cadena en el registro **DI**. El registro **DI** debe quedar sin modificación. El procedimiento debe llamar a la macro **agetch.**

Algunas Rutinas del DOS

En la tabla 8.1 podemos ver que las interrupciones 020h a 03Fh están reservadas para las rutinas del DOS. No todas las interrupciones están ocupadas. La mayoría de las funciones del DOS están agrupadas en la interrupción 21h.

Despliegue de Caracteres

La función 02h de la interrupción 21h del DOS nos permite desplegar un carácter en el dispositivo estándar de salida. Para desplegar un carácter usando esta función debemos hacer lo siguiente:

1. Cargar los siguientes registros:

   AH = 02h
   DL = Carácter a desplegar

2. Llamar a la interrupción 21h.

- La salida estándar puede ser redirigida.

Lectura de un carácter del teclado

La función 01h de la interrupción 21h del DOS nos permite leer un carácter del dispositivo estándar de entrada (el teclado). Para leer un carácter del teclado usando esta función debemos hacer lo siguiente:

1. Cargar el siguiente registro:

   AH = 01h

2. Llamar a la interrupción 21h.

3. El código ASCII del carácter tecleado está en el registro:

   AL = Código ASCII del carácter tecleado.

- La función despliega en el dispositivo estándar de salida (la pantalla) el carácter leído.

- Si la tecla presionada es una tecla especial el valor en AL es cero. Para obtener el código de la tecla presionada debemos volver a llamar a la función. En este caso el código de la tecla presionada está en el registro **AL**.

- La entrada estándar puede ser redirigida.

Termina la ejecución de un proceso

La función 4Ch de la interrupción 21h del DOS termina la ejecución del proceso (programa) actual, regresando el código de salida al proceso padre. Para terminar la ejecución del proceso actual usando esta función debemos hacer lo siguiente:

3. Cargar los siguientes registros:

   AH = 4Ch
   DL = Código de salida

4. Llamar a la interrupción 21h.

Al terminar la ejecución de un proceso, MSDOS hace lo siguiente:

- Libera toda la memoria asignada al proceso
- Cierra todos los archivos o dispositivos que le pertenecen al proceso.

Ejemplo sobre Rutinas del DOS

El siguiente listado que puede almacenarse en un archivo llamado BIOS_MAC.INC contiene macros para desplegar un carácter en la pantalla y para leer un carácter del teclado.

```
;*************************************************************
; DOS_MAC.ASM
;
; Este archivo contiene macros que permiten desplegar un
; carácter a pantalla y leer un carácter del teclado. Las
; macros utilizan llamadas a las rutinas del DOS
;*************************************************************

;*************************************************************
; APUTCHAR
;
; Esta macro despliega un carácter en la pantalla
;
; Parámetro:
;
;       DL = carácter
;
; Regresa:
;
;       Nada
;*************************************************************
```

```
macro   aputchar
        mov     ah, 02h         ;; Despliega un carácter
        int     21h             ;; Interrupción del DOS
endm    aputchar


;*********************************************************
; AGETCHAR
;
; Esta macro lee un carácter del teclado
;
; Parámetro:
;
;       Nada
;
; Regresa:
;
;       AL = Código ASCII del carácter
;*********************************************************
macro   agetchar
        mov     ah, 01h         ;; Lee un carácter
        int     21h             ;; Interrupción del DOS
endm agetchar
```

Ejercicios sobre Rutinas del DOS

1. Modifique el procedimiento **aputs** de los ejercicios sobre rutinas del BIOS para que en lugar de utilizar la macro **aputch** para desplegar un carácter, utilice la macro **aputchar**.

2. Modifique el procedimiento **agets** de los ejercicios sobre rutinas del BIOS para que en lugar de utilizar la macro **agetch** para leer un carácter, utilice la macro **agetchar**.

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Duncan, Ray. Advanced MSDOS. Second Edition. Microsoft Press. U.S.A. 1988.

5. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

6. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documento PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

7. Norton, Peter and Wilton, Richard. The New Peter Norton Programmer´s Guidde to The IBM PC & PS/2. Second Edition. Microsoft Press. U.S.A. 1988.

8. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

Problemas

1. Modifica el módulo **DEMO_OBB** del problema 1 del Tema 7: Arreglos y Cadenas. El módulo **DEMO_OBB** deberá leer del teclado el número de datos **nDatos** y los datos del arreglo **datos**. A continuación el programa deberá entrar en un ciclo en el que pregunte por un dato a buscar en el arreglo y despliegue su posición si se encuentra en el arreglo y un mensaje diciendo que no se encuentra, en caso contrario. A continuación le preguntará al usuario si desea buscar otro dato. El programa terminará cuando el usuario conteste con el carácter 'n'. Renombra a este módulo **DEMO_OB2**.

   Adicionalmente a los módulos del programa original el programa deberá contener los módulos **STRING** y **ASCII_N** desarrollados para el problema 2 del Tema 7: Arreglos y Cadenas y que permiten algunas operaciones para cadenas y convertir de cadenas a enteros y visceversa. También deberá contener el módulo **STR_IO** que contiene los procedimientos **aputs** y **agets** en sus versiones que utilizan a las macros **putch** y **getch** para desplegar y leer caracteres usando las rutinas del BIOS.

2. Modifica el procedimiento principal del módulo **CALCULA** del problema 2 del Tema 7: Arreglos y Cadenas, para que lea la cadena **soper** que contiene los dos datos y la operación y despliegue la cadena **sresul** con el resultado expresado en las tres bases. Renombra a este módulo **CALCULA2**.

   Adicionalmente a los módulos del programa original el programa deberá contener el módulo **STR_IO2** que contiene los procedimientos **aputs** y **agets** en sus versiones que utilizan a las macros **putchar** y **getchar** para desplegar y leer caracteres usando las rutinas del DOS.